

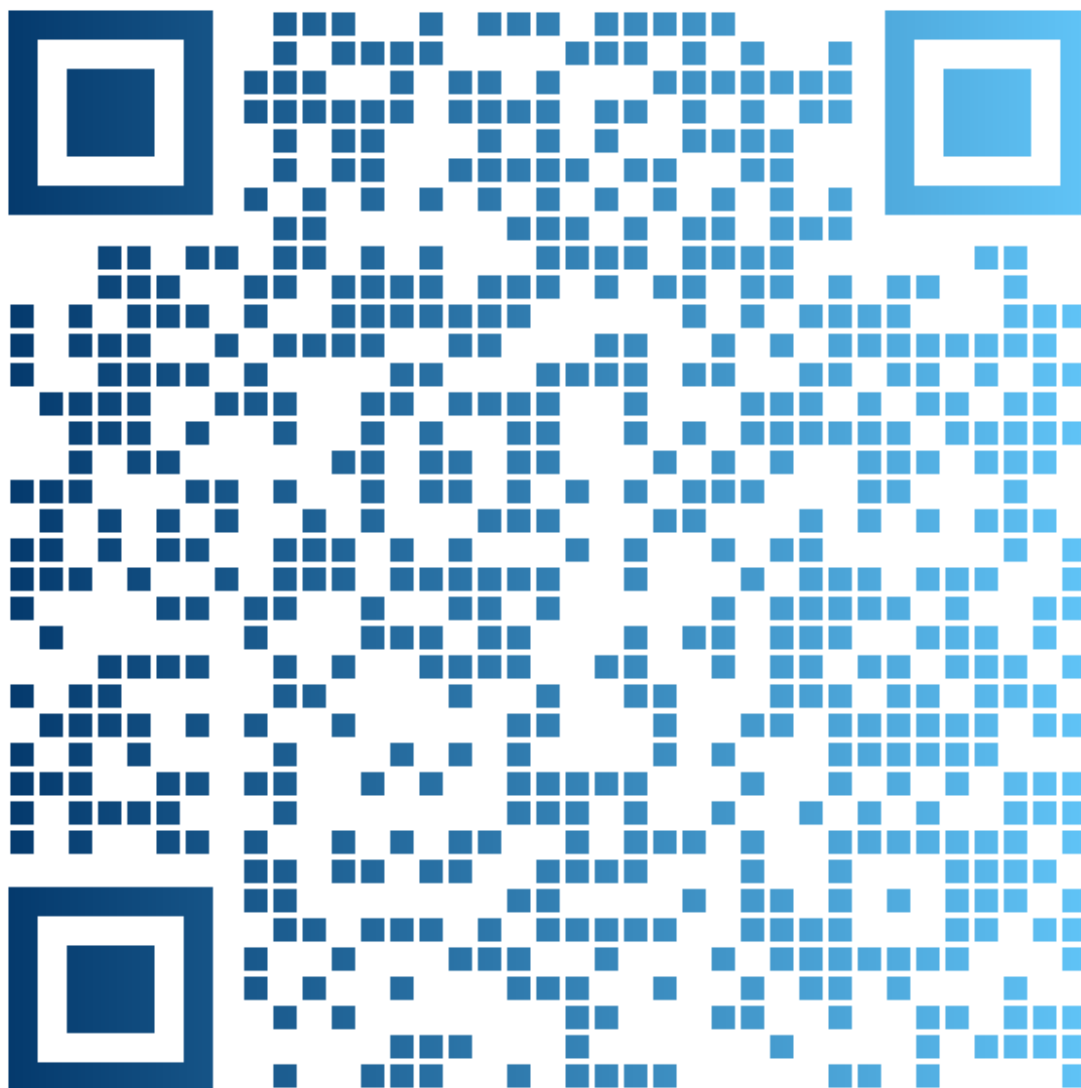


المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Machine Learning

IAOUSSE M'barek

2026



Ce **manuel** a été élaboré pour servir de support de cours principal pour le module *Machine Learning* dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (*Développement d'Applications et Intelligence Artificielle*) en Temps Aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur : M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/).

Autorisations accordées :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Sous réserve de citer l'auteur.

© M'barek IAOUSSE (2026)

Pour citer ce manuel : M'barek IAOUSSE. (2026). *Machine Learning*. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Le présent manuel est le support de cours officiel du module **Machine Learning**, dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (Développement d'Applications et Intelligence Artificielle) en temps aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

À l'origine, la communauté étudiante dans ce programme provient de parcours techniques variés (DUT, BTS, DTS ou diplômes équivalents), ayant déjà acquis un bagage en programmation informatique et les prérequis nécessaires en mathématiques appliquées pour suivre et comprendre les formules de ce manuel.

Le manuel vise à simplifier l'accès aux concepts clés **Machine Learning** du sous forme de ressource éducative libre. Il propose une approche, combinant les bases mathématiques nécessaires, l'ingénierie des données et une transition pratique vers des outils de production.

Prérequis

Le contenu de ce manuel est organisé pour les personnes ayant déjà une base et une expérience avec :

- la programmation en langage Python ;
- les notions élémentaires de mathématiques pour l'informatique (calcul matriciel de base, dérivées, probabilités et Statistique) ;

La familiarité préalable avec le Machine Learning n'est pas un prérequis : les concepts sont réintroduits progressivement depuis leurs fondements mathématiques jusqu'à leur mise en œuvre logicielle.

Livres et ressources complémentaires

Pendant longtemps, l'enseignement du Machine Learning s'est appuyé sur des ouvrages de référence académiques classiques, tels que *Pattern Recognition and Machine Learning* (Bishop 2006) ou *Hands-On Machine Learning* (Géron 2022).

Ce manuel s'inspire largement de leurs approches et de leurs méthodes pour structurer le cours. L'ensemble des applications pratiques et des algorithmes y est développé en utilisant Python comme outil de programmation principal.

Note

Les bibliothèques scientifiques Python évoluant rapidement, veuillez à utiliser un environnement virtuel basé sur Python 3.10 ou supérieur pour garantir la compatibilité des extraits de code présentés dans ce manuel.

Table des matières

Préface	i
Introduction au Machine Learning	1
1. Machine Learning	1
1.1. De l'Intelligence Artificielle au Machine Learning	1
1.2. Définition formelle du Machine Learning	3
2. Programmation classique vs. approche Data-Driven	4
2.1. Le paradigme classique	4
2.2. Le paradigme Data-Driven	4
3. Le cycle de vie d'un projet de Machine Learning	5
4. Python pour le Machine Learning	7
4.1. Quelques bibliothèques scientifiques de l'écosystème Python	7
4.2. Environnements de développement : exploration et production	8
Exploration et prototypage	8
Développement et production	8
4.3. Gestion de projet avec uv	9
Exercices	11
Fondements Théoriques du Machine Learning	13
1. Vocabulaire et représentation des données	13
2. Les grandes familles d'apprentissage	14
Supervisé	14
Non supervisé	15
3. Apprendre et généraliser	15
4. Jeux d'évaluation et Fuite de données	17
4.1. Le cas particulier des données temporelles	17
4.2. La fuite de données (<i>Data Leakage</i>)	18
5. Paramètres et Hyperparamètres	18
6. La Baseline et les limites de l'Accuracy	18
7. Prédiction, Corrélation et Causalité	19
8. Enjeux Éthiques et Reproductibilité	19
Exercices	20
Collecte et Prétraitement des Données	22
1. Stratégies de collecte de données	22
1.1. Les bases de données	22
1.2. Le <i>web scraping</i>	23
1.3. Les API	23
2. Représentativité et biais statistiques	23
3. Nettoyage des données	24
3.1. Les valeurs manquantes	24
3.2. Les valeurs aberrantes (<i>outliers</i>)	25
3.3. Les doublons	25
4. Encodage des variables catégorielles	25
4.1. Encodage ordinal (<i>Ordinal Encoding</i>)	25
4.2. Encodage One-Hot (<i>One-Hot Encoding</i>)	26

4.3. Choisir une méthode simple	26
5. Mise à l'échelle : Standardisation vs Normalisation	27
5.1. La standardisation (<i>Z-score scaling</i>)	27
5.2. La normalisation (<i>Min-Max scaling</i>)	27
5.3. Quand utiliser l'une ou l'autre ?	27
6. Construire un pipeline de prétraitement avec les outils modernes de Python	28
6.1. <code>ColumnTransformer</code> et <code>Pipeline</code> de Scikit-Learn	28
6.2. Mettre à jour l'environnement du projet	29
Exercices	29
Régression	31
1. Régression ou classification ?	31
1.1. Régression	31
1.2. Classification	32
2. Données et notation	32
3. Modèle de régression linéaire	33
3.1. Régression linéaire simple	33
3.2. Régression linéaire multiple	33
3.3. Écriture matricielle	34
3.4. Fonction de coût	34
3.5. Hypothèses et limites	35
4. Calcul des paramètres par les MCO	35
4.1. Principe	35
4.2. Cas de la régression simple	36
4.3. Exemple simple	36
4.4. Solution matricielle	38
5. La descente de gradient	38
5.1. Interprétation du taux d'apprentissage	39
5.2. MCO ou descente de gradient ?	39
6. Évaluer une régression	39
6.1. MAE	39
6.2. MSE	40
6.3. RMSE	40
6.4. Coefficient de détermination R^2	40
6.5. Exemple	41
6.6. Choisir une métrique	41
7. Utilisation avec <code>scikit-learn</code>	41
7.1. Modèle linéaire avec plusieurs variables	42
7.2. Pipeline simple	43
7.3. Limites de l'approche simple	43
8. Exercices théoriques	44
Exercice 1. Identifier le type de problème	44
Exercice 2. Interpréter un coefficient	44
Exercice 3. Calculer une régression simple	44
Exercice 4. Expliquer le résidu	44
Exercice 5. Choisir une métrique	44
Exercice 6. Vérifier les hypothèses	45
Exercice 7. Comprendre le gradient	45
Exercice 8. Comparer les deux approches	45
Classification	46
1. Qu'est-ce que la classification ?	46
1.1. Classification binaire	46
1.2. Classification multiclasse	47

1.3. Classification multilabel	47
2. Formalisation d'un problème de classification	47
3. Régression logistique	48
3.1. Score linéaire	48
3.2. Fonction sigmoïde	48
3.3. Règle de décision	49
3.4. Interprétation des coefficients	50
3.5. Log loss ou entropie croisée binaire	50
3.6. Exemple simple	51
4. Évaluer une classification binaire	51
4.1. Matrice de confusion	52
4.2. Accuracy	53
4.3. Précision	53
4.4. Rappel ou sensibilité	54
4.5. Spécificité	54
4.6. F1-score	55
4.7. Balanced accuracy	56
4.8. Exemple numérique	56
5. Seuil de décision et classes déséquilibrées	58
5.1. Choisir le seuil	58
5.2. Cas de classes déséquilibrées	59
5.3. Bonnes pratiques	59
6. Classification avec <code>scikit-learn</code>	59
6.1. Exemple simple de régression logistique	59
6.2. Matrice de confusion et métriques	60
6.3. Pipeline de prétraitement	60
6.4. Changer le seuil	61
6.5. Validation croisée stratifiée	61
7. En résumé	61
Exercices	62

Apprentissage non supervisé 64

1. Qu'est-ce que l'apprentissage non supervisé?	64
Exemple intuitif	65
2. Préparer les données	65
2.1. Variables numériques et variables qualitatives	65
2.2. Centrage et réduction	66
2.3. Exemple d'échelle	66
3. Analyse en composantes principales (ACP)	66
3.1. Idée géométrique	67
3.2. Matrice centrée	67
3.3. Matrice de covariance	67
3.4. Direction de variance maximale	68
3.5. Valeurs propres et composantes principales	68
3.6. Projection et réduction de dimension	68
3.7. Variance expliquée	69
3.8. Exemple simple	69
3.9. Quand utiliser l'ACP?	70
4. K-Means	70
4.1. Objectif	70
4.2. Affectation des observations	71
4.3. Mise à jour des centroïdes	71
4.4. Algorithme itératif	71
4.5. Exemple simple	72

4.6. Inertie et choix du nombre de clusters	73
4.7. Pourquoi le choix de l'échelle est crucial ?	73
4.8. Limites de K-Means	73
5. Utilisation avec <code>scikit-learn</code>	73
5.1. ACP avec PCA	73
5.2. K-Means avec <code>KMeans</code>	74
5.3. Méthode du coude	74
5.4. Score de silhouette	75
5.5. Pipeline simple	75
6. Points de vigilance	76
6.1. Ne pas surinterpréter les résultats	76
6.2. Égale importance des variables	76
6.3. Choisir un bon K	76
6.4. ACP et variance maximale	76
7. Exercices de compréhension	76
References	79
Lab 0 — Environnement Python et premiers pas avec les données	80
1. Installer <code>uv</code>	80
1.1. Pourquoi <code>uv</code> plutôt que <code>venv</code> + <code>pip</code> ?	80
1.2. Installation	81
1.3. Choisir une version de Python	81
2. Créer le projet	81
2.1. Initialiser avec <code>uv</code>	81
2.2. Le dossier <code>.venv</code>	82
2.3. Installer les bibliothèques du Lab	82
2.4. Vérifier l'installation	82
2.5. Lancer Jupyter	83
3. Organiser le projet	83
4. Premier contact avec des données tabulaires	84
4.1. Explorer un <code>DataFrame</code>	84
4.2. Sélectionner, filtrer, agréger	84
4.3. Un premier graphique	85
5. Récupérer des données réelles via une API publique	85
5.1. Principe	85
5.2. Exemple : l'API météo Open-Meteo (sans clé)	86
6. Vérifier que <code>scikit-learn</code> fonctionne : le filtre anti-spam du chapitre 1	87
7. Synthèse	88
8. Lab Sheet — Projet guidé	88
Partie A — Environnement	88
Partie B — Météo au Maroc, à vous de jouer	88
Partie C — Améliorer le mini filtre anti-spam	89
Livrables	89
Lab 1 — Généralisation et évaluation avec <code>scikit-learn</code>	90
1. Rappel express : construire X et y	90
1.1. L'API commune de <code>scikit-learn</code>	91
2. Séparer Train et Test	91
3. Observer le sur-apprentissage à l'œil nu	92
4. La validation croisée	93
5. Une baseline avant tout modèle	94
6. Un vrai modèle, sa matrice de confusion et ses métriques	94
7. Repérer une fuite de données, concrètement	95

8. Un premier aperçu du réglage d'hyperparamètres	96
9. Synthèse	96
10. Lab Sheet — Projet guidé	96
Partie A — Prise en main	96
Partie B — Baseline et premier modèle	97
Partie C — Une mise à l'échelle change-t-elle tout ?	97
Partie D — Régler l'hyperparamètre k	97
Partie E — Évaluation finale	97
Livrables	97
Lab 2 — Collecte et prétraitement des données	98
1. Lire des données depuis plusieurs sources	98
1.1. CSV local	98
1.2. Une API JSON	98
1.3. Une base SQLite	99
1.4. Extraire une information depuis une page HTML	99
2. Construire un jeu de données volontairement imparfait	100
3. Diagnostiquer avant de nettoyer	100
3.1. Supprimer les doublons	101
3.2. Détecter les valeurs aberrantes (IQR)	101
3.3. Harmoniser les catégories textuelles	101
4. Imputation : à la main, puis avec <code>scikit-learn</code>	101
4.1. Calculer une imputation <i>from scratch</i>	101
4.2. La même chose avec <code>SimpleImputer</code>	101
4.3. Imputation catégorielle	102
5. Encodage : ne pas confondre <code>LabelEncoder</code> et <code>OrdinalEncoder</code>	102
6. Mise à l'échelle : calculer, puis vérifier	103
6.1. Standardisation <i>from scratch</i>	103
6.2. Vérifier avec <code>StandardScaler</code>	103
6.3. Et si une variable a des valeurs extrêmes ?	103
7. Construire le pipeline complet, sans fuite	104
8. Synthèse	105
9. Lab Sheet — Projet guidé	105
Partie A — Diagnostic	105
Partie B — Nettoyage et décisions justifiées	105
Partie C — Encodage et mise à l'échelle	106
Partie D — Pipeline complet	106
Livrables	106
Lab 3 — Régression : prédire une valeur numérique	107
1. Rappel : qu'est-ce qu'apprend une régression linéaire ?	107
1.1. Régression linéaire simple, <i>from scratch</i>	107
1.2. Vérification avec <code>scikit-learn</code>	108
2. Passer à la régression multiple	108
3. Métriques de régression : MAE, MSE, RMSE, R^2	109
4. Diagnostiquer un modèle par ses résidus	110
5. Régularisation : <code>Ridge</code> et <code>Lasso</code>	111
6. Régler α proprement avec <code>GridSearchCV</code>	111
7. Synthèse	112
8. Lab Sheet — Projet guidé	112
Partie A — Prise en main	112
Partie B — Une première régression	113
Partie C — Régularisation et réglage d'hyperparamètre	113
Livrables	113

Lab 4 — Classification : prédire une catégorie	114
1. Construire un jeu de données de <i>churn</i>	114
2. Un pipeline de classification, de bout en bout	115
3. Au-delà de l'accuracy : précision, rappel, F1, ROC-AUC	116
4. Le seuil de décision n'est pas obligatoirement 0,5	116
5. Gérer le déséquilibre des classes	117
6. Comparer plusieurs algorithmes, avec réglage d'hyperparamètres	117
7. Synthèse	118
8. Lab Sheet — Projet guidé	118
Partie A — Reprendre le pipeline du Lab 2	118
Partie B — Premier modèle et métriques	118
Partie C — Seuil et déséquilibre	119
Partie D — Comparaison de modèles et hyperparamètres	119
Livrables	119

Introduction au Machine Learning

💡 Objectifs du chapitre

À la fin de ce chapitre introductif, l'étudiant sera capable de :

- **Définir** formellement le Machine Learning et le **situer** dans l'écosystème de l'Intelligence Artificielle ;
- **Comparer** le paradigme de programmation classique et l'approche guidée par les données (*Data-Driven*) ;
- **Identifier** les grandes étapes du cycle de vie d'un projet de Machine Learning ;
- **Identifier** les outils de base utilisés dans ce cours (Python, JupyterLab, **pandas**, Scikit-Learn et **uv**).

1. Machine Learning

1.1. De l'Intelligence Artificielle au Machine Learning

L'**Intelligence Artificielle (IA)** est le champ scientifique le plus large : il regroupe l'ensemble des théories et techniques mises en œuvre pour concevoir des machines capables de simuler des comportements associés à l'intelligence humaine — raisonner, percevoir, planifier, comprendre le langage, ou encore apprendre.

i Repère historique

Le terme *Artificial Intelligence* est proposé en 1956 lors de la conférence de Dartmouth. Pendant plusieurs décennies, l'IA « symbolique » domine : on cherche à reproduire l'intelligence en manipulant des symboles et des règles logiques explicites. Ce n'est qu'à partir des années 1990-2010, avec l'explosion des données disponibles et de la puissance de calcul, que le **Machine Learning statistique** devient l'approche dominante de l'IA.

Historiquement, les premiers systèmes d'IA reposaient sur des **systèmes experts** : des ensembles de règles logiques (SI ... ALORS ...) écrites explicitement par des humains. Ils fonctionnaient bien dans des domaines circonscrits, mais s'avéraient extrêmement coûteux à maintenir et incapables de s'adapter à des situations non anticipées.

Le **Machine Learning (ML)** constitue une rupture : plutôt que de coder explicitement toutes les règles de décision, on utilise un algorithme pour apprendre, à partir d'exemples, un **modèle** capable de produire des prédictions sur de nouvelles données. Le modèle ne fournit pas toujours des règles lisibles ; il représente surtout une relation mathématique entre les données d'entrée et la sortie attendue.

Le **Deep Learning (Apprentissage Profond, DL)** est un **sous-domaine du ML** qui repose sur des réseaux de neurones artificiels comportant de nombreuses couches, performants sur des données non structurées (images, son, texte).

Comme le montre la Figure 1, ces trois notions s'emboîtent comme des poupées russes : chaque cercle est un cas particulier du cercle qui l'englobe.

Où se situe le Machine Learning ?

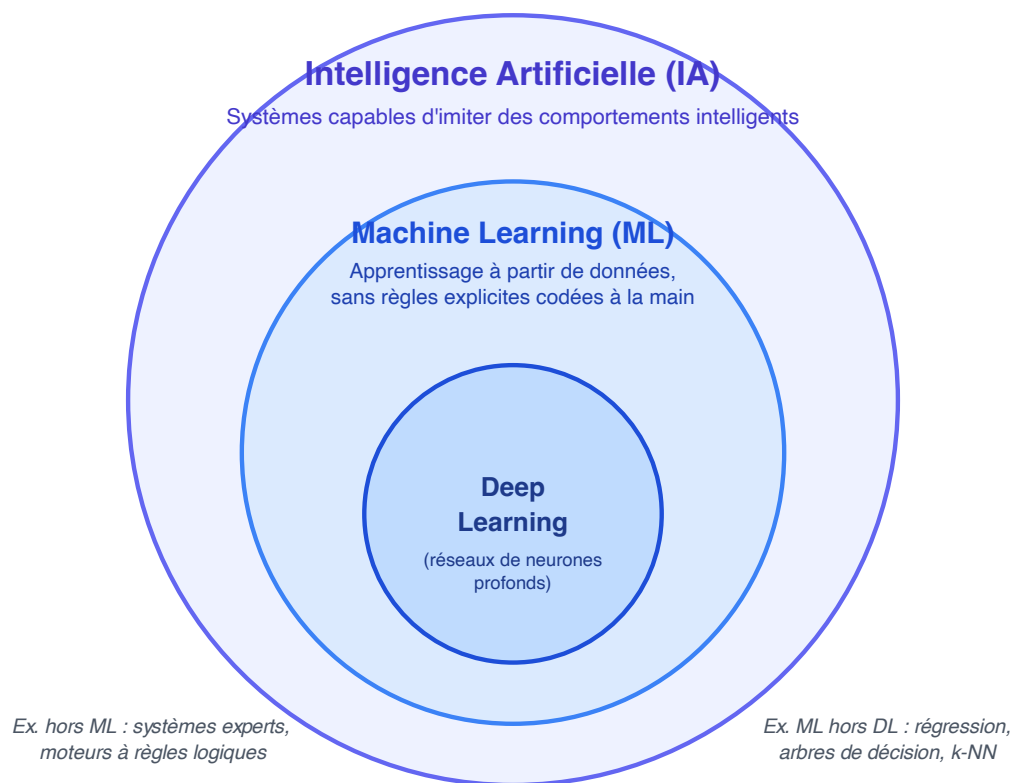


FIGURE 1 – Intelligence Artificielle, Machine Learning et Deep Learning

💡 À retenir

Tout algorithme de Deep Learning est un algorithme de Machine Learning, et tout algorithme de Machine Learning appartient au champ de l'IA — mais la réciproque est fausse (ex : un algorithme de recherche aux échecs, comme Minimax, est de l'IA sans être du ML, car il n'apprend rien à partir de données).

1.2. Définition formelle du Machine Learning

La définition de référence est celle de (Mitchell et Mitchell 1997) :

💡 Apprentissage Automatique (Machine Learning)

Définition 0.0.1. On dit qu'un programme informatique apprend à partir d'une expérience **E**, par rapport à une classe de tâches **T** et une mesure de performance **P**, si sa performance aux tâches de **T**, mesurée par **P**, s'améliore avec l'expérience **E**.

Cette définition se décompose en trois éléments concrets qui doivent être clairement identifiés **avant** de commencer tout projet. Voici son application pour un **filtre anti-spam** :

TABLE 1 – Le cadre TPE de Mitchell et Mitchell (1997) appliqué au filtre anti-spam

Symbole	Signification	Exemple : filtre anti-spam
T (Task)	La tâche que le système doit accomplir	Classer un e-mail comme <i>spam</i> ou <i>non-spam</i>
P (Performance)	La métrique d'évaluation	Proportion d'e-mails correctement classés
E (Experience)	Les données d'apprentissage	Une base d'e-mails déjà étiquetés

💡 Exemple

Reprenons ce triplet pour un second cas d'usage : un système destiné à **prédire si un client va résilier son abonnement téléphonique** (*churn*).

- **T** : prédire, pour chaque client, s'il résiliera dans les 30 prochains jours (classification binaire) ;
- **P** : le taux de bonnes prédictions sur des clients dont on connaît déjà l'issue réelle (mesuré sur des données jamais utilisées pour l'entraînement) ;
- **E** : l'historique des abonnements passés (durée, factures, appels au service client, résiliations effectives).

Cette grille de lecture (**T**, **P**, **E**) est fondamentale : avant même de choisir un algorithme, tout projet ML doit répondre clairement à ces trois questions.

Le chapitre suivant détaillera en profondeur les paradigmes d'apprentissage. Retenons pour l'instant que la première distinction à connaître oppose :

- **L'apprentissage supervisé**, où le système dispose des « bonnes réponses » pendant l'entraînement (ex. régression, classification) ;
- **L'apprentissage non-supervisé**, où le système ne dispose que des données brutes, sans réponse attendue (ex. clustering).

2. Programmation classique vs. approche Data-Driven

2.1. Le paradigme classique

Dans le paradigme **classique** (déductif), le développeur déduit un ensemble de **règles explicites**, puis les traduit en code. Le programme prend des **données** en entrée et produit des **résultats** en appliquant mécaniquement ces règles.

Prenons l'exemple du filtre anti-spam codé « à la main » :

```
def est_spam(email: str) -> bool:
    """
    Approche par règles explicites (programmation classique).
    Retourne:
    True si l'email contient un mot suspect,
    False sinon.
    """
    mots_suspects = ["gratuit", "gagner", "urgent", "viagra"]
    email_minuscule = email.lower()
    return any(mot in email_minuscule for mot in mots_suspects)
```

Ce type d'approche est idéal lorsque les règles métier sont **stables, peu nombreuses et bien comprises** (ex. calcul d'un impôt selon un barème officiel). Mais dès que les spammeurs changent de vocabulaire, la règle doit être **réécrite manuellement** — le programme n'apprend rien de lui-même.

2.2. Le paradigme Data-Driven

Le Machine Learning **inverse la logique** : on fournit au système des **données** et, dans le cas supervisé, les **résultats attendus** correspondants. Un **algorithme d'apprentissage** analyse les régularités entre ces deux éléments pour produire un **modèle**. Ce modèle est une fonction apprise automatiquement, capable de généraliser à de nouvelles données jamais vues.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

# Un texte doit d'abord être transformé en nombres.
modele = make_pipeline(
    TfidfVectorizer(),
    LogisticRegression(max_iter=1000)
)
modele.fit(X_entrainement, y_entrainement)
prediction = modele.predict([nouvel_email])
```

Ici¹, personne n'a écrit explicitement la règle « si le mot X apparaît, alors spam » : c'est l'algorithme `fit()` qui l'a **déduit statistiquement** à partir des exemples fournis.

La Figure 2 résume visuellement cette inversion : ce qui était une **sortie** (les résultats) dans la programmation classique devient une **entrée** (les résultats attendus, ou étiquettes) dans le paradigme Data-Driven.

1. Nous allons détailler et comprendre cette structure ainsi que l'algorithme utilisé dans les chapitres suivants.

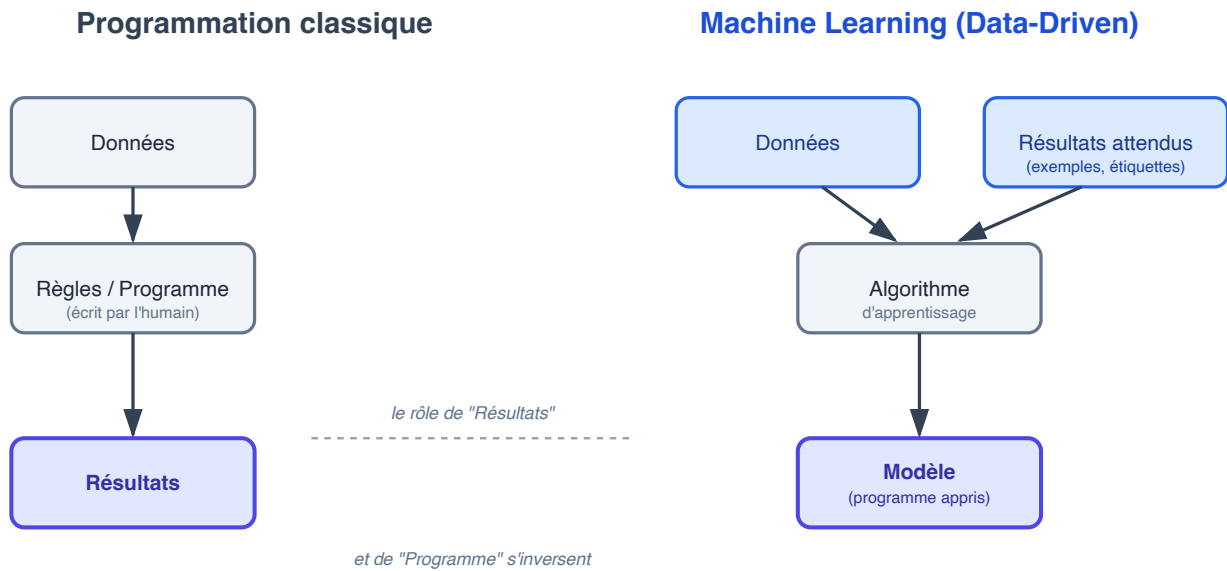


FIGURE 2 – Comparaison entre la programmation classique et l'approche Data-Driven

TABLE 2 – Comparaison synthétique des deux paradigmes

Critère	Programmation classique	Machine Learning
Fourni par l'humain	Règles + Données	Données + Résultats attendus (exemples)
Produit par le système	Résultats	Un modèle (règles apprises)
Adaptation	Faible (réécriture du code)	Élevée (réentraînement)
Interprétabilité	Totale	Variable selon le modèle
Besoin en données	Faible	Généralement élevé

⚠ Remarque

Le Machine Learning **n'est pas une solution universelle**. Si un problème admet une règle simple, déterministe et bien comprise (ex. conversion d'unités), la programmation classique reste souvent plus fiable, plus rapide à développer et plus facile à auditer qu'un modèle statistique.

3. Le cycle de vie d'un projet de Machine Learning

Entraîner un modèle ne représente qu'une fraction du travail réel. Un projet ML se décompose en sept étapes itératives (Géron 2022) :

1. **Cadrage du problème métier** : traduire un besoin en problème ML via la grille (T, P, E) — voir Définition 0.0.1.
2. **Collecte des données** : rassembler un jeu de données représentatif du phénomène à modéliser.
3. **Préparation et nettoyage** : étape la plus chronophage (souvent 60-80 % du temps réel du projet). Traitement des valeurs manquantes, encodage des variables catégorielles, création de nouvelles variables (*feature engineering*).
4. **Modélisation** : choix des algorithmes et entraînement sur les données pour minimiser une erreur.
5. **Évaluation** : mesurer objectivement la performance sur des données jamais vues (jeu de test).
6. **Déploiement** : intégrer le modèle dans un système opérationnel (application, API, tableau de bord).
7. **Monitoring** : surveiller les performances en production pour détecter la **dérive des données** (*data drift*) et déclencher un réentraînement si nécessaire.

Cycle de vie d'un projet Machine Learning

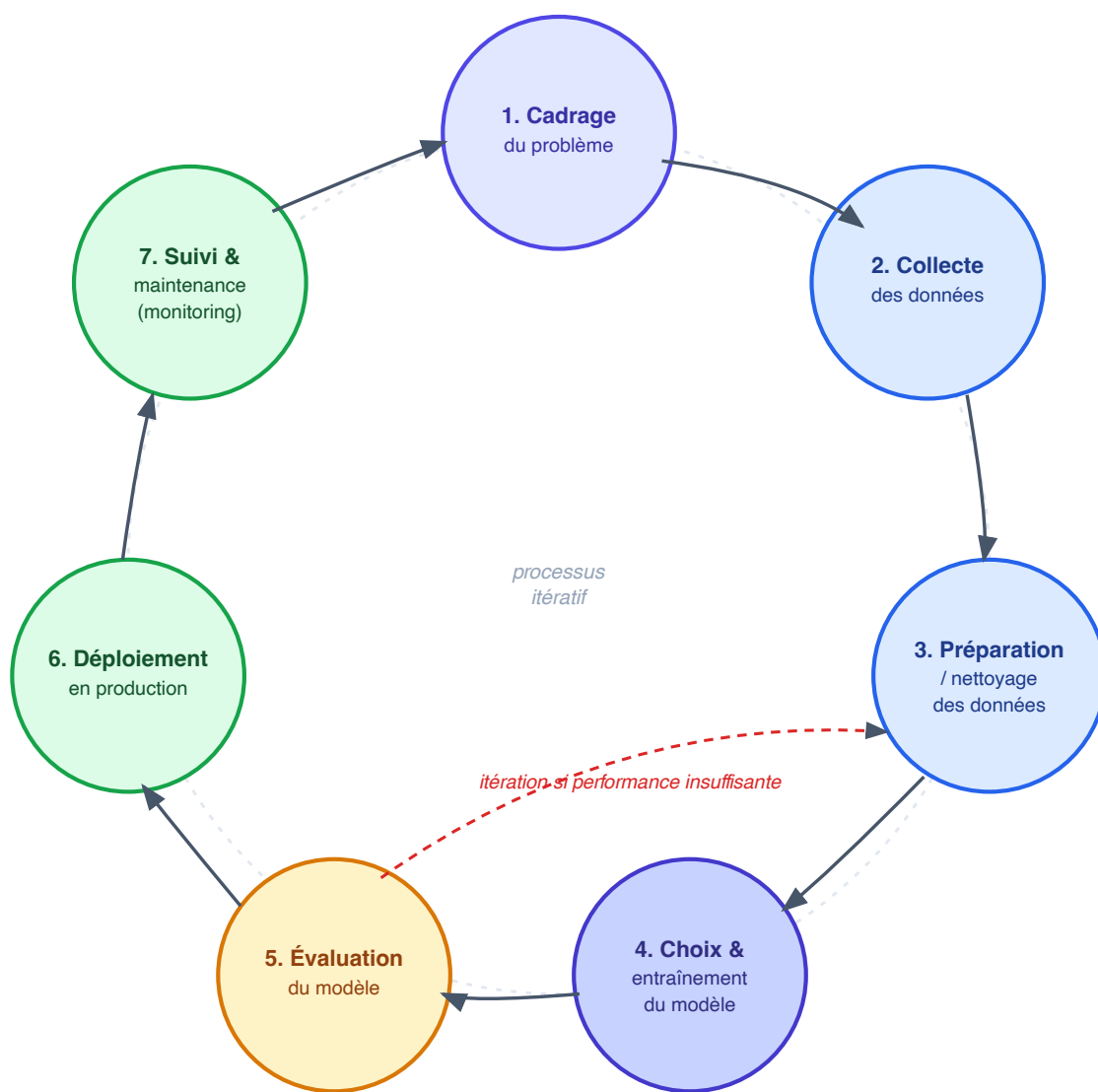


FIGURE 3 – Le cycle de vie d'un projet de Machine Learning

Il est essentiel de comprendre que ce cycle **n'est presque jamais parcouru une seule fois de façon linéaire**. Une évaluation décevante à l'étape 5 renvoie très souvent vers l'étape 3 (améliorer les variables) ou l'étape 4 (changer d'algorithme), plus rarement vers l'étape 1.

4. Python pour le Machine Learning

Pour mener à bien ce cycle, le choix du langage de programmation constitue une étape importante. L'écosystème du *Machine Learning* s'appuie sur plusieurs langages, chacun disposant de domaines d'utilisation et de points forts spécifiques. Parmi eux, **R**, largement utilisé dans les domaines de la Statistique et de l'analyse de données, **Julia**, conçu notamment pour le calcul scientifique et la performance numérique, ainsi que **C++** et **Java**, qui sont également présents dans de nombreux contextes de développement logiciel et d'applications à fortes contraintes de performance ou d'intégration.

Dans le cadre de ce cycle, nous retenons **Python** comme langage principal. Ce choix s'explique notamment par la lisibilité de sa syntaxe, la richesse de son écosystème scientifique et sa large utilisation dans les domaines de l'analyse de données, de l'intelligence artificielle et du *Machine Learning*. Python permet ainsi de couvrir différentes étapes d'un projet, depuis la préparation et l'exploration des données jusqu'à l'entraînement, l'évaluation et la mise en œuvre de modèles.

4.1. Quelques bibliothèques scientifiques de l'écosystème Python

L'écosystème python comprend notamment **NumPy** pour le calcul numérique, **pandas** pour la manipulation et l'analyse des données, **scikit-learn** pour les principales méthodes d'apprentissage automatique, ainsi que **TensorFlow** et **PyTorch** pour le développement de modèles d'apprentissage profond.

Le choix de Python présente également un intérêt pédagogique : il permet de mettre l'accent sur les concepts, les méthodes et le raisonnement propres au *Machine Learning*, tout en s'appuyant sur des bibliothèques largement utilisées dans la pratique. Au cours de ce cycle, nous utiliserons donc Python et cet écosystème d'outils pour illustrer progressivement les différentes étapes de la construction et de l'exploitation d'un modèle de *Machine Learning*.

TABLE 3 – Quelques bibliothèques principales de l'écosystème Python utilisées dans le cycle

Bibliothèque	Rôle dans le cycle de vie du projet
pandas	Manipulation et nettoyage de tableaux de données (DataFrames).
numpy	Calcul numérique, calcul matriciel et opérations mathématiques optimisées.
matplotlib / seaborn	Visualisation et exploration visuelle des données.
scikit-learn	Algorithmes classiques de régression, classification, clustering et outils de prétraitement.

Chaque bibliothèque de la Table 3 correspond typiquement à une ou plusieurs étapes du cycle de vie présenté en Figure 3 : **pandas** intervient surtout aux étapes 2-3, **scikit-learn** à l'étape 4 et **matplotlib** accompagne l'ensemble du processus. Les bibliothèques plus spécialisées seront laissées de côté dans ce cours introductif.

4.2. Environnements de développement : exploration et production

Il convient de distinguer la phase d'**exploration et de prototypage** de la phase de **développement et de mise en production**. Ces deux étapes peuvent mobiliser des outils différents, car leurs objectifs et leurs contraintes ne sont pas les mêmes.

Dans ce cours, nous adoptons une organisation simple et progressive : les **notebooks Jupyter**, notamment à travers **JupyterLab** et **Google Colab**, seront privilégiés pour l'exploration et le prototypage, tandis que **Visual Studio Code (VS Code)** sera utilisé pour structurer et finaliser le code destiné à la production.

Ce choix constitue une convention pédagogique propre à ce cours. Il existe naturellement d'autres environnements et plateformes, tels que **DataSpell**, **PyCharm**, **Databricks** et d'autres solutions, qui peuvent être pertinents selon les contextes, les projets et les habitudes de développement.

Exploration et prototypage

JupyterLab JupyterLab est un environnement de développement interactif permettant notamment de travailler avec des *notebooks* Jupyter, constitués de cellules de texte et de code pouvant être exécutées indépendamment. Il est particulièrement adapté à l'exploration d'un jeu de données, à la visualisation des résultats et à la documentation progressive du raisonnement. **JupyterLab constituera l'un des principaux environnements utilisés pour les travaux pratiques de ce cours.**

Google Colab Google Colab est un environnement de *notebooks* hébergé dans le cloud. Il permet d'exécuter du code Python sans nécessiter l'installation préalable d'un environnement de développement local et facilite le partage des notebooks. Selon les ressources disponibles et le type de compte utilisé, des ressources de calcul spécialisées, notamment des GPU, peuvent également être proposées. L'environnement d'exécution étant temporaire, les fichiers et les résultats importants doivent être sauvegardés dans un emplacement persistant.

Développement et production

Visual Studio Code (VS Code) Visual Studio Code est un éditeur de code généraliste que nous utiliserons pour passer du code exploratoire à une organisation plus structurée du projet. Il permet notamment de travailler avec des fichiers Python, de structurer le code en modules, de gérer les dépendances et l'environnement du projet, d'utiliser Git, d'écrire et d'exécuter des tests, puis de préparer le code pour son déploiement sous différentes formes, par exemple sous la forme d'une API ou d'un service web.

Pourquoi ne pas tout faire dans un seul notebook ?

Un notebook est particulièrement adapté à l'expérimentation et à l'exploration interactive. Cependant, au fur et à mesure qu'un projet se complexifie, conserver l'ensemble du code dans un ou plusieurs notebooks peut rendre sa maintenance, sa réutilisation, ses tests et son déploiement plus difficiles. Les notebooks peuvent également conserver un **état d'exécution implicite**, notamment lorsque les cellules sont exécutées dans un ordre différent de celui dans lequel elles apparaissent.

Dans ce cours, nous adopterons donc une démarche progressive :

1. **explorer et expérimenter dans un notebook ;**
2. **valider les choix méthodologiques et le fonctionnement du modèle ;**
3. **consolider le code dans des fichiers Python (.py) structurés et réutilisables ;**
4. **tester, versionner et préparer le projet pour son utilisation finale ou son déploiement.**

Il ne s'agit pas d'opposer les notebooks au développement logiciel : les notebooks restent utiles pour l'exploration, l'analyse et la présentation des résultats, tandis que les fichiers Python structurés sont généralement plus adaptés lorsque le code doit être maintenu, testé et réutilisé dans un projet plus important.

4.3. Gestion de projet avec uv

Un projet de *Machine Learning* ne dépend pas uniquement du code que nous écrivons. Il repose également sur un **environnement logiciel** : une version de Python, un ensemble de bibliothèques, des versions compatibles de ces bibliothèques et différents outils nécessaires à l'exécution du projet.

La gestion de cet environnement est importante dès qu'un projet doit être **reproduit, partagé, maintenu ou déployé**. Deux utilisateurs disposant du même code peuvent, par exemple, rencontrer des problèmes différents si leurs versions de Python ou de certaines dépendances ne sont pas les mêmes. La gestion des dépendances constitue ainsi une composante essentielle de la reproductibilité et de la maintenance d'un projet.

Dans ce cours, nous présenterons d'abord l'approche classique de l'écosystème Python, puis nous adopterons uv comme outil de gestion de nos projets.

L'approche classique : Python, venv et pip

L'écosystème Python fournit déjà les outils nécessaires pour créer un environnement isolé et installer les bibliothèques d'un projet. Le module `venv` est l'outil standard de Python pour créer des environnements virtuels. Ceux-ci permettent d'isoler les dépendances d'un projet de celles installées dans les autres projets ou dans l'installation générale de Python.

L'installation des bibliothèques s'effectue généralement avec `pip`, l'installateur de paquets Python. Les bibliothèques sont notamment disponibles sur `PyPI`, le Python Package Index.

Une configuration classique peut donc être représentée de manière simplifiée par :

```
python -m venv .venv
```

puis, dans l'environnement virtuel :

```
python -m pip install numpy pandas scikit-learn
```

Les dépendances peuvent ensuite être consignées dans un fichier `requirements.txt`, par exemple :

```
numpy
pandas
scikit-learn
```

Cette approche est parfaitement valable et reste largement utilisée. Le [Python Packaging User Guide](#) présente d'ailleurs cette organisation ainsi que les pratiques modernes de gestion et de distribution des projets Python.

Elle présente cependant une certaine fragmentation : la création de l'environnement, la gestion des dépendances et leur installation sont assurées par des mécanismes distincts. Pour des projets plus structurés, il devient également utile de disposer d'une déclaration centralisée des dépendances et d'un mécanisme de verrouillage permettant de conserver une résolution précise de l'environnement.

Vers une gestion moderne du projet avec `pyproject.toml`

Les projets Python modernes utilisent notamment le fichier `pyproject.toml`. Ce fichier permet de centraliser différentes informations relatives au projet, notamment ses métadonnées et ses dépendances.

Il constitue aujourd'hui un élément central de l'écosystème de *packaging* Python et est utilisé par de nombreux outils modernes.

Dans ce cours, nous allons nous appuyer sur `uv`, développé par Astral, pour gérer nos projets Python.

Pourquoi uv ?

`uv` est un gestionnaire de projets et de paquets Python qui regroupe dans un même outil plusieurs opérations auparavant réalisées avec différents outils. Il permet notamment de gérer les projets Python, les environnements virtuels, les versions de Python, les dépendances et leur résolution, ainsi que l'exécution des commandes dans l'environnement du projet.

La documentation officielle de [uv](#) présente notamment les fonctions suivantes :

- création d'un projet avec `uv init` ;
- ajout et suppression de dépendances ;
- gestion de l'environnement virtuel du projet ;
- synchronisation des dépendances ;
- création et mise à jour du fichier de verrouillage ;
- exécution de commandes dans l'environnement du projet ;
- gestion de projets plus complexes au moyen de *workspaces*.

L'intérêt de `uv` dans notre cours est donc moins de remplacer une commande particulière que de fournir une **organisation cohérente de l'ensemble du projet Python**.

`pyproject.toml`, `uv.lock` et `.venv`

Avec `uv`, trois éléments sont particulièrement importants.

Le fichier `pyproject.toml` décrit le projet et ses besoins, notamment les dépendances et la version de Python requise.

Le fichier `uv.lock` enregistre la résolution précise des dépendances du projet. Contrairement au fichier `pyproject.toml`, qui exprime les besoins du projet de manière générale, le fichier de verrouillage conserve les versions effectivement résolues. Il peut ainsi être versionné avec le projet afin de faciliter la reconstruction d'un environnement cohérent sur différentes machines. La documentation officielle de [uv](#) décrit précisément le rôle de `uv.lock`.

Enfin, `uv` utilise un environnement virtuel associé au projet, généralement situé dans `.venv`.

On peut donc représenter schématiquement l'organisation d'un projet ainsi :

```
projet/  
  .venv/  
  pyproject.toml  
  uv.lock  
  ...
```

Le principe peut être résumé ainsi :

`pyproject.toml` décrit les besoins du projet ; `uv.lock` conserve leur résolution précise ; `.venv` contient l'environnement dans lequel le projet est exécuté.

La synchronisation entre ces éléments est prise en charge par `uv`. La documentation officielle explique notamment que les opérations telles que [uv sync](#) et [uv run](#) peuvent mettre à jour l'environnement à partir du fichier de verrouillage.

Pourquoi uv plutôt qu'Anaconda ou Conda ?

Il existe d'autres solutions parfaitement valables pour gérer les environnements Python. L'une des plus connues est l'écosystème [Conda](#), qui permet notamment de créer, gérer, partager et supprimer des environnements contenant différentes versions de Python et de paquets.

Notre choix de uv dans ce cours ne signifie donc pas que Conda ou [Anaconda](#) sont inadaptés. Il s'agit d'un **choix pédagogique et technique** correspondant à l'organisation que nous souhaitons adopter.

Nous retenons uv principalement pour les raisons suivantes :

1. **Une approche directement centrée sur les projets Python.** uv s'intègre naturellement aux mécanismes modernes du *packaging* Python, notamment `pyproject.toml` et l'écosystème PyPI.
2. **Une gestion unifiée du projet.** Un même outil permet de gérer l'environnement, les dépendances, leur résolution, le verrouillage et l'exécution du projet.
3. **Une structure explicite et versionnable.** L'association de `pyproject.toml` et `uv.lock` fournit une description claire du projet et de son environnement de dépendances.
4. **Une bonne adéquation avec les projets du cours.** Les projets étudiés utiliseront principalement des bibliothèques de l'écosystème Python, telles que `numpy`, `pandas`, `matplotlib` et `scikit-learn`, ainsi que les outils Jupyter. uv fournit une organisation cohérente pour leur gestion dans un projet Python local.
5. **Une continuité entre expérimentation et développement.** Les étudiants pourront commencer par explorer les données et les modèles dans des notebooks, puis retrouver le même environnement de projet lorsqu'ils passeront à une organisation plus structurée du code Python.

Le choix de uv répond donc avant tout aux besoins de ce cours : disposer d'un environnement Python **isolé, explicite, reproductible et facile à partager**, tout en introduisant progressivement les pratiques modernes de gestion de projet.

Exercices

i Consigne

Les corrigés sont fournis juste après chaque énoncé, dans un bloc repliable. Essayez de répondre par écrit **avant** de les consulter.

Exercice 0.0.1 (Vrai ou Faux). Indiquez si chaque affirmation est vraie ou fausse, en justifiant brièvement.

- a. Tout système d'Intelligence Artificielle est un système de Machine Learning.
- b. Tout algorithme de Deep Learning est un algorithme de Machine Learning.
- c. Un système expert à base de règles **SI ... ALORS ...** relève du Machine Learning.
- d. Un modèle de Machine Learning peut être vu comme un « programme appris automatiquement ».

Exercice 0.0.2 (QCM). Dans la définition de ([Mitchell et Mitchell 1997](#)) (Définition 0.0.1), que représente la lettre **E** ?

1. L'erreur (*Error*) commise par le modèle
2. L'expérience (*Experience*), c'est-à-dire les données utilisées pour apprendre
3. L'ensemble (*Ensemble*) des hyperparamètres du modèle
4. L'évaluation (*Evaluation*) finale du système

Exercice 0.0.3 (Formuler le triplet (T, P, E)). Une chaîne de supermarchés marocaine souhaite développer un système capable de **prédire, pour chaque client fidèle, le montant qu’il dépensera lors de sa prochaine visite**, à partir de son historique d’achats des deux dernières années.

Formulez le triplet (T, P, E) pour ce projet.

Exercice 0.0.4 (Programmation classique ou Machine Learning ?). Pour chaque situation, indiquez l’approche la plus appropriée et justifiez en une phrase.

- Convertir un montant de dirhams marocains (MAD) en euros à un taux de change fixé.
- Détecter automatiquement si un commentaire posté sur un réseau social est injurieux.
- Calculer la date du dernier jour du mois, pour n’importe quel mois de l’année.
- Estimer, à partir d’une radiographie, si un patient présente une fracture osseuse.

Exercice 0.0.5 (Reconstituer le schéma Data-Driven). En vous appuyant sur la Figure 2, reconstituez sous la forme $A \rightarrow B$ l’enchaînement correct des quatre éléments suivants du paradigme Machine Learning : **Modèle, Données, Algorithme d’apprentissage, Résultats attendus**.

Exercice 0.0.6 (Remettre les étapes dans l’ordre). Voici, dans le désordre, les sept étapes du cycle de vie d’un projet de Machine Learning :

Évaluation du modèle · Cadrage du problème métier · Monitoring · Modélisation · Préparation et nettoyage · Déploiement · Collecte des données

Remettez-les dans l’ordre correct.

Exercice 0.0.7 (Étude de cas : dérive du modèle). Une banque a déployé, en janvier, un modèle de scoring de crédit qui obtenait d’excellents résultats en test. Six mois plus tard, le modèle rejette de plus en plus de dossiers qui se révèlent être de bons payeurs.

- Comment nomme-t-on ce phénomène dans le cours ?
- À quelle(s) étape(s) du cycle de vie ce constat doit-il renvoyer l’équipe ?

Exercice 0.0.8 (Outils et bibliothèques).

- Associez chaque bibliothèque à son rôle principal : **pandas**, **scikit-learn**, **numpy**. Rôles : (i) modélisation classique (régression/classification); (ii) manipulation de tableaux de données; (iii) calcul numérique et matriciel.
- Un étudiant veut explorer rapidement un nouveau jeu de données et visualiser quelques graphiques avant de structurer un vrai projet. Quel outil de la section Section lui conseillez-vous, et pourquoi ?

Fondements Théoriques du Machine Learning

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Distinguer** l'apprentissage supervisé et non supervisé ;
- **Utiliser** le vocabulaire fondamental (matrice de design, vecteur cible, hyperparamètres) ;
- **Diagnostiquer** le sur-apprentissage et le sous-apprentissage ;
- **Structurer** correctement un jeu de données (Train/Validation/Test) pour éviter la fuite de données ;
- **Évaluer** l'importance d'une baseline et les limites de certaines métriques.

1. Vocabulaire et représentation des données

Avant de choisir un algorithme, il faut formaliser algébriquement le jeu de données. Chaque **ligne** représente une observation et chaque **colonne** une information.

TABLE 1 – Notions et vocabulaire

Terme	Définition	Exemple (Prédiction de prix)
Observation	Un exemple individuel décrit par plusieurs variables	Un appartement particulier
Échantillon	Un ensemble d'observations représentant une population	10 000 appartements
Variable explicative (<i>Feature</i>)	Information utilisée par le modèle pour prédire	Surface, quartier
Cible (<i>Target</i>)	La valeur que le modèle doit apprendre à prédire	Prix de vente
Étiquette (<i>Label</i>)	La réponse connue associée à une observation (en classification)	« Vendu rapidement »

Notation mathématique standard, utilisée dans de ce cours :

- X : la **matrice de design** contenant les variables explicatives (une ligne par observation, une colonne par variable) ;
- y : le **vecteur cible** (ou vecteur des étiquettes) ;
- x_i : les variables explicatives de l'observation i (la i -ème ligne de X) ;
- y_i : la réponse associée à l'observation i (le i -ème élément de y).

! Exemple

Pour le jeu de données ci-dessous (2 appartements, 2 variables explicatives numériques) :

id	surface (m^2)	nbr_chambres	prix (MAD)
1	65	2	850 000
2	42	1	420 000

On note $x_1 = (65, 2)$ et $y_1 = 850\,000$ pour la première observation.

Les variables explicatives sont regroupées dans la matrice des caractéristiques X (de dimension 2×2), et les cibles dans le vecteur-colonne y :

$$X = \begin{pmatrix} 65 & 2 \\ 42 & 1 \end{pmatrix} \quad \text{et} \quad y = \begin{pmatrix} 850\,000 \\ 420\,000 \end{pmatrix}$$

2. Les grandes familles d'apprentissage

Il existe plusieurs façons de classer les méthodes de Machine Learning. Pour ce cours, la distinction fondamentale repose sur la **nature du signal disponible pour l'apprentissage**.

On retient ici deux **grands paradigmes**, étudiés dans les chapitres suivants :

- l'apprentissage supervisé ;
- l'apprentissage non supervisé ;
- l'apprentissage non supervisé.

Supervisé

Le modèle apprend à partir de données **étiquetées** : chaque exemple d'entraînement est associé à une **cible** y que le modèle cherche à prédire à partir des données d'entrée X .

Les deux principales tâches sont :

- **Régression** : la cible y est une valeur numérique, généralement continue.
Exemple : prédire le prix d'un logement.
- **Classification** : la cible y appartient à un ensemble de classes.
Exemple : déterminer si une transaction est frauduleuse ou non.

Pendant l'entraînement, le modèle dispose donc d'exemples de la forme :

$$(X, y)$$

et cherche à apprendre une relation permettant de produire une prédiction $\hat{y}$ à partir d'une nouvelle entrée X .

Non supervisé

Le modèle apprend à partir de données **non étiquetées**, c'est-à-dire sans cible y fournie à l'avance.

L'objectif est de mettre en évidence des **structures, régularités, relations ou regroupements** présents dans les données.

Parmi les tâches courantes :

- **Clustering** : regrouper les observations selon leur similarité ;
- **Réduction de dimension** : représenter les données dans un espace de dimension plus faible ;
- **Détection d'anomalies** : identifier des observations qui s'écartent du comportement habituel.

Exemple : regrouper des clients selon leur comportement d'achat sans disposer de catégories prédéfinies.

On dispose principalement des données X sans cible y fournie pour l'apprentissage.

💡 À retenir

La question à se poser pour choisir un paradigme est toujours la même : **de quelles données est-ce que je dispose ?** Ai-je une cible connue pour chaque observation (supervisé) ou dois-je rechercher une structure sans cible fournie (non supervisé) ?

i Périmètre du cours

Le présent cours se concentrera sur une sélection d'algorithmes fondamentaux issus de l'**apprentissage supervisé** et de l'**apprentissage non supervisé**. L'apprentissage par renforcement et les approches semi-supervisées seront seulement mentionnés, car ils dépassent l'objectif de cette introduction.

3. Apprendre et généraliser

Le meilleur modèle n'est pas celui qui mémorise le mieux les données d'entraînement, mais celui qui **généralise** le mieux à des données inédites.

- Principe fondamental du Machine Learning

La **généralisation** est la capacité du modèle à produire une prédiction correcte face à une observation qu'il n'a jamais rencontrée. L'état d'un modèle se classe dans l'une de ces trois situations :

1. **Sous-apprentissage (*Underfitting*)** : modèle trop simple. Incapable de capturer la structure des données. Performances médiocres partout — y compris sur les données d'entraînement.
2. **Ajustement optimal (*Good fit*)** : le modèle extrait les règles générales (le **signal**) et ignore les anomalies (le **bruit**).
3. **Sur-apprentissage (*Overfitting*)** : modèle trop complexe. Il mémorise le bruit spécifique aux données d'entraînement. Il excelle artificiellement sur l'historique mais échoue sur de nouvelles données.

La Figure 1 illustre le compromis entre un modèle trop simple et un modèle trop complexe. L'objectif est de choisir une complexité pour laquelle l'erreur sur des données nouvelles est faible. Le vocabulaire biais-variance pourra être approfondi plus tard ; ici, il suffit de retenir que mémoriser les données d'entraînement ne garantit pas une bonne généralisation.

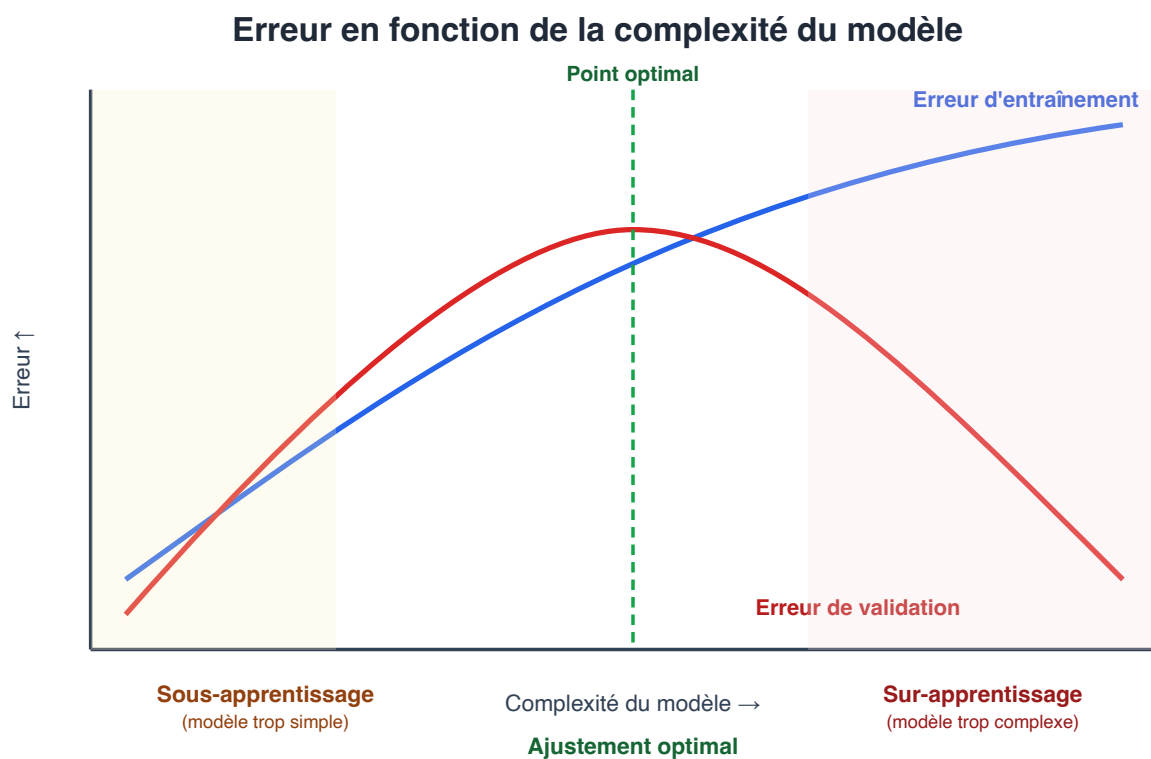


FIGURE 1 – Évolution de l'erreur d'entraînement et de validation selon la complexité du modèle

i Comment détecter concrètement un sur-apprentissage ?

Le signal le plus caractéristique est un **grand écart entre la performance sur le jeu d'entraînement et sur le jeu de validation** : une accuracy de 99 % en entraînement contre 65 % en validation est une signature typique de sur-apprentissage. À l'inverse, une accuracy médiocre et similaire sur les deux jeux (par exemple 55 % partout) évoque plutôt un sous-apprentissage.

4. Jeux d'évaluation et Fuite de données

Pour diagnostiquer la généralisation, on divise l'échantillon initial en **trois sous-ensembles disjoints** :

TABLE 2 – Les jeux d'évaluation

Jeu de données	Rôle
Entraînement (<i>Train</i>)	Ajuster les paramètres internes du modèle.
Validation	Comparer différents modèles et régler les hyperparamètres.
Test	Estimer la performance finale sur des données strictement inédites.

Séparation du jeu de données

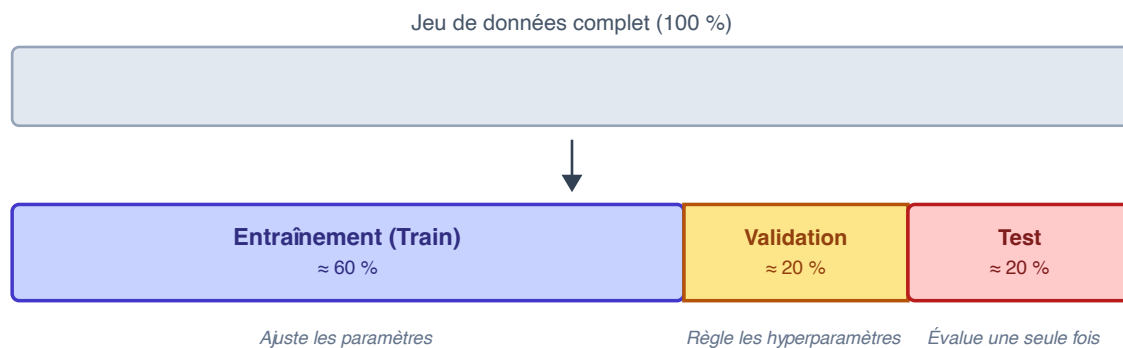


FIGURE 2 – Séparation typique d'un jeu de données en Train / Validation / Test

⚠ Règle d'or

Le jeu de **Test** ne doit être utilisé **qu'une seule fois**, tout à la fin du projet, pour obtenir une estimation honnête de la performance. Le consulter à répétition pour ajuster des choix de modélisation revient, de fait, à l'intégrer au jeu d'entraînement — et fausse l'estimation finale.

4.1. Le cas particulier des données temporelles

Lorsque les observations sont **datées** (ex. transactions bancaires, ventes mensuelles), un découpage **aléatoire** de Train/Validation/Test est dangereux : le modèle pourrait apprendre à partir d'informations « futures » par

rapport à certaines observations de test, ce qui **surestime artificiellement** sa performance réelle. Dans ce cas, on applique plutôt un **découpage chronologique** : les données les plus anciennes forment le jeu d'entraînement, les plus récentes le jeu de test — reproduisant ainsi la situation réelle où le modèle doit prédire l'avenir à partir du passé.

4.2. La fuite de données (*Data Leakage*)

Il y a **fuite de données** lorsqu'une information qui ne devrait pas être disponible au moment de la prédiction influence malgré tout l'entraînement. Cela génère des performances artificiellement excellentes, qui s'effondrent en production.

Exemples caractéristiques :

- Normaliser (centrer-réduire) **l'ensemble** du jeu de données *avant* de le séparer en Train/Test : les statistiques de normalisation (moyenne, écart-type) « voient » alors des données de test.
- Inclure une variable qui n'est calculée ou connue **qu'après** l'événement à prédire (ex. la date d'une consultation médicale future pour prédire une réadmission hospitalière).

Pour éviter ce type d'erreur, on utilise des **Pipelines** : des enchaînements d'étapes (normalisation, encodage, modélisation) que l'on ajuste **uniquement** sur le jeu d'entraînement, puis que l'on applique tel quel aux jeux de validation et de test.

5. Paramètres et Hyperparamètres

TABLE 3 – Paramètres et hyperparamètres

Élément	Qui le fixe ?	Exemple
Paramètre	Le modèle l'apprend automatiquement à partir des données	Coefficients d'une régression linéaire
Hyperparamètre	Le développeur le règle avant l'entraînement	Profondeur maximale d'un arbre de décision

💡 Comment choisir un hyperparamètre sans tricher ?

On ne règle **jamais** un hyperparamètre en observant sa performance sur le jeu de **test** — cela reviendrait à une fuite de données (voir §4.2). La bonne pratique consiste à comparer plusieurs valeurs d'hyperparamètres uniquement sur le jeu de **validation** (Table 2), et à ne consulter le jeu de test qu'une fois le meilleur réglage définitivement choisi.

6. La Baseline et les limites de l'Accuracy

Une **baseline** est une règle de décision extrêmement simple (ex. « toujours prédire la classe majoritaire »). Elle répond à la question : *le modèle ML complexe fait-il réellement mieux que l'approche la plus basique possible ?*

 Le piège des classes déséquilibrées

Si 99 % des transactions sont légitimes et 1 % sont des fraudes, une baseline qui prédit systématiquement « Légitime » obtiendra **99 % d'*accuracy*** (exactitude) — un score en apparence excellent — alors même que le modèle est totalement inutile, puisqu'il ne détecte jamais aucune fraude.

Pour étudier plus finement les erreurs d'un modèle de classification, on utilise notamment la **matrice de confusion**, qui sera définie et étudiée dans le chapitre consacré à la classification.

7. Prédiction, Corrélation et Causalité

Un modèle ML excelle à trouver des **corrélations** (ex. les ventes de glaces et les coups de soleil augmentent ensemble l'été). Cependant :

- La **prédiction** exploite une corrélation pour estimer une valeur inconnue.
- La **causalité** prouve qu'agir sur A modifie effectivement B.

Un bon modèle prédictif **ne justifie pas** une décision causale : arrêter de vendre des glaces ne réduira évidemment pas le nombre de coups de soleil, car la vraie cause commune des deux phénomènes est... le beau temps.

 Remarque

Avant de transformer une corrélation détectée par un modèle en **décision d'action** (« si on fait X, alors Y va s'améliorer »), il faut se demander s'il n'existe pas une **variable cachée** expliquant conjointement les deux phénomènes observés.

8. Enjeux Éthiques et Reproductibilité

L'éthique dans le ML :

- **Biais de représentation** : les données d'entraînement reflètent-elles fidèlement la population sur laquelle le modèle sera utilisé, ou risquent-elles de discriminer certains groupes ?
- **Explicabilité** : peut-on justifier, a posteriori, la décision prise par la machine — en particulier lorsque celle-ci affecte directement une personne (crédit refusé, diagnostic médical) ?

La Reproductibilité : un résultat scientifique ou industriel doit pouvoir être relancé à l'identique par une autre personne. Pratiques obligatoires :

1. Figurer les versions des bibliothèques utilisées (`uv.lock`, voir le chapitre d'introduction).
2. Fixer les « graines aléatoires » (*random seeds*) de tous les processus stochastiques.
3. Versionner le code (Git) et documenter précisément la pipeline de données.

Exercices

i Consigne

Les corrigés sont fournis juste après chaque énoncé, dans un bloc repliable. Essayez de répondre par écrit **avant** de les consulter.

Exercice 0.0.1 (Vocabulaire et représentation). On dispose du tableau suivant, pour la tâche « prédire si un appartement sera vendu rapidement » :

id	surface	quartier	prix	vendu_rapidement
1	65	Centre	850000	oui
2	42	Nord	420000	non
3	80	Centre	990000	oui

1. Qu'est-ce qu'une observation, ici ?
2. Quel est l'échantillon ?
3. Quelles sont les variables explicatives ?
4. Quelle est la cible ?
5. Quel est le rôle de la colonne `id` ?

Exercice 0.0.2 (Diagnostiquer sous/sur-apprentissage). Deux modèles donnent les résultats suivants :

Modèle	Accuracy train	Accuracy test
A	99 %	61 %
B	84 %	81 %

1. Quel modèle généralise le mieux ?
2. Quel modèle semble en sur-apprentissage ?
3. Pourquoi ne faut-il pas choisir le modèle A en regardant uniquement l'accuracy d'entraînement ?

Exercice 0.0.3 (Fuite de données). Pour prédire si un patient sera réadmis dans les 30 jours suivant sa sortie, une équipe utilise :

- l'âge ;
- le diagnostic initial ;
- la durée d'hospitalisation ;
- **la date de la prochaine consultation** ;
- le nombre de médicaments prescrits pendant le séjour.

1. Quelle variable est potentiellement une fuite de données ?
2. Pourquoi ?

Exercice 0.0.4 (Paramètres vs Hyperparamètres). Pour chacun des éléments suivants, indiquez s'il s'agit d'un **paramètre** appris ou d'un **hyperparamètre** choisi par le développeur :

1. les coefficients d'une régression linéaire ;
2. le nombre maximal de niveaux d'un arbre de décision ;
3. le nombre de voisins k dans l'algorithme des k plus proches voisins.

Exercice 0.0.5 (Baseline et classes déséquilibrées). Dans un jeu de classification de 1 000 observations, la classe A représente 900 observations et la classe B seulement 100.

1. Quelle accuracy obtient la baseline qui prédit toujours A ?
2. Pourquoi cette baseline peut-elle être utile malgré sa simplicité ?
3. Pourquoi faut-il compléter l'accuracy par d'autres mesures lorsque la classe B est minoritaire ?

Exercice 0.0.6 (Préparer une évaluation). Une banque utilise un modèle pour repérer des demandes de prêt potentiellement frauduleuses. Les fraudes sont rares, mais manquer une fraude peut coûter très cher.

1. Pourquoi l'accuracy seule risque-t-elle d'être insuffisante ?
2. Pourquoi faut-il discuter avec le métier avant de choisir une métrique ?
3. Dans quel chapitre trouvera-t-on les outils permettant de détailler les erreurs positives et négatives ?

Exercice 0.0.7 (Corrélation et causalité). Une analyse montre que les personnes qui utilisent davantage une application de e-learning réussissent mieux une formation. L'équipe pédagogique conclut que rendre l'application obligatoire fera automatiquement réussir davantage d'étudiants.

1. Quelle est la corrélation observée ?
2. Pourquoi cette conclusion causale peut-elle être injustifiée ?

Exercice 0.0.8 (Choisir entre supervisé et non supervisé). Pour chaque situation, indiquez si elle relève plutôt de l'apprentissage supervisé ou non supervisé, et justifiez brièvement.

1. Prédire le prix d'un logement à partir de sa surface et de son quartier.
2. Regrouper des clients selon leurs habitudes d'achat sans catégorie connue à l'avance.
3. Déterminer si un e-mail est un spam à partir d'e-mails déjà étiquetés.

Collecte et Prétraitement des Données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Identifier** les principales stratégies de collecte de données (bases de données, *web scraping*, API) ;
- **Comprendre** l'importance de la représentativité d'un échantillon ;
- **Expliquer** la notion d'encodage des variables et pourquoi elle est indispensable ;
- **Appliquer** les techniques de nettoyage, d'encodage et de mise à l'échelle des données ;
- **Construire** un pipeline de prétraitement simple avec Scikit-Learn.

Si les données sont de mauvaise qualité, corrompues ou biaisées, les résultats produits par le modèle seront tout aussi défectueux, quelle que soit la sophistication de l'algorithme (*Garbage In, Garbage Out*).

- Principe fondamental de la Data Science

Un modèle de Machine Learning, aussi sophistiqué soit-il, ne peut produire des prédictions fiables qu'à partir de données de qualité. En effet, aucune finesse algorithmique ne peut compenser des données mal collectées, non représentatives ou mal préparées. Ce chapitre couvre l'ensemble du travail qui précède la modélisation à proprement parler (Figure 3 du chapitre 1, étapes 2 et 3) : où trouver les données, comment s'assurer qu'elles sont représentatives, puis comment les nettoyer, les encoder et les mettre à l'échelle avant de les transmettre à un algorithme.

1. Stratégies de collecte de données

Avant tout projet, il faut identifier la ou les sources permettant de constituer le jeu de données X, y introduit au chapitre précédent. Trois grandes familles de sources coexistent dans la pratique professionnelle.

1.1. Les bases de données

La source la plus courante en entreprise reste la base de données interne, qu'elle soit :

- **relationnelle (SQL)** : les données sont organisées en tables reliées par des clés (ex. PostgreSQL, MySQL) et interrogées via le langage **SQL** (*Structured Query Language*) ;
- **non-relationnelle (NoSQL)** : les données sont stockées sous une forme plus flexible — documents JSON (MongoDB), paires clé-valeur (Redis), ou graphes (Neo4j) — adaptée aux données peu structurées ou à très grande échelle.

1.2. Le web scraping

Lorsque les données ne sont disponibles que sur des pages web, on a recours au **web scraping** : l'extraction automatisée d'informations à partir du code HTML de sites web. Les bibliothèques Python de référence sont :

- **BeautifulSoup** : idéale pour analyser (*parser*) le HTML d'une page unique et en extraire des éléments ciblés (balises, attributs) ;
- **Scrapy** : un *framework* complet, conçu pour des collectes à grande échelle sur de nombreuses pages, avec gestion native de la pagination, des files d'attente de requêtes et de l'exportation des résultats.

⚠ Éthique du web scraping

Le *web scraping* n'est pas toujours autorisé. Avant de collecter des données sur un site, il convient de vérifier :

- le fichier `robots.txt` du site, qui précise les zones autorisées ou interdites aux robots ;
- les conditions générales d'utilisation (CGU) du site, qui peuvent explicitement interdire la collecte automatisée ;
- le cadre réglementaire applicable à la protection des données personnelles (ex. loi marocaine 09-08, ou RGPD pour les données concernant des ressortissants européens), en particulier si des données à caractère personnel sont collectées.

1.3. Les API

Une **API** (*Application Programming Interface*) est une interface mise à disposition par un fournisseur de données, qui permet d'obtenir des données structurées de façon normalisée — sans avoir à analyser du HTML. On distingue notamment :

- les **API REST**, les plus répandues, où chaque ressource est accessible via une URL et une méthode HTTP (GET, POST...), avec une réponse généralement au format JSON ;
- les **API GraphQL**, plus récentes, qui permettent au client de préciser exactement les champs qu'il souhaite récupérer en une seule requête, réduisant ainsi le volume de données transférées inutilement.

TABLE 1 – Sources de données courantes

Source	Exemple
Bases de données	SQL (PostgreSQL, MySQL), NoSQL (MongoDB)
Web Scraping	BeautifulSoup, Scrapy
APIs	REST, GraphQL

2. Représentativité et biais statistiques

Collecter des données ne suffit pas : encore faut-il qu'elles soient **représentatives** de la population sur laquelle le modèle sera utilisé en production. Un échantillon est représentatif lorsque sa distribution reflète fidèlement celle de la population cible.

! À retenir

Un échantillon non représentatif introduit un **bias** qui se propage à toutes les étapes du modèle, même si l'algorithme est parfaitement implémenté. Aucune technique de modélisation, aussi avancée soit-elle, ne peut corriger un biais présent dès la collecte des données.

On distingue notamment trois types de biais fréquents en pratique :

- **Biais de sélection** : l'échantillon collecté ne couvre pas correctement certains sous-groupes de la population (ex. un sondage de satisfaction réalisé uniquement auprès des clients ayant un compte en ligne exclut de fait les clients moins digitalisés).
- **Biais de couverture temporelle** : les données ne couvrent qu'une période particulière, non représentative du comportement général (ex. collecter des données de vente uniquement pendant une période de soldes).
- **Biais de mesure** : l'instrument ou le protocole de collecte lui-même déforme la donnée (ex. un capteur mal calibré, un formulaire dont la formulation oriente la réponse).

Exemple historique classique

Lors de l'élection présidentielle américaine de 1936, le magazine *Literary Digest* prédit, sur la base de plus de 2 millions de réponses à un sondage, la victoire du candidat Alf Landon. Le sondage fut envoyé par la poste à partir de listes d'abonnés au téléphone et de propriétaires de voitures — une population nettement plus aisée que la moyenne en pleine Grande Dépression. Malgré une taille d'échantillon considérable, le sondage souffrait d'un **biais de sélection** majeur, et la prédiction se révéla fausse : Franklin D. Roosevelt fut réélu avec une large majorité. Cet exemple illustre qu'un grand échantillon **non représentatif** reste moins fiable qu'un échantillon plus petit mais correctement constitué.

3. Nettoyage des données

Une fois les données collectées, il est rare qu'elles soient directement exploitables : valeurs manquantes, doublons, incohérences et valeurs aberrantes sont la norme plutôt que l'exception.

3.1. Les valeurs manquantes

Une valeur manquante peut avoir des origines très différentes, ce qui conditionne la stratégie à adopter :

- **MCAR** (*Missing Completely At Random*) : l'absence de la valeur ne dépend d'aucune variable, observée ou non (ex. un bug technique aléatoire lors de l'enregistrement).
- **MAR** (*Missing At Random*) : l'absence dépend d'une autre variable observée (ex. les hommes répondent moins souvent à une question sur le poids, mais indépendamment du poids lui-même).
- **MNAR** (*Missing Not At Random*) : l'absence dépend de la valeur manquante elle-même (ex. les personnes aux revenus les plus élevés refusent plus souvent de déclarer leur revenu).

Remarque

Le cas **MNAR** est le plus problématique : ignorer ou imputer naïvement ces valeurs manquantes peut introduire un biais systématique difficile à corriger, car l'absence de la donnée est elle-même porteuse d'information.

Stratégies de traitement des valeurs manquantes :

TABLE 2 – Stratégies d'imputation des valeurs manquantes

Stratégie	Principe	Limite
Suppression des lignes	Retirer toute observation comportant une valeur manquante	Perte d'information, biaise l'échantillon si les valeurs manquent de façon non aléatoire
Imputation simple	Remplacer par la moyenne, la médiane (numérique) ou le mode (catégoriel)	Réduit artificiellement la variance de la variable

💡 Règle d'or

Pour éviter la fuite de données (*data leakage*), la stratégie d'imputation (moyenne, médiane...) doit être **calculée uniquement sur le jeu d'entraînement**, puis appliquée telle quelle aux jeux de validation et de test — sous peine de fuite de données (notion introduite dans le chapitre précédent, §4.2).

3.2. Les valeurs aberrantes (*outliers*)

Une valeur aberrante est une observation qui s'écarte fortement du comportement général des données — parfois une erreur de saisie, parfois une observation légitime mais extrême. Deux méthodes de détection sont couramment enseignées :

- **La méthode de l'écart interquartile (IQR)** : toute valeur en dehors de l'intervalle $[Q_1 - 1,5 \times IQR, Q_3 + 1,5 \times IQR]$ est considérée comme aberrante, où $IQR = Q_3 - Q_1$.
- **Le score-z** : toute valeur dont le score $z = \frac{x - \mu}{\sigma}$ dépasse un seuil (souvent $|z| > 3$) est considérée comme aberrante ; cette méthode suppose implicitement une distribution proche de la loi normale.

🔥 Une valeur aberrante n'est pas toujours une erreur

Avant de supprimer systématiquement les valeurs aberrantes détectées, il faut se demander si elles reflètent une erreur de mesure (à corriger ou supprimer) ou un phénomène réel mais rare (par exemple, un très gros client dans une base de transactions commerciales) — auquel cas leur suppression pourrait faire perdre au modèle une information précieuse.

3.3. Les doublons

Un doublon (deux lignes strictement identiques, ou identiques sur les variables clés) doit généralement être supprimé, car il donne artificiellement plus de poids à une observation dans l'apprentissage — un risque de biais parfois qualifié de « double comptage ».

4. Encodage des variables catégorielles

La quasi-totalité des algorithmes de Machine Learning (dont l'ensemble de ceux étudiés dans ce cours) reposent sur des opérations **mathématiques** — distances, produits scalaires, sommes pondérées — qui ne peuvent s'appliquer qu'à des **nombres**. Or de nombreuses variables sont, par nature, catégorielles (un quartier, une couleur, une catégorie socio-professionnelle). L'**encodage** désigne l'ensemble des techniques permettant de transformer ces variables catégorielles en une représentation numérique exploitable par un algorithme, **sans dénaturer l'information** qu'elles portent.

4.1. Encodage ordinal (*Ordinal Encoding*)

Chaque catégorie est remplacée par un entier (0, 1, 2...). Cette méthode est adaptée **uniquement** aux variables **ordinales**, c'est-à-dire dont les catégories possèdent un ordre naturel (ex. **Faible** < **Moyen** < **Élevé**). Dans Scikit-Learn, on utilise `OrdinalEncoder` pour les variables explicatives ; `LabelEncoder` est plutôt réservé à la variable cible.

⚠ Piège fréquent

Appliquer un encodage ordinal à une variable **nominale** (sans ordre naturel, ex. `quartier` : Centre, Nord, Sud) introduit un ordre artificiel que l'algorithme interprétera à tort comme porteur de sens (il pourrait, par exemple, en déduire que `Sud` (codé 2) est « plus grand » que `Centre` (codé 0), ce qui n'a aucune signification réelle).

4.2. Encodage One-Hot (*One-Hot Encoding*)

Pour les variables **nominales**, on utilise plutôt l'encodage *one-hot* : chaque catégorie devient une **nouvelle colonne binaire** (0 ou 1), sans introduire d'ordre artificiel.

Par exemple, la variable `quartier` (Centre, Nord, Sud) devient :

TABLE 3 – Encodage One-Hot de la variable `quartier`

<code>quartier_Centre</code>	<code>quartier_Nord</code>	<code>quartier_Sud</code>
1	0	0
0	1	0
0	0	1

i Le piège de la variable factice (Dummy Variable Trap)

Lorsqu'une variable comporte k catégories, il suffit en réalité de $k - 1$ colonnes binaires pour représenter toute l'information : la dernière catégorie est déductible des autres (si `quartier_Centre` = 0 et `quartier_Nord` = 0, alors nécessairement `quartier_Sud` = 1). Conserver les k colonnes introduit une redondance parfaite entre variables (colinéarité), problématique pour certains algorithmes comme la régression linéaire (chapitre 3). L'argument `drop='first'` de `OneHotEncoder` permet de supprimer automatiquement cette colonne redondante.

4.3. Choisir une méthode simple

TABLE 4 – Comparaison des principales méthodes d'encodage

Méthode	Adaptée à	Nombre de colonnes créées
Ordinal Encoding	Variables ordinales	1 (inchangé)
One-Hot Encoding	Variables nominales , peu de catégories	k (ou $k - 1$)

Dans ce cours, on privilégie donc deux solutions faciles à vérifier : l'encodage ordinal pour les catégories ordonnées et l'encodage one-hot pour les catégories nominales. Les encodages destinés aux variables comportant un très grand nombre de catégories seront étudiés seulement dans un cours plus avancé.

5. Mise à l'échelle : Standardisation vs Normalisation

De nombreux algorithmes — en particulier ceux fondés sur une notion de **distance** entre observations (k plus proches voisins, K-means, chapitre 5) ou sur une **descente de gradient** (régression linéaire, réseaux de neurones) — sont sensibles à l'échelle des variables. Une variable exprimée en milliers de dirhams (ex. un prix) dominera artificiellement une variable exprimée en unités (ex. un nombre de pièces), même si cette dernière est tout aussi pertinente.

5.1. La standardisation (*Z-score scaling*)

La **standardisation** centre chaque variable autour d'une moyenne de 0 avec un écart-type de 1 :

$$x' = \frac{x - \mu}{\sigma} \quad (0.1)$$

où μ est la moyenne et σ l'écart-type de la variable, **calculés sur le jeu d'entraînement uniquement**.

5.2. La normalisation (*Min-Max scaling*)

La **normalisation** ramène chaque variable dans un intervalle fixe, généralement $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (0.2)$$

5.3. Quand utiliser l'une ou l'autre ?

TABLE 5 – Choix entre standardisation et normalisation

Critère	Standardisation (Équation 0.1)	Normalisation (Équation 0.2)
Sensibilité aux valeurs aberrantes	Modérée	Élevée (les valeurs extrêmes déterminent $x_{\min}/x_{\max}$)
Hypothèse sur la distribution	Aucune (mais adaptée aux distributions proches de la normale)	Aucune
Intervalle de sortie	Non borné (généralement $[-3, 3]$ environ)	Borné, $[0, 1]$
Cas d'usage typique	Algorithmes basés sur le gradient, PCA, régression	Réseaux de neurones (fonctions d'activation bornées), image (pixels)

Le choix entre la Équation 0.1 et la Équation 0.2 dépend donc de l'algorithme cible et de l'échelle des variables. En présence d'*outliers* marqués, il faut d'abord vérifier s'ils sont des erreurs ou des observations réelles avant de choisir une transformation.

⚠ Règle d'or (rappel du chapitre précédent)

Comme pour l'imputation (Table 2), les paramètres de mise à l'échelle (μ , σ , $x_{\min}$, $x_{\max}$) doivent être **estimés uniquement sur le jeu d'entraînement** (méthode `.fit()`), puis appliqués tels quels aux jeux de validation et de test (méthode `.transform()`) — toute autre pratique constitue une fuite de données.

6. Construire un pipeline de prétraitement avec les outils modernes de Python

En pratique, un même jeu de données combine généralement des variables numériques et catégorielles, chacune nécessitant un traitement distinct. Les bibliothèques modernes de l'écosystème Python permettent d'**enchaîner** et d'**automatiser** l'ensemble de ces étapes, plutôt que de les appliquer manuellement une à une.

6.1. ColumnTransformer et Pipeline de Scikit-Learn

Scikit-Learn propose deux briques complémentaires pour construire un prétraitement reproductible et sans fuite de données :

- **Pipeline** : enchaîne plusieurs transformations (par ex. imputation puis mise à l'échelle) appliquées à **toutes** les colonnes d'un même sous-ensemble ;
- **ColumnTransformer** : applique des **Pipeline différents** selon le type de colonne (numérique ou catégorielle), puis **fusionne** les résultats en une seule matrice.

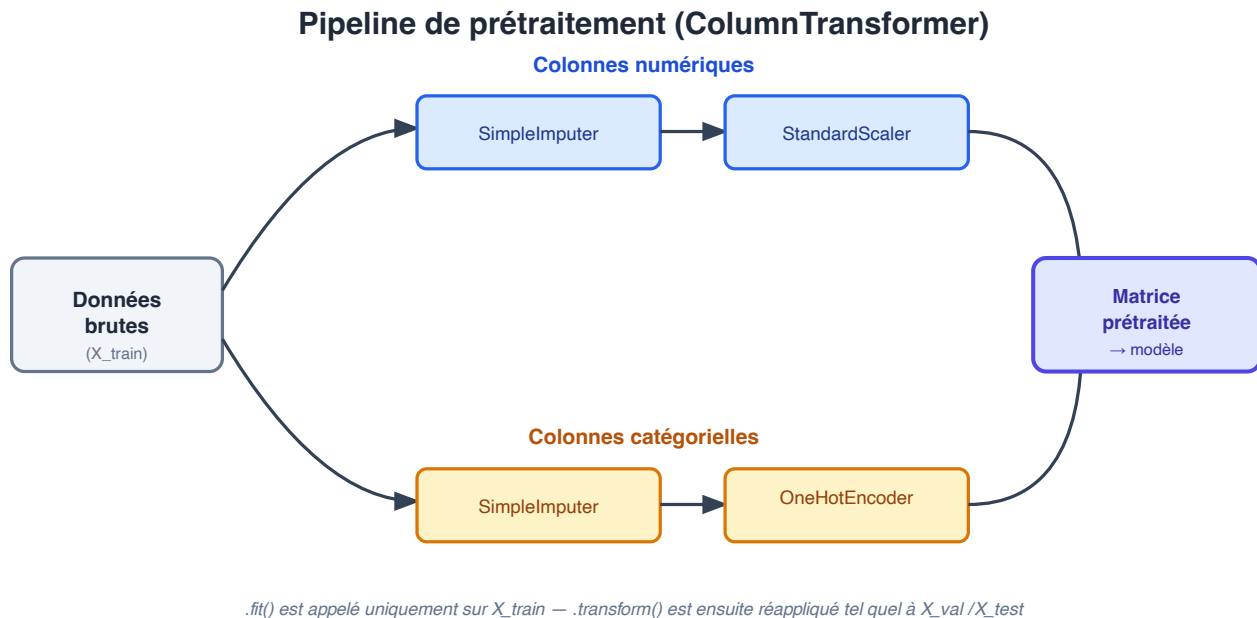


FIGURE 1 – Architecture d'un pipeline de prétraitement combinant colonnes numériques et catégorielles

La Figure 1 illustre cette architecture : les colonnes numériques suivent une chaîne `SimpleImputer` → `StandardScaler`, tandis que les colonnes catégorielles suivent une chaîne `SimpleImputer` → `OneHotEncoder`, avant que les deux résultats ne soient recombinaés en une seule matrice, transmise au modèle.

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

colonnes_numeriques = ["surface", "nb_pieces"]
colonnes_catégorielles = ["quartier"]

```

```

pipeline_numerique = Pipeline([
    ("imputation", SimpleImputer(strategy="median")),
    ("mise_a_l_echelle", StandardScaler()),
])

pipeline_categorielle = Pipeline([
    ("imputation", SimpleImputer(strategy="most_frequent")),
    ("encodage", OneHotEncoder(drop="first", handle_unknown="ignore")),
])

pretraitement = ColumnTransformer([
    ("num", pipeline_numerique, colonnes_numeriques),
    ("cat", pipeline_categorielle, colonnes_categorielles),
])

# Fit UNIQUEMENT sur les données d'entraînement (voir §5.3 et le chapitre précédent, §4.2)
X_train_pret = pretraitement.fit_transform(X_train)
# Le même prétraitement, déjà appris, est simplement réappliqué :
X_test_pret = pretraitement.transform(X_test)

```

💡 Fonctionnalité récente : sortie en DataFrame

Depuis les versions récentes de Scikit-Learn, l'appel `pretraitement.set_output(transform="pandas")` permet d'obtenir directement un **DataFrame pandas** en sortie du pipeline (avec des noms de colonnes explicites comme `num__surface` ou `cat__quartier_Nord`), plutôt qu'un simple tableau **numpy** sans noms — un gain de lisibilité appréciable lors du diagnostic d'un pipeline.

6.2. Mettre à jour l'environnement du projet

En cohérence avec la gestion de projet introduite au chapitre 1 (§4.3), les nouvelles dépendances nécessaires à ce chapitre sont ajoutées explicitement au projet :

```

uv add scikit-learn
uv lock # fige les versions exactes dans uv.lock, pour la reproductibilité

```

Exercices

i Consigne

Ces exercices sont **théoriques** : ils testent votre compréhension des concepts du chapitre, sans nécessiter d'écrire de code. La mise en pratique sur des jeux de données réels avec Scikit-Learn fera l'objet du TP associé à ce chapitre. Les corrigés sont fournis juste après chaque énoncé, dans un bloc repliable.

Exercice 0.0.1 (Choisir une source de données). Pour chacun des besoins suivants, indiquez la stratégie de collecte la plus appropriée (base de données, *web scraping*, ou API) et justifiez en une phrase.

- Récupérer en temps réel le cours de change EUR/MAD.
- Constituer un historique des prix de l'immobilier à partir d'annonces publiées sur un site sans API publique.

- c. Extraire l'historique des commandes déjà stockées dans le système d'information interne de l'entreprise.

Exercice 0.0.2 (Identifier un biais). Une application mobile de course à pied souhaite entraîner un modèle pour prédire le temps qu'un coureur mettra sur un semi-marathon. Les données d'entraînement proviennent exclusivement des utilisateurs ayant activé le partage de leurs statistiques sur les réseaux sociaux.

- a. Quel type de biais cette collecte introduit-elle probablement ?
- b. Le modèle entraîné sur cet échantillon sera-t-il fiable pour prédire le temps d'un coureur occasionnel qui ne partage jamais ses performances ?

Exercice 0.0.3 (Nature d'une valeur manquante). Dans une enquête de satisfaction, on observe que les clients ayant donné une note très basse au service client laissent beaucoup plus souvent la question « Recommanderiez-vous ce service ? » sans réponse que les autres clients.

- a. À quelle catégorie (MCAR, MAR, MNAR) ce type d'absence appartient-il le plus probablement ?
- b. Pourquoi une imputation simple par le mode (la réponse la plus fréquente) serait-elle risquée dans ce cas précis ?

Exercice 0.0.4 (Choisir un type d'encodage). Pour chacune des variables suivantes, indiquez si un *Label/Ordinal Encoding* ou un *One-Hot Encoding* est le plus adapté, et justifiez.

- a. Le niveau de scolarité d'un client : Primaire, Secondaire, Supérieur.
- b. La couleur préférée d'un client : Rouge, Bleu, Vert, Jaune.
- c. Le mois de la transaction (Janvier, ..., Décembre).

Exercice 0.0.5 (Standardisation ou normalisation?). Un jeu de données contient une variable `revenu_annuel` dont la quasi-totalité des valeurs se situent entre 30 000 et 200 000 MAD, à l'exception de trois valeurs extrêmes dépassant 5 000 000 MAD (probablement des grandes fortunes, mais authentiques).

- a. Que se passerait-il si l'on appliquait une **normalisation** (*Min-Max scaling*) à cette variable en l'état ?
- b. Quelle alternative, parmi celles évoquées dans ce chapitre, serait plus robuste face à ces valeurs extrêmes ?

Exercice 0.0.6 (Repérer une fuite de données dans un prétraitement). Un étudiant applique la démarche suivante :

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)           # sur l'ensemble des données
X_train, X_test = train_test_split(X_scaled, test_size=0.2)
```

- a. Ce code contient-il une fuite de données ? Justifiez.
- b. Proposez, en français (sans code), la démarche correcte à suivre.

Exercice 0.0.7 (Rôle du `ColumnTransformer`). Un jeu de données comporte à la fois des variables numériques (`surface`, `age_bien`) et catégorielles (`quartier`, `type_bien`).

- a. Pourquoi ne peut-on pas appliquer un unique `Pipeline` (imputation puis `StandardScaler`) à l'ensemble des colonnes sans distinction ?
- b. Quel est le rôle précis du `ColumnTransformer` dans ce contexte ?

Exercice 0.0.8 (Doublons et double comptage). Une équipe fusionne deux exports de données provenant de deux systèmes différents de l'entreprise, contenant chacun une partie commune de clients. Après fusion, certains clients apparaissent deux fois avec des valeurs identiques.

- a. Quel est le risque si ces doublons ne sont pas détectés avant l'entraînement du modèle ?
- b. Cela affecte-t-il davantage un algorithme de classification ou serait-ce sans conséquence ?

Régression

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- distinguer clairement une tâche de régression d'une tâche de classification ;
- formuler une régression linéaire simple et multiple ;
- interpréter les coefficients, les résidus et les prédictions ;
- calculer les paramètres d'une régression par les moindres carrés ordinaires ;
- expliquer le principe de la descente de gradient ;
- choisir des métriques adaptées à un problème de régression ;
- utiliser `scikit-learn` pour ajuster un modèle linéaire simple ;
- diagnostiquer les limites d'un modèle de régression.

1. Régression ou classification ?

La **régression** et la **classification** sont deux tâches d'apprentissage supervisé. Dans les deux cas, on dispose d'observations décrites par des variables explicatives X et d'une cible connue y . La différence essentielle est la nature de la cible.

Aspect	Régression	Classification
Nature de la cible y	Numérique, continue	Catégorielle
Exemple	Prix, température, durée	Spam / non spam, animal
Sortie du modèle	Une valeur réelle $\hat{y}$	Une classe ou une probabilité
Métriques courantes	MAE, MSE, RMSE, R^2	Accuracy, précision, rappel, F1
Question posée	« Combien ? »	« Quelle catégorie ? »

1.1. Régression

On cherche à approximer une fonction inconnue f telle que :

$$y = f(x) + \varepsilon$$

Ici, ε représente le bruit, c'est-à-dire la part imprévisible du phénomène. Le modèle apprend une approximation $\hat{f}$ et produit :

$$\hat{y} = \hat{f}(x)$$

Exemples :

- prévoir le prix d'un appartement ;
- estimer la consommation d'électricité ;
- prédire le temps de livraison ;
- estimer la quantité mensuelle.

Une régression est dite **simple** si elle utilise une seule variable explicative, et **multiple** si elle en utilise plusieurs.

1.2. Classification

En classification, la cible appartient à un ensemble fini de catégories. Par exemple, pour une classification binaire :

$$y \in \{0, 1\}$$

Le modèle peut produire une probabilité :

$$\hat{p}(y = 1 \mid x)$$

Puis, selon un seuil, il choisit une classe. Ainsi, une probabilité de 0,82 peut conduire à la classe « oui » si le seuil est fixé à 0,5.

La distinction est donc claire : en régression, on cherche une valeur numérique ; en classification, on cherche une catégorie.

! La frontière n'est pas toujours absolue

Une variable numérique peut parfois être utilisée comme cible d'une régression même si elle est discrète, comme le nombre de commandes ou le nombre d'accidents. La vraie question est de savoir si l'objectif consiste à prédire une quantité numérique ou une catégorie.

2. Données et notation

Chaque observation i est décrite par un vecteur de p variables explicatives :

$$x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$$

et par une valeur cible numérique y_i . Pour n observations, on écrit :

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \in \mathbb{R}^{n \times p}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n$$

Les quantités importantes sont :

Symbole	Signification
n	Nombre d'observations
p	Nombre de variables explicatives
X	Matrice des caractéristiques
x_i	Vecteur des caractéristiques de l'observation i
y	Vecteur des réponses observées
$\hat{y}_i$	Prédiction du modèle pour l'observation i
$e_i = y_i - \hat{y}_i$	Résidu de l'observation i

Le rôle du modèle est d'apprendre à associer des valeurs de x à des valeurs de y en minimisant l'erreur de prédiction.

i Paramètre, hyperparamètre et prédiction

Dans un modèle linéaire, les coefficients $\beta_0, \beta_1, \dots$ sont des **paramètres** appris sur les données. Un **hyperparamètre** est choisi avant l'apprentissage, comme le nombre d'itérations d'un algorithme. Une **prédiction** $\hat{y}$ est la valeur calculée par le modèle à partir des paramètres appris.

3. Modèle de régression linéaire

3.1. Régression linéaire simple

Avec une seule variable explicative x , le modèle linéaire s'écrit :

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

- β_0 est l'ordonnée à l'origine ;
- β_1 est la pente ;
- $\hat{y}_i$ est la valeur prédite.

On peut aussi écrire le modèle statistique :

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

Le terme ε_i tient compte de l'écart entre la relation moyenne et les observations réelles. Le but n'est pas d'ajuster une droite parfaite, mais une tendance utile.

3.2. Régression linéaire multiple

Avec p variables explicatives :

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}$$

Le coefficient β_j représente l'effet moyen d'une variation d'une unité de x_j sur la cible, en gardant les autres variables constantes.

Par exemple, pour le prix d'un appartement :

$$\widehat{\text{prix}} = \beta_0 + \beta_1 \times \text{surface} + \beta_2 \times \text{nombre de chambres}$$

Si $\beta_1 = 1200$, alors une surface supplémentaire de 1 m^2 est associée, toutes choses égales par ailleurs, à une augmentation moyenne de 1200 de la valeur prédite.

3.3. Écriture matricielle

Pour inclure l'interception, on ajoute une colonne de 1 à X . On obtient :

$$\widetilde{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix} \in \mathbb{R}^{n \times (p+1)}$$

Le vecteur des coefficients est :

$$\beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} \in \mathbb{R}^{p+1}$$

Le modèle devient :

$$\hat{y} = \widetilde{X}\beta$$

Cette écriture compacte est très utile pour le calcul et la compréhension mathématique du modèle.

3.4. Fonction de coût

Le résidu de l'observation i est :

$$e_i = y_i - \hat{y}_i$$

Pour mesurer l'erreur globale, on utilise souvent la somme des carrés des erreurs :

$$SSE(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

ou la moyenne des carrés des erreurs :

$$MSE(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Le modèle linéaire cherche les coefficients qui minimisent cette quantité :

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} MSE(\beta)$$

Le carré fait deux choses :

1. les erreurs positives et négatives ne s'annulent pas ;
2. les grandes erreurs sont pénalisées davantage que les petites.

3.5. Hypothèses et limites

La régression linéaire reste utile même si toutes ses hypothèses ne sont pas parfaitement vérifiées. Mais il faut connaître les conditions qui soutiennent son interprétation :

Hypothèse	Signification
Linéarité	La relation moyenne entre x et y est approximativement linéaire
Indépendance	Les observations ne dépendent pas les unes des autres
Homoscédasticité	La dispersion des erreurs est à peu près constante
Absence de multicollinéarité forte	Les variables explicatives ne sont pas presque redondantes
Normalité des erreurs	Utile pour certains intervalles de confiance, pas indispensable pour la prédiction

Une relation non linéaire peut souvent être approchée par des transformations, comme x^2 , $\log(x)$ ou des interactions. Mais si le modèle devient trop complexe, il peut sur-apprendre.

Extrapolation

Une droite peut avoir une bonne performance autour des données observées, mais devenir peu fiable en dehors de cette plage. Une prédiction faite très loin des valeurs d'entraînement est une extrapolation.

4. Calcul des paramètres par les MCO

4.1. Principe

Les **moindres carrés ordinaires** (MCO) choisissent les coefficients qui minimisent la somme des carrés des résidus :

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

En notation matricielle :

$$J(\beta) = \|y - \tilde{X}\beta\|_2^2 = (y - \tilde{X}\beta)^\top (y - \tilde{X}\beta)$$

4.2. Cas de la régression simple

Pour :

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

les coefficients MCO sont :

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

et

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Ici, $\bar{x}$ et $\bar{y}$ sont les moyennes de x et de y .

La pente mesure la variation conjointe de x et y , normalisée par la variation de x . Si toutes les valeurs de x sont identiques, la pente ne peut pas être estimée.

4.3. Exemple simple

Considérons les observations :

x_i	y_i
1	2
2	3
3	5

Les moyennes sont :

$$\bar{x} = 2, \quad \bar{y} = \frac{10}{3}$$

4. Calcul des paramètres par les MCO

Le numérateur de la pente vaut :

$$\sum_i (x_i - \bar{x})(y_i - \bar{y}) = (-1)(-\frac{4}{3}) + 0(-\frac{1}{3}) + 1(\frac{5}{3}) = 3$$

Le dénominateur vaut :

$$\sum_i (x_i - \bar{x})^2 = (-1)^2 + 0^2 + 1^2 = 2$$

Donc :

$$\hat{\beta}_1 = \frac{3}{2} = 1,5$$

et :

$$\hat{\beta}_0 = \frac{10}{3} - 1,5 \times 2 = \frac{1}{3}$$

La droite ajustée est donc :

$$\hat{y} = \frac{1}{3} + 1,5x$$

Les prédictions sont :

$$\left(\frac{11}{6}, \frac{10}{3}, \frac{29}{6} \right)$$

et les résidus sont :

$$\left(\frac{1}{6}, -\frac{1}{3}, \frac{1}{6} \right)$$

La somme des résidus est nulle, comme c'est souvent le cas lorsque le modèle contient une ordonnée à l'origine.

4.4. Solution matricielle

En annulant le gradient de la fonction de coût, on obtient les **équations normales** :

$$\tilde{X}^\top \tilde{X} \hat{\beta} = \tilde{X}^\top y$$

Si la matrice $\tilde{X}^\top \tilde{X}$ est inversible, alors :

$$\boxed{\hat{\beta} = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top y}$$

Cette formule est centrale en régression linéaire. En pratique, on évite souvent de calculer explicitement l'inverse. Les bibliothèques numériques utilisent des méthodes plus stables, comme QR ou SVD.

Les cas où la solution n'est pas unique ou n'est pas stable se produisent lorsque :

- une variable est une combinaison exacte d'autres variables ;
- deux colonnes sont redondantes ;
- le nombre de variables est trop élevé par rapport au nombre d'observations ;
- les variables sont fortement corrélées.

i MCO, corrélation et causalité

Une régression donne une association entre variables et cible, pas automatiquement un effet causal. Un coefficient important ne prouve pas qu'une variable est la cause de la variation observée de y .

5. La descente de gradient

La formule MCO donne une solution directe pour la régression linéaire, mais de nombreux modèles n'ont pas de solution analytique simple. La **descente de gradient** propose une méthode générale d'optimisation.

L'idée est simple :

1. choisir des paramètres initiaux ;
2. calculer la fonction de coût ;
3. calculer le gradient ;
4. se déplacer dans la direction opposée au gradient ;
5. répéter jusqu'à convergence.

Pour la régression linéaire :

$$J(\beta) = \frac{1}{n} \|y - \tilde{X}\beta\|_2^2$$

le gradient est :

$$\nabla J(\beta) = \frac{2}{n} \tilde{X}^\top (\tilde{X}\beta - y)$$

La mise à jour est alors :

$$\beta^{(t+1)} = \beta^{(t)} - \eta \nabla J(\beta^{(t)})$$

avec $\eta > 0$ le **taux d'apprentissage**.

5.1. Interprétation du taux d'apprentissage

Valeur de η	Comportement
Trop petite	Convergence très lente
Bien choisie	Réduction régulière du coût
Trop grande	Oscillations ou divergence

Pour une régression linéaire, la fonction de coût est convexe. Quand le problème est bien posé, elle a un minimum global unique. Un taux trop grand peut empêcher l'algorithme d'atteindre ce minimum.

5.2. MCO ou descente de gradient ?

Critère	MCO	Descente de gradient
Type de solution	Directe	Itérative
Idéal pour	Régression linéaire	Modèles plus complexes
Hyperparamètre important	Peu nombreux	Taux d'apprentissage
Coût	Très efficace sur les petits problèmes	Souvent plus lent

La régression linéaire de `scikit-learn` utilise généralement des méthodes de moindres carrés, pas une descente de gradient explicite.

6. Évaluer une régression

Une métrique doit refléter le coût réel des erreurs. Deux modèles peuvent avoir une RMSE similaire mais des comportements très différents.

6.1. MAE

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

La MAE mesure l'erreur moyenne en valeur absolue. Elle est exprimée dans la même unité que la cible.

6.2. MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

La MSE pénalise fortement les grandes erreurs. Elle est utile lorsque des erreurs importantes sont particulièrement coûteuses.

6.3. RMSE

$$RMSE = \sqrt{MSE}$$

La RMSE garde la même unité que y , tout en continuant à pénaliser les erreurs importantes.

6.4. Coefficient de détermination R^2

On définit :

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

Alors :

$$R^2 = 1 - \frac{SSE}{SST}$$

R^2 compare le modèle à une baseline qui prédit toujours la moyenne $\bar{y}$.

- $R^2 = 1$: prédictions parfaites ;
- $R^2 = 0$: le modèle n'est pas meilleur qu'une prédiction constante ;
- $R^2 < 0$: le modèle est pire que cette baseline.

6.5. Exemple

Pour :

$$y = (3, 5, 7), \quad \hat{y} = (2, 6, 8)$$

les erreurs sont $(1, -1, -1)$ et les erreurs absolues sont $(1, 1, 1)$. Donc :

$$MAE = 1, \quad MSE = 1, \quad RMSE = 1$$

Comme $\bar{y} = 5$, on a :

$$SST = (3 - 5)^2 + (5 - 5)^2 + (7 - 5)^2 = 8$$

$$SSE = 1^2 + (-1)^2 + (-1)^2 = 3$$

D'où :

$$R^2 = 1 - \frac{3}{8} = 0,625$$

6.6. Choisir une métrique

Situation	Métrique à privilégier
Coût proportionnel à l'erreur	MAE
Erreurs très grandes particulièrement coûteuses	MSE ou RMSE
Comparer au modèle de référence à la moyenne	R^2

! Évaluer sur le bon jeu

Les métriques doivent être calculées sur des données que le modèle n'a pas vues pendant l'apprentissage. La métrique d'entraînement mesure la capacité d'ajustement ; la métrique de validation ou de test mesure la généralisation.

7. Utilisation avec `scikit-learn`

La plupart des modèles de régression de `scikit-learn` partagent une interface simple :

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X = [[120], [150], [170], [210], [240], [280]]
y = [1500, 1800, 2100, 2500, 2900, 3400]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = mean_squared_error(y_test, predictions, squared=False)
r2 = r2_score(y_test, predictions)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
```

Le schéma général est :

1. définir X et y ;
2. séparer les données en entraînement et test ;
3. créer un estimateur ;
4. appeler `.fit(X_train, y_train)` ;
5. appeler `.predict(X_test)` ;
6. calculer une métrique sur les données de test.

7.1. Modèle linéaire avec plusieurs variables

On peut aussi utiliser plusieurs variables explicatives :

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [120, 2],
    [150, 3],
    [170, 3],
    [210, 4],
    [240, 5],
], dtype=float)
y = np.array([1500, 1800, 2100, 2500, 2900], dtype=float)

model = LinearRegression()
model.fit(X, y)
```

```
print(model.intercept_)
print(model.coef_)
```

Ici, `intercept_` est β_0 et `coef_` contient les coefficients $\beta_1, \beta_2, \dots$

7.2. Pipeline simple

Quand on a des données incomplètes ou des variables de types différents, il faut souvent préparer les données avant de les donner au modèle. `Pipeline` permet de regrouper les étapes.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression

num_features = ["surface", "age"]
cat_features = ["quartier"]

num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", num_pipeline, num_features),
    ("cat", cat_pipeline, cat_features),
])

model = Pipeline([
    ("preprocessing", preprocessor),
    ("regression", LinearRegression()),
])
```

Le point important est que les statistiques de préparation sont calculées sur les données d'entraînement uniquement. Cela évite la fuite de données.

7.3. Limites de l'approche simple

La régression linéaire est puissante, mais elle n'est pas adaptée à toute situation :

- relation fortement non linéaire ;
- variables très corrélées ;
- présence de variables qualitatives nombreuses ;
- phénomène avec des effets non additifs.

Dans ces cas, il faut parfois transformer les variables, ajouter des caractéristiques, ou utiliser d'autres familles de modèles. Mais la logique centrale reste la même : expliquer une cible numérique par des variables explicatives.

8. Exercices théoriques

Les exercices suivants sont purement théoriques. Ils servent à consolider les idées, sans demander de code exécutable.

Exercice 1. Identifier le type de problème

Pour chaque situation, indique si l'on parle d'une tâche de régression ou de classification :

1. Prédire la température maximale demain.
2. Déterminer si un email est un spam ou non.
3. Estimer le montant d'une facture.
4. Classer une image en "chien" ou "chat".
5. Prédire le nombre d'accidents pendant un mois.

Exercice 2. Interpréter un coefficient

On considère le modèle :

$$\hat{y} = 50 + 2x_1 - 3x_2$$

Interprète le sens de $\beta_1 = 2$ et $\beta_2 = -3$.

Exercice 3. Calculer une régression simple

On donne les observations suivantes :

x_i	y_i
1	2
2	5
3	6

Calculez la pente et l'ordonnée à l'origine de la droite de régression par les MCO.

Exercice 4. Expliquer le résidu

Pourquoi le résidu $e_i = y_i - \hat{y}_i$ est-il important ? Donnez deux raisons.

Exercice 5. Choisir une métrique

Un responsable veut éviter les grandes erreurs de prévision, même si elles sont rares. Quelle métrique entre MAE, MSE et RMSE est la plus adaptée ? Justifiez.

Exercice 6. Vérifier les hypothèses

Donnez une situation où une régression linéaire risque d'être peu adaptée. Expliquez pourquoi.

Exercice 7. Comprendre le gradient

Expliquez en une phrase le rôle du gradient dans la descente de gradient.

Exercice 8. Comparer les deux approches

Comparez brièvement la méthode MCO et la descente de gradient pour la régression linéaire.

Classification

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- distinguer une classification binaire et une classification multiclasse ;
- formuler mathématiquement un problème de classification supervisée ;
- comprendre la régression logistique, sa fonction sigmoïde et ses probabilités ;
- interpréter une matrice de confusion et les principales métriques ;
- choisir un seuil de décision adapté au contexte ;
- construire une chaîne de classification simple avec `scikit-learn` ;
- résoudre des exercices théoriques sans programmation pratique.

i Lien avec le chapitre de régression

Ce chapitre repose sur les mêmes bases que le chapitre précédent : matrice X , vecteur cible y , séparation train/test, généralisation et fuite de données. On se concentre ici sur ce qui est particulier à la classification : classes, probabilités, seuils et métriques.

1. Qu'est-ce que la classification ?

En classification, on cherche à prédire une variable discrète : une classe, pas une valeur continue. La cible y appartient à un ensemble fini de catégories.

Un modèle apprend une fonction :

$$\hat{y} = h(x)$$

où x représente une observation et $\hat{y}$ la classe prédite.

1.1. Classification binaire

Dans une classification binaire, la cible ne prend que deux valeurs :

$$y \in \{0, 1\}$$

Exemples :

- 0 : e-mail normal, 1 : spam ;
- 0 : client fidèle, 1 : client qui part ;
- 0 : patient non malade, 1 : patient malade.

Le modèle produit souvent une probabilité :

$$\hat{p}(y = 1 \mid x)$$

Ensuite, on applique un seuil pour transformer cette probabilité en classe.

1.2. Classification multiclasse

En classification multiclasse, une observation appartient à l'une de plusieurs classes :

$$y \in \{1, 2, \dots, K\}$$

Exemples :

- reconnaître un chiffre manuscrit ;
- classer un animal dans une catégorie ;
- prédire une catégorie de produit.

Le modèle renvoie alors souvent un vecteur de probabilités sur les K classes.

1.3. Classification multilabel

Dans une classification multilabel, une observation peut appartenir à plusieurs classes à la fois. Une image peut contenir une voiture et une personne simultanément. On représente alors la cible par un vecteur binaire.

! Probabilité et classe ne sont pas la même chose

Une probabilité de 0,51 et une probabilité de 0,99 conduisent à la même classe si le seuil est 0,5. Mais elles ne représentent pas le même niveau de certitude. Il est donc utile d'étudier les probabilités, pas seulement les étiquettes finales.

2. Formalisation d'un problème de classification

La structure des données est la même que pour la régression :

$$X \in \mathbb{R}^{n \times p}$$

Chaque ligne de X correspond à une observation, et chaque colonne à une variable explicative. La différence importante est la cible :

$$y_i \in \{0, 1\}$$

dans le cas binaire.

Un classifieur cherche à apprendre une règle de décision :

$$\hat{y} = h(x)$$

en utilisant des observations déjà étiquetées.

Le modèle peut fournir :

1. une probabilité de classe ;
2. une classe finale après application d'un seuil.

La frontière de décision est l'ensemble des points pour lesquels le modèle hésite entre les classes. Dans le cas d'une régression logistique, cette frontière est un hyperplan.

3. Régression logistique

La régression logistique est un modèle de référence pour la classification binaire. Elle est simple, interprétable et souvent très efficace comme base de comparaison.

3.1. Score linéaire

Pour une observation $x = (x_1, \dots, x_p)$, le modèle calcule d'abord un score linéaire :

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

ou, en notation matricielle :

$$z = \tilde{x}^\top \beta$$

Le score z peut prendre n'importe quelle valeur réelle. Ce n'est pas encore une probabilité.

3.2. Fonction sigmoïde

On transforme ce score grâce à la fonction sigmoïde :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Cette fonction donne une valeur comprise entre 0 et 1. La probabilité prédite de la classe 1 est alors :

$$\boxed{\hat{p}(y = 1 \mid x) = \sigma(z)}$$

Quelques valeurs utiles :

Score z	Probabilité $\sigma(z)$
$-\infty$	proche de 0
0	0,5
$+\infty$	proche de 1

La dérivée de la sigmoïde est :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Cette propriété est utile pour l'optimisation par descente de gradient.

3.3. Règle de décision

On décide de la classe positive si la probabilité dépasse un seuil t :

$$\hat{y} = \begin{cases} 1 & \text{si } \hat{p}(y = 1 \mid x) \geq t \\ 0 & \text{sinon} \end{cases}$$

Pour $t = 0,5$, on a :

$$\sigma(z) \geq 0,5 \iff z \geq 0$$

Ainsi, le seuil 0,5 correspond simplement à la frontière :

$$\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p = 0$$

Le modèle sépare donc l'espace des variables par un hyperplan.

! Le seuil dépend du contexte

Le seuil de 0,5 est souvent utilisé, mais il ne convient pas forcément à tous les problèmes. Si un faux négatif est très coûteux, on peut baisser le seuil. Si les faux positifs sont excessivement coûteux, on peut le relever.

3.4. Interprétation des coefficients

Les coefficients β_j n'agissent pas directement sur la probabilité, mais sur le score z .

Le rapport entre probabilité et odds est donné par :

$$\text{logit}(p) = \ln \left(\frac{p}{1-p} \right)$$

Dans un modèle logistique :

$$\ln \left(\frac{p(y=1 | x)}{1-p(y=1 | x)} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Ainsi, une augmentation d'une unité de x_j ajoute β_j aux log-odds. Cela signifie que :

$$e^{\beta_j}$$

est le facteur multiplicatif des odds. Cette interprétation est utile, mais il ne faut pas conclure que la probabilité est multipliée par e^{β_j} .

3.5. Log loss ou entropie croisée binaire

Pour une observation i , on note :

$$p_i = \hat{p}(y_i = 1 | x_i)$$

La vraisemblance d'une observation est :

$$p_i^{y_i} (1 - p_i)^{1-y_i}$$

Cette expression vaut p_i si $y_i = 1$ et $1 - p_i$ si $y_i = 0$.

La log loss est la moyenne négative du log de cette vraisemblance :

$$J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)]$$

Cette fonction pénalise fortement les prédictions qui attribuent une probabilité très faible à la vraie classe. Elle évalue donc la qualité des probabilités, pas seulement la classe prédite.

3.6. Exemple simple

Considérons le modèle :

$$z = -1 + 0,5x$$

Pour $x = 2$:

$$z = -1 + 0,5 \times 2 = 0$$

donc :

$$p = \sigma(0) = \frac{1}{2} = 0,5$$

Le modèle est exactement à la frontière.

Pour $x = 4$:

$$z = -1 + 0,5 \times 4 = 1$$

et :

$$p = \frac{1}{1 + e^{-1}} \approx 0,731$$

Pour $x = 0$:

$$z = -1, \quad p = \frac{1}{1 + e} \approx 0,269$$

Ce modèle montre très bien comment un score linéaire est transformé en probabilité et comment le seuil de décision agit sur la classification.

4. Évaluer une classification binaire

La **matrice de confusion** constitue le point de départ de l'évaluation d'un modèle de classification binaire. Elle permet de comparer, pour chaque observation, sa **classe réelle** à la **classe prédite par le modèle**. Toutes les principales métriques présentées dans cette section sont directement construites à partir de ses quatre cases : vrais positifs, vrais négatifs, faux positifs et faux négatifs.

4.1. Matrice de confusion

Pour une classification binaire, on considère deux classes : une classe **positive** et une classe **négative**. La classe positive correspond à l'événement que l'on cherche généralement à détecter. Selon l'application, il peut s'agir par exemple d'un malade, d'une fraude, d'un défaut ou d'un client susceptible de se désabonner.

La matrice de confusion croise donc la **classe réelle** avec la **classe prédite** :

TABLE 1 – Illustration de la matrice de confusion

	Prédit négatif	Prédit positif
Réel négatif	VN	FP
Réel positif	FN	VP

Les quatre cases ont la signification suivante :

- **VP (vrai positif)** : l'observation est réellement positive et le modèle la prédit positive. Le modèle a donc correctement détecté un cas positif ;
- **VN (vrai négatif)** : l'observation est réellement négative et le modèle la prédit négative. Le modèle a donc correctement identifié un cas négatif ;
- **FP (faux positif)** : l'observation est réellement négative, mais le modèle la prédit positive. Il s'agit d'une **fausse alerte** ;
- **FN (faux négatif)** : l'observation est réellement positive, mais le modèle la prédit négative. Il s'agit d'un **cas positif manqué**.

Ces quatre quantités décrivent entièrement les résultats d'un classifieur binaire sur un ensemble de données. Les différentes métriques de classification ne font ensuite que combiner ces quatre nombres de manière différente.

La matrice permet également de lire directement plusieurs informations utiles. Le nombre total d'observations est

$$N = VP + VN + FP + FN.$$

Les deux lignes correspondent aux observations selon leur **classe réelle** :

$$N_{\text{réel négatif}} = VN + FP,$$

$$N_{\text{réel positif}} = FN + VP.$$

Les deux colonnes correspondent aux observations selon leur **classe prédite** :

$$N_{\text{prédit négatif}} = VN + FN,$$

$$N_{\text{prédit positif}} = FP + VP.$$

Cette lecture de la matrice est essentielle, car elle permet de comprendre la signification de chacune des métriques introduites ci-dessous.

4.2. Accuracy

L'**accuracy**, ou taux de bonnes classifications, mesure la proportion totale d'observations correctement classées.

Dans la matrice de confusion, les prédictions correctes correspondent exactement à la diagonale : **VN** pour les observations négatives et **VP** pour les observations positives. On obtient donc :

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN}.$$

Autrement dit :

Accuracy = nombre de prédictions correctes / nombre total de prédictions.

Elle tient donc compte simultanément des vrais positifs et des vrais négatifs, mais également, au dénominateur, des erreurs de classification.

L'accuracy est particulièrement facile à interpréter lorsque les classes sont relativement équilibrées. En revanche, elle peut devenir trompeuse en présence d'un fort déséquilibre entre les classes.

Par exemple, si 95 % des observations sont négatives, un modèle qui prédit **toujours négatif** obtient :

$$Accuracy = \frac{VN}{VN + FP + FN + VP} \approx 0,95,$$

alors qu'il ne détecte aucun cas positif. L'accuracy peut donc être élevée alors que le modèle est incapable d'identifier la classe minoritaire.

4.3. Précision

La **précision** (*precision*) s'intéresse uniquement aux observations que le modèle a **prédites positives**.

Dans la matrice, la colonne « Prédit positif » contient :

$$FP + VP.$$

Parmi ces observations prédites positives, seules les **VP** sont réellement positives. La précision est donc :

$$Precision = \frac{VP}{VP + FP}.$$

Autrement dit :

Parmi toutes les observations que le modèle a déclarées positives, quelle proportion est réellement positive ?

La relation avec la matrice de confusion est donc directe :

- **VP** augmente la précision, car il s'agit d'une prédiction positive correcte ;
- **FP** diminue la précision, car il s'agit d'une prédiction positive erronée.

Une précision élevée signifie donc que les **alertes positives produites par le modèle sont généralement fiables**.

Cette métrique est particulièrement importante lorsque les **faux positifs** sont coûteux. Par exemple, dans un système d'alerte, une faible précision signifie que de nombreuses alertes correspondent en réalité à des cas négatifs.

4.4. Rappel ou sensibilité

Le **rappel**, également appelé **sensibilité** (*recall* ou *sensitivity*), adopte le point de vue opposé à celui de la précision. Il part non plus des observations prédites positives, mais des observations qui sont **réellement positives**.

Dans la matrice, la ligne « Réel positif » contient :

$$FN + VP.$$

Parmi tous les cas réellement positifs, les **VP** sont ceux que le modèle a correctement détectés. Le rappel est donc :

$$Recall = \frac{VP}{VP + FN}.$$

Autrement dit :

Parmi toutes les observations réellement positives, quelle proportion le modèle parvient-il à détecter ?

Ici :

- **VP** correspond aux cas positifs correctement détectés ;
- **FN** correspond aux cas positifs manqués.

Un rappel élevé signifie donc que le modèle laisse **peu de cas positifs non détectés**.

Le rappel devient particulièrement important lorsque les **faux négatifs** sont coûteux. Dans un problème de détection médicale, par exemple, manquer un cas réellement positif peut être beaucoup plus problématique que déclencher une alerte sur un cas finalement négatif.

4.5. Spécificité

La **spécificité** mesure la capacité du modèle à reconnaître correctement les observations négatives. Elle est donc l'équivalent, pour la classe négative, du rappel pour la classe positive.

Dans la matrice, la ligne « Réel négatif » contient :

$$VN + FP.$$

Parmi tous les cas réellement négatifs, les **VN** sont correctement reconnus comme négatifs. La spécificité est donc :

$$Specificity = \frac{VN}{VN + FP}.$$

Autrement dit :

Parmi toutes les observations réellement négatives, quelle proportion le modèle reconnaît-il correctement comme négative ?

La spécificité dépend donc de :

- **VN**, qui correspond aux négatifs correctement identifiés ;
- **FP**, qui correspond aux négatifs classés à tort comme positifs.

Une spécificité élevée signifie que le modèle produit **peu de faux positifs parmi les observations réellement négatives**.

On peut ainsi établir le parallèle suivant :

$$Recall = \frac{VP}{VP + FN}$$

mesure la capacité à détecter les **positifs réels**, tandis que

$$Specificity = \frac{VN}{VN + FP}$$

mesure la capacité à reconnaître les **négatifs réels**.

4.6. F1-score

Le **F1-score** combine la **précision** et le **rappel**. Il est défini comme leur moyenne harmonique :

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

En remplaçant la précision et le rappel par leurs expressions à partir de la matrice de confusion, on peut également écrire :

$$F1 = \frac{2VP}{2VP + FP + FN}.$$

Cette deuxième expression met directement en évidence le lien avec la matrice de confusion.

Le F1-score dépend donc de :

- **VP**, qui contribue positivement au score ;
- **FP**, qui pénalise le score ;
- **FN**, qui pénalise également le score.

En revanche, **VN n'intervient pas directement dans la formule du F1-score**.

Le F1-score est donc particulièrement intéressant lorsque l'on souhaite obtenir un compromis entre :

- la capacité à éviter les fausses alertes, mesurée par la **précision** ;

- la capacité à détecter les positifs réels, mesurée par le **rappel**.

La moyenne harmonique fait qu'un score très faible pour l'une des deux composantes ne peut pas être compensé simplement par un score très élevé pour l'autre. Un modèle doit donc obtenir simultanément une précision et un rappel raisonnables pour obtenir un F1-score élevé.

4.7. Balanced accuracy

La **balanced accuracy** permet de donner le même poids aux performances obtenues sur les deux classes, ce qui est particulièrement utile lorsque les classes sont déséquilibrées.

Pour une classification binaire, elle est définie comme la moyenne du **rappel de la classe positive** et de la **spécificité**, c'est-à-dire du rappel de la classe négative :

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}.$$

En utilisant directement les éléments de la matrice de confusion :

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{VP}{VP + FN} + \frac{VN}{VN + FP} \right).$$

La relation avec la matrice de confusion est donc particulièrement claire :

- le premier terme évalue la capacité à détecter correctement les **positifs réels** ;
- le second évalue la capacité à reconnaître correctement les **négatifs réels**.

Contrairement à l'accuracy, la balanced accuracy ne permet pas à une classe majoritaire de dominer à elle seule la mesure globale. Elle accorde la même importance aux deux classes, indépendamment de leur fréquence.

4.8. Exemple numérique

Considérons la matrice de confusion suivante :

	Prédit négatif	Prédit positif
Réel négatif	90	10
Réel positif	5	15

On lit immédiatement :

$$VP = 15, \quad VN = 90, \quad FP = 10, \quad FN = 5.$$

Le nombre total d'observations est :

$$N = 90 + 10 + 5 + 15 = 120.$$

Les prédictions correctes correspondent à la diagonale de la matrice, soit :

$$VN + VP = 90 + 15 = 105.$$

L'accuracy vaut donc :

$$Accuracy = \frac{90 + 15}{120} = 0,875.$$

Le modèle classe ainsi correctement **87,5 %** des observations.

Pour comprendre les performances sur la classe positive, on peut ensuite regarder séparément la précision et le rappel.

Parmi les **25 observations prédites positives**,

$$FP + VP = 10 + 15 = 25,$$

15 sont réellement positives. On obtient donc :

$$Precision = \frac{15}{15 + 10} = 0,6.$$

Autrement dit, **60 % des observations déclarées positives par le modèle sont réellement positives**.

Par ailleurs, il existe **20 observations réellement positives** :

$$FN + VP = 5 + 15 = 20.$$

Le modèle en détecte correctement 15, d'où :

$$Recall = \frac{15}{15 + 5} = 0,75.$$

Le modèle détecte donc **75 % des cas réellement positifs** et en manque 25 %.

Pour la classe négative, il y a :

$$VN + FP = 90 + 10 = 100$$

observations réellement négatives. Parmi elles, 90 sont correctement reconnues comme négatives. La spécificité vaut donc :

$$Specificity = \frac{90}{90 + 10} = 0,9.$$

Le modèle reconnaît ainsi correctement **90 % des cas réellement négatifs**.

Le F1-score combine enfin la précision de 0,6 et le rappel de 0,75 :

$$F1 = 2 \frac{0,6 \times 0,75}{0,6 + 0,75} \approx 0,667.$$

On peut également le calculer directement à partir de la matrice :

$$F1 = \frac{2 \times 15}{2 \times 15 + 10 + 5} = \frac{30}{45} \approx 0,667.$$

La balanced accuracy vaut :

$$Balanced Accuracy = \frac{0,75 + 0,90}{2} = 0,825.$$

Cet exemple illustre pourquoi il est important de ne pas se limiter à l'accuracy. Une accuracy de **87,5 %** paraît élevée, mais l'examen de la matrice montre que le modèle produit **10 faux positifs** et **5 faux négatifs**. La précision de **60 %** indique notamment qu'une part importante des prédictions positives correspond à de fausses alertes, tandis que le rappel de **75 %** indique qu'un quart des cas réellement positifs n'est pas détecté.

La matrice de confusion fournit ainsi la lecture la plus fondamentale des résultats :

**VP et VN correspondent aux prédictions correctes ; FP et FN correspondent aux erreurs.
Les différentes métriques ne font que porter un regard différent sur ces quatre catégories.**

L'accuracy considère les bonnes prédictions dans leur ensemble, la précision se concentre sur les prédictions positives, le rappel sur les positifs réellement présents, la spécificité sur les négatifs réellement présents, le F1-score combine précision et rappel, et la balanced accuracy équilibre les performances sur les deux classes.

5. Seuil de décision et classes déséquilibrées

5.1. Choisir le seuil

Le seuil par défaut est souvent 0,5. Mais ce seuil ne dépend pas seulement du modèle : il dépend du contexte.

- seuil faible : plus de cas positifs prédits ;
- seuil élevé : moins de cas positifs prédits.

Un seuil plus faible augmente généralement le rappel et réduit les faux négatifs. Un seuil plus élevé augmente souvent la précision et réduit les faux positifs.

5.2. Cas de classes déséquilibrées

Une classification est déséquilibrée quand une classe est beaucoup plus fréquente que l'autre. Les conséquences sont :

- l'accuracy peut paraître élevée alors que la classe minoritaire est ignorée ;
- le modèle peut prédire presque toujours la classe majoritaire ;
- le seuil par défaut peut être inadapté.

En pratique, il faut alors examiner la matrice de confusion, la précision, le rappel, le F1 et la balanced accuracy.

5.3. Bonnes pratiques

- utiliser `stratify=y` lors du découpage train/test ;
- examiner la matrice de confusion ;
- choisir les métriques avant d'évaluer le modèle ;
- ajuster le seuil selon le coût métier des erreurs ;
- ne pas choisir le seuil en utilisant le jeu de test final comme critère d'optimisation.

6. Classification avec *scikit-learn*

6.1. Exemple simple de régression logistique

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[: , 1]

print(classification_report(y_test, y_pred))
```

- `predict()` donne la classe prédite ;
- `predict_proba()` donne les probabilités ;
- `[: , 1]` permet d'obtenir la probabilité de la classe positive dans un problème binaire.

6.2. Matrice de confusion et métriques

```
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)

matrix = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
balanced_accuracy = balanced_accuracy_score(y_test, y_pred)

print(matrix)
print(accuracy, precision, recall, f1, balanced_accuracy)
```

6.3. Pipeline de prétraitement

Le prétraitement doit être appris uniquement sur les données d'entraînement. On l'intègre dans un pipeline.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

numeric_features = ["age", "revenu"]
categorical_features = ["ville", "type_client"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features),
])

model = Pipeline([
    ("preprocessing", preprocessor),
```

```

    ("classifier", LogisticRegression(max_iter=1000)),
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]

```

6.4. Changer le seuil

```

threshold = 0.30
y_pred_custom = (y_proba >= threshold).astype(int)

```

Le seuil peut être choisi selon le contexte métier. Un seuil plus bas favorise le rappel ; un seuil plus haut favorise la précision.

6.5. Validation croisée stratifiée

```

from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = cross_validate(
    model,
    X_train,
    y_train,
    cv=cv,
    scoring={
        "accuracy": "accuracy",
        "balanced_accuracy": "balanced_accuracy",
        "f1": "f1",
        "roc_auc": "roc_auc",
    },
    return_train_score=False,
)

```

La validation croisée stratifiée est particulièrement utile si la classe positive est minoritaire.

7. En résumé

La classification binaire repose sur trois idées simples :

1. distinguer les classes avec un modèle ;
2. produire des probabilités, pas seulement des labels ;
3. évaluer le modèle avec des métriques adaptées au coût des erreurs.

La régression logistique est un excellent point de départ. Elle est mathématiquement claire, interprétable et suffisamment robuste pour servir de base de comparaison. L'évaluation doit toujours s'appuyer sur la matrice de confusion et sur des métriques comme la précision, le rappel et le F1, surtout lorsque les classes sont déséquilibrées.

Exercices

i Consigne

Les exercices suivants portent uniquement sur le calcul et la compréhension. Les corrigés sont fournis juste après chaque énoncé, dans un bloc repliable.

Exercice 0.0.1 (Probabilité logistique simple). Un modèle logistique est défini par :

$$z = -2 + x$$

1. Calculez z pour $x = 2$.
2. Calculez $\sigma(z)$.
3. Quelle classe est prédite avec un seuil de 0,5 ?
4. Calculez la probabilité pour $x = 4$.

Exercice 0.0.2 (Log loss d'une observation). Pour une observation positive, le modèle prédit :

- $p = 0,9$;
- $p = 0,5$;
- $p = 0,1$.

La perte est :

$$L = -\ln(p)$$

Classez ces trois prédictions de la meilleure à la pire.

Exercice 0.0.3 (Matrice de confusion). Un modèle de détection donne :

	Prédit négatif	Prédit positif
Réel négatif	80	20
Réel positif	10	40

Calculez :

1. l'accuracy ;
2. la précision ;
3. le rappel ;
4. la spécificité ;
5. le F1-score.

Exercice 0.0.4 (Effet du seuil). Un modèle produit les probabilités suivantes :

Observation	Probabilité de la classe 1	Vraie classe
A	0,90	1
B	0,70	0
C	0,55	1
D	0,40	1

Observation	Probabilité de la classe 1	Vraie classe
E	0,20	0

1. Donnez les classes prédites avec un seuil de 0,5.
2. Calculez le rappel avec ce seuil.
3. Donnez les classes prédites avec un seuil de 0,3.
4. Expliquez qualitativement le phénomène.

Exercice 0.0.5 (Classe majoritaire et accuracy trompeuse). Dans un jeu de 1 000 observations, la classe 0 représente 950 observations et la classe 1 représente 50 observations.

Un modèle prédit toujours la classe 0.

1. Quelle accuracy obtient-il ?
2. Quel est son rappel pour la classe 1 ?
3. Pourquoi l'accuracy est-elle insuffisante ici ?
4. Quelles métriques complémentaires faut-il examiner ?

Exercice 0.0.6 (Choisir un bon premier modèle). Choisissez un premier algorithme raisonnable dans chaque situation et justifiez :

1. on veut un modèle simple à expliquer et une frontière approximativement linéaire ;
2. on dispose d'un petit jeu de données et la décision dépend des observations voisines ;
3. on veut des règles de décision lisibles sous forme de questions successives ;
4. on veut commencer par un modèle simple et interprétable avant d'essayer des méthodes plus complexes.

Apprentissage non supervisé

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- distinguer apprentissage supervisé et apprentissage non supervisé ;
- expliquer l'idée d'une structure découverte sans variable cible ;
- comprendre l'ACP comme un outil de réduction de dimension ;
- formuler l'objectif de K-Means et calculer un centroïde ;
- interpréter la variance expliquée et l'inertie ;
- utiliser `scikit-learn` pour appliquer des méthodes simples ;
- éviter les erreurs fréquentes liées à l'échelle, au nombre de groupes et à l'interprétation.

i Lien avec les chapitres précédents

Les chapitres précédents ont surtout considéré des problèmes supervisés : on dispose d'une variable cible y et on cherche à prédire cette cible à partir de X .

Ici, on ne connaît pas de cible. On cherche plutôt à découvrir une structure dans les données :

$$X \longrightarrow \text{structure, regroupement ou projection}$$

Dans ce chapitre, nous abordons de manière introductive deux outils très utilisés :

- l'**analyse en composantes principales** (ACP) ;
- le **clustering K-Means**.

1. Qu'est-ce que l'apprentissage non supervisé ?

On dispose d'une matrice de données :

$$X = \begin{pmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{pmatrix} \in \mathbb{R}^{n \times p}$$

Mais il n'y a pas de variable cible y à prédire. L'objectif est de découvrir une organisation dans les données :

- regrouper des observations semblables ;
- réduire le nombre de variables sans perdre trop d'information ;
- repérer des observations inhabituelles ;
- résumer visuellement les données.

La différence principale est donc la suivante :

Type d'apprentissage	Informations disponibles	Objectif
Supervisé	X et y	Prédire y
Non supervisé	X seulement	Découvrir une structure

Exemple intuitif

Supposons que l'on observe des clients avec des variables comme :

- le montant total dépensé ;
- le nombre d'achats ;
- la fréquence de visite ;
- la durée moyenne de connexion.

Sans savoir à l'avance quels sont les clients “fidèles” ou “occasionnels”, on peut :

1. chercher des groupes de clients similaires avec K-Means ;
2. résumer les données dans un plan 2D avec l'ACP ;
3. interpréter ensuite les groupes ou axes dans le contexte métier.

! Un cluster n'est pas une vérité naturelle

Un algorithme ne décide pas seul “ce qui est vraiment vrai”. Il découvre une organisation selon des règles mathématiques. Un cluster est donc une structure utile, pas une catégorie intrinsèque.

2. Préparer les données

Avant d'appliquer des méthodes non supervisées, il faut regarder la nature des variables.

2.1. Variables numériques et variables qualitatives

Les méthodes comme l'ACP et K-Means travaillent sur des distances et des moyennes. Elles sont donc naturellement adaptées aux variables numériques.

Les problèmes viennent souvent avec :

- des identifiants sans sens statistique ;
- des variables dans des unités différentes ;
- des valeurs manquantes ;
- des variables qualitatives codées de manière arbitraire.

2.2. Centrage et réduction

Pour une variable j , la moyenne est :

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$$

Le centrage consiste à retirer cette moyenne :

$$x_{ij}^{(c)} = x_{ij} - \bar{x}_j$$

La réduction consiste à diviser par l'écart-type :

$$x_{ij}^{(s)} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Après standardisation, une variable a approximativement :

$$\text{moyenne} = 0, \quad \text{écart-type} = 1$$

Pourquoi c'est important ?

- en ACP, le centrage est presque toujours nécessaire ;
- en K-Means, la standardisation est souvent essentielle car la distance euclidienne dépend des échelles.

2.3. Exemple d'échelle

Supposons que l'on compare :

- le revenu annuel, entre 20 000 et 200 000 ;
- un taux, entre 0 et 1.

La première variable a naturellement une amplitude beaucoup plus grande. Dans une distance euclidienne, elle peut dominer complètement le regroupement. C'est pourquoi on standardise souvent les variables avant K-Means.

Prétraitement et fuite de données

En apprentissage supervisé comme en non supervisé, il faut éviter les fuites d'information. Si on calcule une moyenne ou un écart-type sur l'ensemble des données avant de séparer train/test, on utilise indirectement des informations de validation ou de test. En pratique, le prétraitement doit être ajusté sur le jeu d'entraînement puis appliqué aux autres données.

3. Analyse en composantes principales (ACP)

L'ACP cherche des directions dans lesquelles les données varient le plus. Ce n'est pas un algorithme de clustering ; c'est un outil de réduction de dimension.

3.1. Idée géométrique

Imaginez un nuage de points dans le plan. S'il est presque aligné selon une droite, on peut résumer la variation principale par cette direction.

L'ACP cherche :

1. une première direction de variance maximale ;
2. une deuxième direction orthogonale à la première ;
3. puis les directions suivantes, si nécessaire.

La première composante principale correspond à la direction qui explique le plus de variance dans les données.

3.2. Matrice centrée

On part de $X \in \mathbb{R}^{n \times p}$ et on calcule la moyenne de chaque colonne :

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$$

La matrice centrée est :

$$(X_c)_{ij} = x_{ij} - \bar{x}_j$$

Sous forme matricielle :

$$X_c = X - \mathbf{1}\bar{x}^\top$$

Ici, $\mathbf{1}$ est un vecteur colonne de 1 et $\bar{x}$ est le vecteur des moyennes.

3.3. Matrice de covariance

La matrice de covariance empirique est :

$$\Sigma = \frac{1}{n-1} X_c^\top X_c$$

Elle contient :

- sur la diagonale : les variances de chaque variable ;
- hors de la diagonale : les covariances entre variables.

Si deux variables bougent ensemble, leur covariance est positive. Si elles bougent en sens opposé, la covariance est négative.

3.4. Direction de variance maximale

On cherche un vecteur unitaire w tel que :

$$\max_{\|w\|_2=1} \text{Var}(X_c w)$$

Comme :

$$\text{Var}(X_c w) = w^\top \Sigma w$$

on cherche le vecteur w qui maximise $w^\top \Sigma w$ sous la contrainte $w^\top w = 1$.

La solution est le vecteur propre associé à la plus grande valeur propre de Σ .

3.5. Valeurs propres et composantes principales

Les vecteurs propres w_j et valeurs propres λ_j vérifient :

$$\Sigma w_j = \lambda_j w_j$$

où :

- w_j : direction de la composante principale j ;
- λ_j : variance expliquée par cette composante.

On ordonne :

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$$

Les composantes sont orthogonales entre elles :

$$w_j^\top w_k = 0 \quad \text{si } j \neq k$$

3.6. Projection et réduction de dimension

Si l'on garde les m premières composantes, on construit la matrice :

$$W_m = [w_1, w_2, \dots, w_m]$$

La projection des données centrées est :

$$Z = X_c W_m$$

On passe ainsi d'une matrice $n \times p$ à une matrice $n \times m$ avec $m < p$.

3.7. Variance expliquée

La variance totale est la somme des valeurs propres :

$$\sum_{j=1}^p \lambda_j$$

La proportion de variance expliquée par la composante j est :

$$EV R_j = \frac{\lambda_j}{\sum_{\ell=1}^p \lambda_{\ell}}$$

La variance expliquée cumulée par les m premières composantes est :

$$EV R_{cum}(m) = \sum_{j=1}^m EV R_j$$

Exemple : si $\lambda = (6, 3, 1)$, alors :

$$EV R_1 = \frac{6}{10} = 60\%, \quad EV R_{cum}(2) = \frac{6+3}{10} = 90\%$$

Donc deux composantes expliquent 90 % de la variance totale.

3.8. Exemple simple

Considérons les données suivantes :

Observation	x_1	x_2
1	1	1
2	2	2
3	3	3

Les moyennes sont :

$$\bar{x}_1 = 2, \quad \bar{x}_2 = 2$$

La matrice centrée est :

$$X_c = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

La covariance vaut :

$$\Sigma = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

Les valeurs propres sont :

$$\lambda_1 = 2, \quad \lambda_2 = 0$$

La première composante suit la direction $(1, 1)$: les deux variables bougent exactement de la même manière. Une seule composante suffit ici pour résumer toute la variation.

3.9. Quand utiliser l'ACP ?

L'ACP est utile pour :

- réduire la dimension visuellement ;
- supprimer une partie de la redondance linéaire ;
- mieux comprendre la structure des variables ;
- préparer des données avant un autre modèle.

Elle présente aussi des limites :

- elle ne capte pas forcément des relations non linéaires ;
- elle est sensible à l'échelle ;
- les composantes peuvent être difficiles à interpréter si les variables sont hétérogènes ;
- une grande variance n'est pas toujours synonyme d'information utile pour une tâche précise.

4. K-Means

K-Means est un algorithme de clustering. Il cherche à découper les observations en K groupes, ou clusters, autour de K centres appelés centroïdes.

4.1. Objectif

On a n observations $x_1, \dots, x_n$ dans $\mathbb{R}^p$. On choisit un nombre de clusters K .

Chaque observation reçoit une affectation :

$$c_i \in \{1, \dots, K\}$$

Chaque cluster k a un centroïde $\mu_k \in \mathbb{R}^p$.

L'objectif de K-Means est de minimiser l'inertie :

$$J(c, \mu) = \sum_{i=1}^n \|x_i - \mu_{c_i}\|_2^2$$

Cette quantité mesure la somme des carrés des distances entre chaque observation et le centroïde de son cluster.

4.2. Affectation des observations

Si les centroïdes sont connus, on attribue chaque observation au centroïde le plus proche :

$$c_i = \arg \min_{k \in \{1, \dots, K\}} \|x_i - \mu_k\|_2^2$$

4.3. Mise à jour des centroïdes

Si les affectations sont fixes, le meilleur centroïde d'un cluster est la moyenne des observations qui lui sont affectées :

$$\mu_k = \frac{1}{n_k} \sum_{i: c_i = k} x_i$$

où n_k est le nombre d'observations du cluster k .

4.4. Algorithme itératif

K-Means alterne donc deux étapes :

1. affecter chaque observation au centroïde le plus proche ;
2. recalculer chaque centroïde comme la moyenne de son cluster.

On répète jusqu'à convergence.

Pseudo-code :

```
Choisir K centroïdes initiaux
Répéter :
    affecter chaque point au centroïde le plus proche
    calculer le nouveau centroïde de chaque cluster
Jusqu'à ce que les affectations ne changent plus
```

4.5. Exemple simple

Considérons quatre points :

$$A = (1, 1), \quad B = (1, 2), \quad C = (4, 4), \quad D = (5, 4)$$

On prend $K = 2$ et des centroïdes initiaux :

$$\mu_1 = (1, 1), \quad \mu_2 = (5, 4)$$

Les distances au carré sont :

Point	Vers μ_1	Vers μ_2	Cluster
$A = (1, 1)$	0	25	1
$B = (1, 2)$	1	20	1
$C = (4, 4)$	18	1	2
$D = (5, 4)$	25	0	2

Les clusters sont donc :

$$C_1 = \{A, B\}, \quad C_2 = \{C, D\}$$

Les nouveaux centroïdes deviennent :

$$\mu'_1 = \frac{(1, 1) + (1, 2)}{2} = \left(1, \frac{3}{2}\right)$$

et

$$\mu'_2 = \frac{(4, 4) + (5, 4)}{2} = \left(\frac{9}{2}, 4\right)$$

On recommence ensuite jusqu'à ce que les affectations se stabilisent.

4.6. Inertie et choix du nombre de clusters

L'inertie totale est :

$$Inertie = \sum_{k=1}^K \sum_{i:c_i=k} \|x_i - \mu_k\|_2^2$$

Plus l'inertie est petite, plus les clusters sont compacts.

Cependant, l'inertie diminue presque toujours lorsque K augmente. Elle ne suffit donc pas à choisir K .

Deux méthodes simples sont souvent utilisées :

- la **méthode du coude** : on regarde la courbe d'inertie en fonction de K ;
- le **score de silhouette** : il compare cohésion interne et séparation entre clusters.

4.7. Pourquoi le choix de l'échelle est crucial ?

K-Means repose sur une distance euclidienne. Si une variable a des valeurs très grandes, elle peut dominer la distance.

Par exemple, une variable exprimée en milliers et une autre en unités peuvent ne pas avoir le même poids. La standardisation corrige ce problème en ramenant les variables à une échelle comparable.

4.8. Limites de K-Means

K-Means est simple et utile, mais il a des limites :

- il faut choisir le nombre K ;
- il dépend de l'initialisation ;
- il est sensible aux outliers ;
- il privilégie des groupes de forme compacte et sphérique ;
- il n'est pas adapté à des groupes de densités très différentes.

On l'utilise souvent comme première méthode de segmentation, puis on compare à d'autres approches si nécessaire.

5. Utilisation avec *scikit-learn*

5.1. ACP avec PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
print(pca.explained_variance_ratio_)
print(pca.explained_variance_ratio_.cumsum())
```

Les éléments importants sont :

- `components_` : directions principales;
- `explained_variance_` : variances associées;
- `explained_variance_ratio_` : part de variance expliquée;
- `mean_` : moyenne utilisée pour le centrage.

5.2. K-Means avec KMeans

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

kmeans = KMeans(
    n_clusters=3,
    init="k-means++",
    n_init=10,
    random_state=42,
)

labels = kmeans.fit_predict(X_scaled)
centers = kmeans.cluster_centers_
inertia = kmeans.inertia_
```

Les principaux attributs sont :

- `labels_` : cluster attribué à chaque observation;
- `cluster_centers_` : centroïdes;
- `inertia_` : inertie finale;
- `n_iter_` : nombre d'itérations.

5.3. Méthode du coude

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X_scaled = StandardScaler().fit_transform(X)

inertias = []
for k in range(2, 11):
    model = KMeans(n_clusters=k, n_init=10, random_state=42)
    model.fit(X_scaled)
    inertias.append(model.inertia_)

plt.plot(range(2, 11), inertias, marker="o")
```

```
plt.xlabel("Nombre de clusters K")
plt.ylabel("Inertie")
plt.title("Méthode du coude")
plt.show()
```

Le coude est un point de compromis : si l'inertie baisse fortement jusqu'à une valeur de K , puis beaucoup moins après, on peut choisir cette valeur comme compromis entre compacité et simplicité.

5.4. Score de silhouette

```
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans

X_scaled = StandardScaler().fit_transform(X)

for k in range(2, 7):
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    print(k, score)
```

Le score de silhouette est proche de 1 pour de bons clusters, proche de 0 pour des observations frontières, et négatif si le cluster semble mauvais.

5.5. Pipeline simple

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
])

X_reduced = pipe.fit_transform(X)
```

Le pipeline garantit que la transformation est ajustée sur les données d'entraînement, puis réutilisée sur les données de test ou de validation.

i Règle pratique

En non supervisé, la simplicité et la cohérence sont souvent meilleures que la complexité. Pour débiter, il vaut mieux bien maîtriser ACP et K-Means avant d'explorer des méthodes plus avancées.

6. Points de vigilance

6.1. Ne pas surinterpréter les résultats

Un cluster n'est pas forcément une catégorie naturelle. Il est surtout un regroupement selon une métrique et des variables données.

6.2. Égale importance des variables

Si aucune standardisation n'est faite, une variable à grande échelle peut dominer l'analyse.

6.3. Choisir un bon K

Le bon nombre de clusters dépend du problème, des données et du contexte métier. Un score numérique ne remplace pas l'interprétation.

6.4. ACP et variance maximale

L'ACP cherche à maximiser la variance, pas à optimiser une performance métier. Une composante de faible variance peut néanmoins contenir une information très importante pour une tâche précise.

7. Exercices de compréhension

Consigne

Les exercices ci-dessous portent sur le calcul et la compréhension. Essayez de répondre par écrit avant de consulter la solution.

Exercice 0.0.1 (1. Supervisé ou non supervisé ?). Pour chaque situation, indiquez s'il s'agit d'un problème supervisé ou non supervisé, puis précisez la tâche principale :

1. prédire le prix d'un appartement à partir d'exemples dont le prix est connu ;
2. regrouper des clients sans connaître à l'avance leurs segments ;
3. réduire 20 variables en 2 axes pour visualiser les données ;
4. prédire si une transaction est frauduleuse à partir d'exemples étiquetés.

Exercice 0.0.2 (2. Centrage d'une matrice). On considère :

$$X = \begin{pmatrix} 1 & 4 \\ 3 & 2 \\ 5 & 6 \end{pmatrix}$$

1. Calculez la moyenne de chaque colonne.
2. Construisez la matrice centrée X_c .
3. Vérifiez que la moyenne de chaque colonne de X_c est nulle.

Exercice 0.0.3 (3. Matrice de covariance). On considère les données centrées :

$$X_c = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

Calculez :

1. $X_c^\top X_c$;
2. $\Sigma = \frac{1}{n-1} X_c^\top X_c$;
3. la relation entre les deux variables.

Exercice 0.0.4 (4. Variance expliquée). Une matrice de covariance possède les valeurs propres suivantes :

$$\lambda_1 = 6, \quad \lambda_2 = 3, \quad \lambda_3 = 1$$

Calculez :

1. la variance totale ;
2. la variance expliquée par la première composante ;
3. la variance expliquée cumulée par les deux premières composantes.

Exercice 0.0.5 (5. Affectation K-Means). On considère un point :

$$x = (3, 2)$$

et deux centroïdes :

$$\mu_1 = (1, 1), \quad \mu_2 = (5, 3)$$

1. Calculez les distances au carré vers μ_1 et μ_2 .
2. À quel cluster le point est-il affecté ?

Exercice 0.0.6 (6. Recalculer un centroïde). Un cluster contient les observations :

$$(1, 2), \quad (3, 2), \quad (2, 5)$$

Calculez le centroïde du cluster.

Exercice 0.0.7 (7. Inertie d'un cluster). Un cluster contient :

$$(1, 1), \quad (1, 3)$$

Son centroïde est :

$$\mu = (1, 2)$$

Calculez son inertie :

$$I = \|x_1 - \mu\|^2 + \|x_2 - \mu\|^2$$

Exercice 0.0.8 (8. Effet de l'échelle). Une équipe applique K-Means à deux variables :

- le revenu annuel, entre 20 000 et 200 000 ;
 - un taux, entre 0 et 1.
1. Quelle variable risque de dominer la distance euclidienne ?
 2. Quelle transformation peut être envisagée ?
 3. Pourquoi cette transformation modifie-t-elle la notion de similarité ?

Exercice 0.0.9 (9. Lecture d'un résultat de clustering). K-Means produit trois groupes de clients. Le groupe 2 a en moyenne une dépense plus élevée que les autres.

Indiquez quelles conclusions sont justifiées et lesquelles sont abusives :

1. le groupe 2 possède une dépense moyenne plus élevée dans les données ;
2. appartenir au groupe 2 provoque une dépense élevée ;
3. le groupe 2 peut être décrit comme un groupe de clients à forte dépense selon les variables utilisées ;
4. le groupe 2 représente une catégorie naturelle et stable de clients.

References

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Springer.

Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.

Mitchell, Tom M, et Tom M Mitchell. 1997. *Machine learning*. Vol. 1. McGraw-hill New York.

Lab 0 — Environnement Python et premiers pas avec les données

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Installer** et utiliser `uv` pour gérer un projet Python reproductible ;
- **Créer** un projet structuré, sans appeler directement `venv` ou `pip` ;
- **Installer** et vérifier les bibliothèques nécessaires au cours ;
- **Charger** des données depuis un fichier local et depuis une API publique ;
- **Explorer** un premier jeu de données avec `pandas` (aperçu, types, statistiques) ;
- **Produire** un premier graphique avec `matplotlib` ;
- **Vérifier** que `scikit-learn` fonctionne, en reprenant l'exemple du filtre anti-spam du chapitre 1.

i Lien avec le cours

Ce Lab met en pratique le chapitre 1 (*Introduction au Machine Learning et à son écosystème*) : la distinction Exploration/Production (Section), la gestion de projet avec `uv` (§4.3), et l'exemple du filtre anti-spam (§2.2). Aucune notion de modélisation avancée n'est requise ici — l'objectif est uniquement de disposer d'un environnement de travail fiable et de premiers réflexes de manipulation de données. La théorie de l'apprentissage (§ généralisation, évaluation) sera mise en pratique au Lab 1.

Ce Lab suit cette progression :

1. installer et vérifier `uv` ;
2. créer un projet Python reproductible ;
3. installer et vérifier les bibliothèques ;
4. organiser le projet avec `pathlib` ;
5. premier contact avec des données tabulaires locales (`pandas`) ;
6. récupérer des données réelles via une API publique (météo) ;
7. vérifier que `scikit-learn` fonctionne (mini filtre anti-spam) ;
8. **Lab Sheet** — projet guidé à réaliser et à rendre.

1. Installer uv

1.1. Pourquoi uv plutôt que venv + pip ?

`uv` est un outil moderne, rapide, qui gère à la fois la version de Python, l'environnement virtuel et les dépendances d'un projet. Il remplace, dans ce cours :

- `python -m venv` (création manuelle d'environnement) ;
- `pip install` (installation non tracée des paquets) ;
- des notes éparses du type « chez moi ça marche ».

! Règle de ce cours

On ne crée jamais un environnement avec `python -m venv` et on n'installe jamais de paquet avec `pip install` directement. Toutes les commandes passent par `uv`.

1.2. Installation

Sur macOS ou Linux, dans un terminal :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Sur Windows, dans PowerShell :

```
irm https://astral.sh/uv/install.ps1 | iex
```

! Sécurité

Comme pour tout script d'installation téléchargé depuis Internet, vérifiez la source officielle (`astral.sh/uv`) avant d'exécuter la commande. Sur un poste institutionnel, suivez la procédure validée par l'administrateur réseau.

Fermez puis rouvrez le terminal, puis vérifiez :

```
uv --version
```

1.3. Choisir une version de Python

```
uv python list          # versions disponibles ou déjà installées
uv python install 3.12  # installe Python 3.12, géré par uv
```

2. Créer le projet

2.1. Initialiser avec uv

```
mkdir -p cours-machine-learning
cd cours-machine-learning
uv init --python 3.12 ml-lab
cd ml-lab
```

Cette commande crée un `pyproject.toml` minimal, qui décrira le projet et ses dépendances tout au long du semestre.

2.2. Le dossier `.venv`

L'environnement virtuel est créé automatiquement au premier besoin :

```
uv run python --version
```

Il n'est **jamais nécessaire de l'activer manuellement** : `uv run <commande>` exécute directement la commande dans l'environnement du projet.

```
uv run python script.py    # exécute un script
uv run python               # lance un interpréteur interactif
```

i Activation facultative

Si vous préférez activer l'environnement dans votre terminal, c'est possible (`source .venv/bin/activate` sur macOS/Linux, `.venv\Scripts\Activate.ps1` sous PowerShell), mais ce Lab privilégie `uv run` pour éviter toute confusion entre plusieurs environnements installés sur la même machine.

2.3. Installer les bibliothèques du Lab

```
uv add numpy pandas scikit-learn matplotlib requests jupyter ipykernel
```

Bibliothèque	Rôle dans ce Lab
numpy	Calcul numérique, tableaux
pandas	Données tabulaires, lecture de fichiers
scikit-learn	Vérification que le ML fonctionne (mini filtre anti-spam)
matplotlib	Premiers graphiques
requests	Requêtes HTTP vers une API publique
jupyter, ipykernel	Exécution en notebook

2.4. Vérifier l'installation

Créez un fichier `check_environment.py` à la racine du projet :

```
import sys

import numpy
import pandas
import sklearn
import requests
import matplotlib

print("Python      :", sys.version.split()[0])
print("NumPy       :", numpy.__version__)
print("pandas       :", pandas.__version__)
print("scikit-learn  :", sklearn.__version__)
print("requests     :", requests.__version__)
print("matplotlib   :", matplotlib.__version__)
```

```
uv run python check_environment.py
```

Si le script s'exécute sans erreur, l'environnement est prêt.

2.5. Lancer Jupyter

```
uv run jupyter lab
```

Pour enregistrer le noyau du projet (utile s'il existe déjà d'autres environnements sur la machine) :

```
uv run python -m ipykernel install --user --name ml-lab --display-name "Python (ml-lab)"
```

! Reproductibilité — ce qu'on versionne

Fichier / dossier	À versionner (Git) ?
pyproject.toml	
uv.lock	
.venv/	jamais
data/raw/ (petits fichiers)	selon le cours

uv.lock fixe les versions exactes de toutes les dépendances : n'importe qui peut reconstruire un environnement strictement identique avec `uv sync`. `.venv/` se reconstruit automatiquement et ne doit jamais être transmis.

3. Organiser le projet

```
ml-lab/
  pyproject.toml
  uv.lock
  data/
    raw/
    processed/
  notebooks/
  outputs/
```

```
mkdir -p data/raw data/processed notebooks outputs
```

Utilisez `pathlib` plutôt que des chaînes concaténées pour construire vos chemins — cela évite les problèmes entre Windows, macOS et Linux :

```
from pathlib import Path

PROJECT_DIR = Path.cwd()
RAW_DIR = PROJECT_DIR / "data" / "raw"
PROCESSED_DIR = PROJECT_DIR / "data" / "processed"
OUTPUT_DIR = PROJECT_DIR / "outputs"

for directory in (RAW_DIR, PROCESSED_DIR, OUTPUT_DIR):
    directory.mkdir(parents=True, exist_ok=True)
```

4. Premier contact avec des données tabulaires

Créons un tout petit jeu de données local — quelques annonces immobilières, dans l'esprit de l'exemple fil rouge du chapitre 1 (§3) — pour manipuler `pandas` sans dépendre d'Internet.

```
import pandas as pd

annonces = pd.DataFrame({
    "id": [1, 2, 3, 4, 5, 6],
    "quartier": ["Maarif", "Hay Riad", "Gueliz", "Maarif", "Agdal", "Hay Riad"],
    "surface_m2": [65, 90, 120, 48, 75, 110],
    "nb_chambres": [2, 3, 4, 1, 2, 3],
    "prix_mad": [850_000, 1_450_000, 1_900_000, 520_000, 980_000, 1_680_000],
})

annonces.to_csv("data/raw/annonces.csv", index=False)
print(annonces)
```

4.1. Explorer un DataFrame

```
df = pd.read_csv("data/raw/annonces.csv")

print(df.head())          # les premières lignes
print(df.shape)           # (nombre de lignes, nombre de colonnes)
print(df.dtypes)          # le type de chaque colonne
print(df.describe())      # statistiques descriptives des colonnes numériques
```

4.2. Sélectionner, filtrer, agréger

```
# sélectionner une colonne
print(df["prix_mad"])

# sélectionner plusieurs colonnes
print(df[["quartier", "prix_mad"]])

# filtrer des lignes
grands_biens = df[df["surface_m2"] > 80]
```

```
print(grands_biens)

# prix moyen par quartier
print(df.groupby("quartier")["prix_mad"].mean().round(0))

# créer une nouvelle colonne
df["prix_m2"] = (df["prix_mad"] / df["surface_m2"]).round(0)
print(df.sort_values("prix_m2", ascending=False))
```

i Remarque

Ce petit jeu de données est directement lisible par des humains — c'est volontaire. Le chapitre 3 (Lab 2) travaillera sur des données plus grandes et volontairement imparfaites (valeurs manquantes, doublons, incohérences).

4.3. Un premier graphique

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df["surface_m2"], df["prix_mad"], s=60)
ax.set_xlabel("Surface (m2)")
ax.set_ylabel("Prix (MAD)")
ax.set_title("Prix des annonces selon la surface")
fig.tight_layout()
fig.savefig("outputs/prix_vs_surface.png", dpi=150)
plt.show()
```

Même avec six observations, la relation entre surface et prix commence déjà à se dessiner à l'œil nu — c'est précisément ce type de relation qu'un modèle de régression (chapitre 4 / Lab 3) apprendra à quantifier.

5. Récupérer des données réelles via une API publique

5.1. Principe

Une lecture de données en ligne suit toujours le même schéma : construire une URL, envoyer une requête HTTP, vérifier le code de réponse, puis interpréter le contenu (souvent du JSON).

```
import requests

response = requests.get("https://example.com", timeout=20)
response.raise_for_status() # lève une erreur si le code n'est pas 2xx
```

💡 Toujours fixer un `timeout`

Sans `timeout`, un programme peut rester bloqué indéfiniment si le serveur ne répond jamais. `raise_for_status()` transforme un code d'erreur HTTP (404, 500...) en exception Python explicite, plutôt que de laisser passer silencieusement une réponse invalide.

5.2. Exemple : l'API météo Open-Meteo (sans clé)

```
import pandas as pd
import requests

params = {
    "latitude": 33.5731,      # Casablanca
    "longitude": -7.5898,
    "hourly": "temperature_2m",
    "timezone": "Africa/Casablanca",
    "forecast_days": 2,
}

response = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params=params,
    timeout=30,
)
response.raise_for_status()
payload = response.json()

meteo = pd.DataFrame(payload["hourly"])
meteo["time"] = pd.to_datetime(meteo["time"])
print(meteo.head())

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(meteo["time"], meteo["temperature_2m"])
ax.set_xlabel("Heure")
ax.set_ylabel("Température (°C)")
ax.set_title("Prévisions horaires - Casablanca")
fig.autofmt_xdate()
fig.tight_layout()
fig.savefig("outputs/meteo_casablanca.png", dpi=150)
plt.show()
```

⚠️ Une requête peut échouer

URL incorrecte, absence de connexion, délai dépassé, service temporairement indisponible : une requête réseau n'est jamais garantie. Dans un vrai projet, on entoure ce type d'appel d'une gestion d'erreurs (`try/except`) et on sauvegarde une copie brute de la réponse pour pouvoir rejouer l'analyse hors ligne.

6. Vérifier que *scikit-learn* fonctionne : le filtre anti-spam du chapitre 1

Le chapitre 1 (§2.2) présente l'idée du filtre anti-spam comme premier exemple du paradigme Data-Driven. Reproduisons-le concrètement, avec un tout petit jeu d'e-mails fait main.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

emails_entrainement = [
    "Gagnez de l'argent facilement, cliquez ici maintenant",
    "Offre urgente : gagnez un iPhone gratuit aujourd'hui",
    "Réunion projet reportée à demain 10h",
    "Voici le compte-rendu de la réunion d'hier",
    "Urgent : votre compte a gagné un prix, réclamez-le",
    "Peux-tu relire mon rapport avant vendredi ?",
]

y_entrainement = ["spam", "spam", "non-spam", "non-spam", "spam", "non-spam"]

vectoriseur = TfidfVectorizer()
X_entrainement = vectoriseur.fit_transform(emails_entrainement)

modele = LogisticRegression()
modele.fit(X_entrainement, y_entrainement)

nouveaux_emails = [
    "Cliquez ici pour gagner un cadeau gratuit",
    "Peux-tu m'envoyer le fichier avant la réunion ?",
]

X_nouveaux = vectoriseur.transform(nouveaux_emails)
predictions = modele.predict(X_nouveaux)

for email, prediction in zip(nouveaux_emails, predictions):
    print(f"[{prediction}] {email}")
```

Si ce code s'exécute et affiche une prédiction **spam** ou **non-spam** pour chaque e-mail, votre environnement est prêt pour la suite du cours : **scikit-learn**, comme discuté au chapitre 1, a bien **appris** une règle de décision à partir des six exemples fournis, sans qu'aucune règle **SI ... ALORS ...** n'ait été écrite à la main.

i Pourquoi si peu d'exemples ne suffisent pas en pratique

Six exemples d'entraînement, c'est très peu : le modèle risque de mal généraliser à des e-mails très différents de ceux vus à l'entraînement (voir la notion de généralisation, chapitre 2 / Lab 1). Ce mini-exemple sert uniquement à vérifier que l'environnement fonctionne de bout en bout.

7. Synthèse

i Auto-évaluation

- ☐ `uv --version` fonctionne sur ma machine.
- ☐ Mon projet possède un `pyproject.toml` et un `uv.lock`.
- ☐ Je sais lancer une commande avec `uv run` sans activer manuellement `.venv`.
- ☐ Je sais charger un CSV local avec `pandas` et en afficher un résumé (`head`, `dtypes`, `describe`).
- ☐ Je sais effectuer une requête HTTP avec `requests` et transformer une réponse JSON en `DataFrame`.
- ☐ J'ai exécuté avec succès le mini filtre anti-spam et compris ce qu'il illustre.

8. Lab Sheet — Projet guidé

! Ce qui est attendu

Ce projet reprend, sur des données différentes de celles utilisées en cours, les compétences pratiquées dans ce Lab. Répondez aux questions dans un notebook ou un fichier `reponses.md`, et conservez tout le code utilisé pour y répondre.

Partie A — Environnement

- Donnez la sortie de `uv --version` et de `uv python list` sur votre machine.
- Exécutez `check_environment.py`. Copiez la sortie obtenue.
- Expliquez, en deux phrases, la différence de rôle entre `pyproject.toml` et `uv.lock`.

Partie B — Météo au Maroc, à vous de jouer

Choisissez **trois villes marocaines différentes** de celle utilisée en exemple (§5.2) et récupérez, pour chacune, les températures horaires prévues sur 2 jours via l'API Open-Meteo.

💡 Coordonnées de quelques villes

Ville	Latitude	Longitude
Rabat	34.020882	-6.841650
Marrakech	31.629472	-7.981084
Tanger	35.759465	-5.833954
Fès	34.033124	-5.000282
Agadir	30.427755	-9.598107

- Construisez un seul `DataFrame` combinant les trois villes (ajoutez une colonne `ville`).
- Quelle ville affiche la température **moyenne** la plus élevée sur la période? Et l'**écart** entre température minimale et maximale le plus grand?
- Produisez un graphique superposant les trois courbes de température (une couleur par ville) et sauvegardez-le dans `outputs/`.

7. En une phrase : que se passerait-il dans votre code si l'API Open-Meteo était temporairement indisponible ? Que faudrait-il ajouter pour s'en protéger ?

Partie C — Améliorer le mini filtre anti-spam

8. Ajoutez **quatre nouveaux e-mails** à la liste d'entraînement de la §6 (deux spams, deux non-spams), rédigés par vous-même, en français ou en darija translittérée.
9. Réentraînez le modèle avec ces dix exemples, puis testez-le sur **trois nouveaux e-mails** que vous inventez (non utilisés à l'entraînement).
10. Le modèle se trompe-t-il sur l'un de vos trois e-mails de test ? Si oui, proposez une explication plausible en vous appuyant sur le fonctionnement de `TfidfVectorizer` (quels mots le modèle a-t-il pu associer à tort au spam ?).

Livrables

- `pyproject.toml` et `uv.lock` ;
- un notebook ou script contenant le code des parties A, B et C ;
- au moins un graphique dans `outputs/` ;
- vos réponses écrites aux questions 3, 5, 7 et 10.

💡 Checklist finale

- ☐ L'environnement est reproductible (`uv sync` fonctionne sur une autre machine).
- ☐ Les données météo de trois villes sont récupérées et combinées.
- ☐ Un graphique comparatif est produit et sauvegardé.
- ☐ Le mini filtre anti-spam est réentraîné avec de nouveaux exemples et testé.
- ☐ Toutes les questions écrites ont une réponse.

Lab 1 — Généralisation et évaluation avec `scikit-learn`

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Construire** la matrice X et le vecteur y d'un jeu de données avec `pandas/scikit-learn`;
- **Utiliser** l'API commune de `scikit-learn` (`fit`, `predict`, `score`);
- **Séparer** un jeu de données en Train/Test avec `train_test_split`;
- **Observer concrètement** le sur-apprentissage et le sous-apprentissage sur un graphique ;
- **Mettre en œuvre** une validation croisée (`KFold`, `cross_val_score`);
- **Comparer** un modèle à une baseline (`DummyClassifier`);
- **Calculer** une matrice de confusion et les métriques associées ;
- **Repérer**, sur un exemple concret, une fuite de données liée à un mauvais ordre des opérations.

i Lien avec le cours

Ce Lab met en pratique le chapitre 2 (*Fondements Théoriques de l'Apprentissage Automatique*) : notation X/y (Table 1), généralisation et sur-apprentissage (Figure 1), jeux Train/Validation/Test (Table 2), baseline et matrice de confusion (Table 1). Le classifieur des k plus proches voisins (`KNeighborsClassifier`) est utilisé ici uniquement comme **véhicule pédagogique** pour manipuler concrètement un hyperparamètre (k) — son fonctionnement détaillé est étudié au chapitre 5 (Lab 4). De même, les pipelines de prétraitement complets (imputation, encodage) sont réservés au chapitre 3 (Lab 2) ; nous travaillons ici sur des données déjà numériques et propres, pour isoler les notions d'évaluation.

1. Rappel express : construire X et y

Reprenons le petit jeu d'annonces immobilières du Lab 0, mais agrandi et légèrement bruité pour ressembler à un vrai jeu de données.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)

n = 150
quartiers = rng.choice(["Maarif", "Hay Riad", "Gueliz", "Agdal"], size=n)
surface = rng.uniform(30, 180, size=n).round(0)
chambres = rng.integers(1, 6, size=n)

prix_base = 8_000 * surface + 60_000 * chambres
bonus_quartier = pd.Series(quartiers).map({
    "Maarif": 150_000, "Hay Riad": 250_000, "Gueliz": 100_000, "Agdal": 180_000,
```

```

}).to_numpy()
bruit = rng.normal(0, 60_000, size=n)

annonces = pd.DataFrame({
    "surface_m2": surface,
    "nb_chambres": chambres,
    "quartier": quartiers,
    "prix_mad": (prix_base + bonus_quartier + bruit).round(0),
})
print(annonces.head())

```

Conformément à la notation du chapitre 2 (Table 1), on sépare **variables explicatives** (X) et **cible** (y). Pour rester sur des variables numériques dans cette première section, on ignore temporairement **quartier** (l'encodage complet est traité au Lab 2) :

```

X = annonces[["surface_m2", "nb_chambres"]]
y = annonces["prix_mad"]

print("X :", X.shape)
print("y :", y.shape)

```

1.1. L'API commune de scikit-learn

Quel que soit l'algorithme utilisé, **scikit-learn** propose toujours la même interface :

```

from sklearn.linear_model import LinearRegression

modele = LinearRegression() # 1. instancier
modele.fit(X, y)             # 2. apprendre les paramètres à partir de (X, y)
predictions = modele.predict(X) # 3. prédire
score = modele.score(X, y)     # 4. évaluer (ici, le  $R^2$ )

print("R² sur les données d'entraînement :", round(score, 3))

```

 Ce score est trompeur

Ce score est calculé sur les **mêmes données** que celles utilisées pour l'entraînement. Comme vu au chapitre 2, cela ne mesure en rien la capacité de généralisation du modèle — ce n'est qu'une mesure de qualité d'ajustement (*fit*). La suite du Lab corrige ce problème.

2. Séparer Train et Test

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

```

```
print("Entraînement :", X_train.shape[0], "observations")
print("Test          :", X_test.shape[0], "observations")

modele = LinearRegression()
modele.fit(X_train, y_train)

print("R² train :", round(modele.score(X_train, y_train), 3))
print("R² test  :", round(modele.score(X_test, y_test), 3))
```

Les deux scores sont proches : c'est un bon signe (voir Figure 1). Un très grand écart entre les deux serait au contraire la signature d'un sur-apprentissage.

 Règle d'or (rappel du chapitre 2, Table 2)

`random_state=42` fixe le générateur aléatoire du découpage : cela garantit que vous obtenez toujours **le même** partage Train/Test en réexécutant le code — indispensable pour la reproductibilité (chapitre 1, §4.3).

3. Observer le sur-apprentissage à l'œil nu

Rien ne rend la notion de sur-apprentissage plus concrète que de la provoquer soi-même. Construisons un jeu de données 1D synthétique, où la vraie relation est une courbe, puis ajustons des polynômes de degré croissant.

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures

rng = np.random.default_rng(0)
x = rng.uniform(0, 1, size=60).reshape(-1, 1)
y_vrai = np.sin(2 * np.pi * x).ravel()
y_bruit = y_vrai + rng.normal(0, 0.25, size=60)

x_train, x_val, y_train_1d, y_val_1d = train_test_split(
    x, y_bruit, test_size=0.3, random_state=0,
)

degres = range(1, 15)
erreurs_train, erreurs_val = [], []

for degre in degres:
    poly = PolynomialFeatures(degree=degre)
    x_train_poly = poly.fit_transform(x_train)
    x_val_poly = poly.transform(x_val)

    modele_poly = LinearRegression()
    modele_poly.fit(x_train_poly, y_train_1d)

    erreurs_train.append(mean_squared_error(y_train_1d, modele_poly.predict(x_train_poly)))
    erreurs_val.append(mean_squared_error(y_val_1d, modele_poly.predict(x_val_poly)))
```

```
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(degrees, erreurs_train, marker="o", label="Erreur d'entraînement")
ax.plot(degrees, erreurs_val, marker="o", label="Erreur de validation")
ax.set_xlabel("Degré du polynôme (complexité du modèle)")
ax.set_ylabel("Erreur quadratique moyenne (MSE)")
ax.set_title("Sous-apprentissage   Sur-apprentissage")
ax.legend()
fig.tight_layout()
fig.savefig("outputs/courbe_biais_variance.png", dpi=150)
plt.show()
```

Vous devriez observer, comme sur la Figure 1 du chapitre 2 :

- pour un **faible degré** (1-2), les deux erreurs sont élevées et proches : c'est du **sous-apprentissage** — le modèle (une droite ou presque) est trop simple pour capturer la courbe sinusoïdale ;
- pour un **degré intermédiaire**, l'erreur de validation atteint son minimum : c'est le meilleur compromis ;
- pour un **degré élevé** (10+), l'erreur d'entraînement continue de baisser (le modèle mémorise le bruit) tandis que l'erreur de validation **remonte** : c'est le **sur-apprentissage**.

💡 À retenir

Ce graphique est l'un des plus importants du cours. Reproduisez-le mentalement chaque fois que vous choisissez la complexité d'un modèle (profondeur d'un arbre, degré d'un polynôme, valeur de k en KNN, force de régularisation...).

4. La validation croisée

Un unique découpage Train/Validation dépend du hasard du tirage : sur un petit jeu de données, deux découpages différents peuvent donner des résultats sensiblement différents. La **validation croisée** répète l'évaluation sur plusieurs découpages et moyenne les résultats — une notion annoncée mais non détaillée au chapitre 2, que nous mettons ici en pratique.

```
from sklearn.model_selection import KFold, cross_val_score

modele = LinearRegression()

kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(modele, X, y, cv=kfold, scoring="r2")

print("Scores R² sur les 5 plis :", scores.round(3))
print("Moyenne :", round(scores.mean(), 3), " Écart-type :", round(scores.std(), 3))
```

Chaque pli (*fold*) sert tour à tour de jeu de validation, pendant que les quatre autres servent à l'entraînement : chaque observation est donc utilisée exactement une fois pour valider, et quatre fois pour entraîner. La moyenne des 5 scores est une estimation plus stable de la performance réelle qu'un unique découpage.

i StratifiedKFold pour la classification

Pour une tâche de **classification**, on utilise **StratifiedKFold** plutôt que **KFold** : il garantit que chaque pli conserve la même proportion de chaque classe que le jeu complet — important si les classes sont déséquilibrées

(voir le chapitre de classification pour les limites de l'accuracy).

5. Une baseline avant tout modèle

Avant de juger un modèle « bon », il faut le comparer à une règle triviale. Passons à une tâche de **classification** pour illustrer ce point avec le jeu de données **iris** (fourni avec **scikit-learn** — trois espèces de fleurs, décrites par 4 mesures).

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
X_iris, y_iris = iris.data, iris.target

X_iris_train, X_iris_test, y_iris_train, y_iris_test = train_test_split(
    X_iris, y_iris, test_size=0.25, random_state=42, stratify=y_iris,
)

from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_iris_train, y_iris_train)
print("Accuracy de la baseline :", round(baseline.score(X_iris_test, y_iris_test), 3))
```

Le jeu **iris** étant parfaitement équilibré (50 fleurs par espèce), cette baseline plafonne autour de 33 % — un score que **tout** modèle sérieux doit largement dépasser pour être utile (voir chapitre 2, §6).

6. Un vrai modèle, sa matrice de confusion et ses métriques

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import ConfusionMatrixDisplay, classification_report, confusion_matrix

modele_knn = KNeighborsClassifier(n_neighbors=5)
modele_knn.fit(X_iris_train, y_iris_train)

y_pred = modele_knn.predict(X_iris_test)
print("Accuracy KNN :", round(modele_knn.score(X_iris_test, y_iris_test), 3))

matrice = confusion_matrix(y_iris_test, y_pred)
print(matrice)

fig, ax = plt.subplots(figsize=(5, 5))
ConfusionMatrixDisplay(matrice, display_labels=iris.target_names).plot(ax=ax, cmap="Blues")
ax.set_title("Matrice de confusion - KNN sur Iris")
fig.tight_layout()
fig.savefig("outputs/confusion_iris.png", dpi=150)
plt.show()

print(classification_report(y_iris_test, y_pred, target_names=iris.target_names))
```

i Lecture de la matrice

Comme précisé au chapitre 2 (Table 1), la convention adoptée dans ce cours est : **lignes = classes réelles**, **colonnes = classes prédites**. La diagonale contient les prédictions correctes ; toute valeur hors diagonale est une erreur, et sa position indique précisément quelle classe a été confondue avec quelle autre — une information que l'*accuracy* seule ne donne jamais.

`classification_report` calcule automatiquement, pour chaque classe, la **précision**, le **rappel** et le **F1-score** (moyenne harmonique des deux) — trois métriques annoncées au chapitre 2 (§6) et détaillées au chapitre 5.

7. Repérer une fuite de données, concrètement

Le chapitre 2 (§4.2) insiste sur la fuite de données. Provoquons-en une, volontairement, pour en mesurer l'effet chiffré.

```
from sklearn.preprocessing import StandardScaler

# --- Version INCORRECTE : standardisation avant la séparation ---
scaler_fuite = StandardScaler()
X_iris_standardise = scaler_fuite.fit_transform(X_iris)    # fit sur TOUT le jeu

X_train_f, X_test_f, y_train_f, y_test_f = train_test_split(
    X_iris_standardise, y_iris, test_size=0.25, random_state=42, stratify=y_iris,
)

modele_fuite = KNeighborsClassifier(n_neighbors=5)
modele_fuite.fit(X_train_f, y_train_f)
score_fuite = modele_fuite.score(X_test_f, y_test_f)

# --- Version CORRECTE : standardisation après la séparation ---
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_iris, y_iris, test_size=0.25, random_state=42, stratify=y_iris,
)

scaler_correct = StandardScaler()
X_train_c_scaled = scaler_correct.fit_transform(X_train_c)    # fit UNIQUEMENT sur train
X_test_c_scaled = scaler_correct.transform(X_test_c)          # transform seul sur test

modele_correct = KNeighborsClassifier(n_neighbors=5)
modele_correct.fit(X_train_c_scaled, y_train_c)
score_correct = modele_correct.score(X_test_c_scaled, y_test_c)

print("Accuracy (avec fuite)  :", round(score_fuite, 4))
print("Accuracy (sans fuite)  :", round(score_correct, 4))
```

Sur `iris`, l'écart est ici modeste (le jeu de données est petit et propre), mais sur des jeux plus grands ou plus bruités, l'écart peut être significatif et **toujours optimiste en faveur de la version avec fuite** — un piège d'autant plus dangereux qu'il donne une fausse confiance dans le modèle avant son déploiement.

8. Un premier aperçu du réglage d'hyperparamètres

k (le nombre de voisins de `KNeighborsClassifier`) est un **hyperparamètre** (Table 3) : il n'est pas appris, il doit être choisi. La bonne pratique, rappelée au chapitre 2 (§5), est de comparer plusieurs valeurs **uniquement via la validation croisée**, jamais sur le jeu de test.

```
for k in [1, 3, 5, 7, 9, 15]:
    scores_k = cross_val_score(
        KNeighborsClassifier(n_neighbors=k), X_iris_train, y_iris_train, cv=5,
    )
    print(f"k={k:2d} → accuracy moyenne (CV) = {scores_k.mean():.3f}")
```

i Pour aller plus loin : `GridSearchCV`

Comparer les valeurs une à une avec une boucle fonctionne, mais devient vite fastidieux dès qu'il y a plusieurs hyperparamètres à combiner. Le Lab 3 (régression, chapitre 4) introduit `GridSearchCV`, qui automatise entièrement cette recherche par validation croisée. Nous nous limitons ici à une boucle simple pour bien comprendre le principe sous-jacent avant d'utiliser l'outil automatisé.

9. Synthèse

i Auto-évaluation

- ☐ Je sais construire X et y à partir d'un `DataFrame pandas`.
- ☐ Je sais utiliser `train_test_split` et expliquer pourquoi `random_state` est fixé.
- ☐ Je peux tracer et interpréter une courbe d'erreur train/validation selon la complexité.
- ☐ Je sais utiliser `cross_val_score` et expliquer ce qu'apporte la validation croisée par rapport à un split unique.
- ☐ Je sais comparer un modèle à une `DummyClassifier`.
- ☐ Je sais produire et lire une matrice de confusion et un `classification_report`.
- ☐ Je peux reproduire, sur un exemple, l'écart de score causé par une fuite de données.

10. Lab Sheet — Projet guidé

! Jeu de données du projet

Ce projet utilise le jeu de données `wine`, également fourni par `scikit-learn` (from `sklearn.datasets` import `load_wine`) : 178 vins italiens, décrits par 13 mesures chimiques, répartis en 3 cultivars (classes). C'est un jeu **différent** de celui utilisé en cours — à vous de refaire, seul(e), l'ensemble de la démarche.

Partie A — Prise en main

- Chargez `load_wine(as_frame=True)`. Combien d'observations et de variables explicatives le jeu contient-il ? Les classes sont-elles équilibrées (comptez les occurrences de chaque classe) ?
- Séparez les données en Train (75 %) / Test (25 %), avec `stratify` sur la cible et un `random_state` de votre choix. Justifiez en une phrase l'usage de `stratify` ici.

Partie B — Baseline et premier modèle

3. Entraînez un `DummyClassifier` et notez son accuracy sur le jeu de test. À quoi correspond ce chiffre, compte tenu de votre réponse à la question 1 ?
4. Entraînez un `KNeighborsClassifier` avec $k = 5$ **sans aucune mise à l'échelle préalable**. Comparez son accuracy à celle de la baseline.

Partie C — Une mise à l'échelle change-t-elle tout ?

5. Refaites la question 4 en standardisant les données au préalable (avec la méthode **correcte**, sans fuite). L'accuracy change-t-elle ? Proposez une explication liée à la nature de l'algorithme KNN (indice : pensez à la notion de distance entre observations, chapitre 3 §5).

Partie D — Régler l'hyperparamètre k

6. Par validation croisée à 5 plis **sur le jeu d'entraînement uniquement**, testez au moins six valeurs de k entre 1 et 25. Tracez un graphique accuracy moyenne (CV) en fonction de k .
7. Identifiez la valeur de k qui vous semble la meilleure. Justifiez votre choix en vous appuyant sur le graphique (y a-t-il un signe de sur-apprentissage pour les très petites valeurs de k ? de sous-apprentissage pour les très grandes ?).

Partie E — Évaluation finale

8. Réentraînez un modèle avec le k retenu sur l'ensemble du jeu d'entraînement, puis évaluez-le **une seule fois** sur le jeu de test : accuracy, matrice de confusion et `classification_report`.
9. Le modèle confond-il davantage certaines paires de cultivars que d'autres ? Formulez une hypothèse.
10. En une phrase : pourquoi le jeu de test n'a-t-il été utilisé qu'une seule fois, à la toute fin de votre démarche ?

Livrables

- un notebook ou script contenant le code des parties A à E ;
- au moins deux graphiques (courbe de k , matrice de confusion) dans `outputs/` ;
- vos réponses écrites aux questions 1, 2, 3, 5, 7, 9 et 10.

💡 Checklist finale

- ☐ Le split Train/Test est stratifié et reproductible.
- ☐ Une baseline est calculée avant tout modèle sérieux.
- ☐ k est choisi par validation croisée sur le train, jamais sur le test.
- ☐ Le jeu de test n'est utilisé qu'une seule fois, à la fin.
- ☐ La matrice de confusion et le `classification_report` sont produits et interprétés.

Lab 2 — Collecte et prétraitement des données

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Lire** des données depuis un CSV, une API JSON et une base SQLite ;
- **Extraire** une information simple depuis une page HTML (*web scraping*) ;
- **Diagnostiquer** des valeurs manquantes, des doublons et des valeurs aberrantes ;
- **Comparer** une implémentation *from scratch* de l'imputation et de la standardisation à **scikit-learn** ;
- **Encoder** correctement des variables catégorielles ordinales et nominales ;
- **Construire** un pipeline complet (**ColumnTransformer** + **Pipeline**) sans fuite de données.

i Lien avec le cours

Ce Lab met en pratique le chapitre 3 (*Collecte et Prétraitement des Données*) dans son intégralité : sources de données (§1), valeurs manquantes et MCAR/MAR/MNAR (§3.1), valeurs aberrantes (§3.2), encodage (§4), mise à l'échelle (§5) et pipelines (§6). Les exercices « papier » du chapitre 3 vous ont fait calculer ces transformations à la main sur un jeu minuscule ; ce Lab vous fait passer à l'échelle avec du vrai code, sur des données plus volumineuses et réellement imparfaites.

1. Lire des données depuis plusieurs sources

1.1. CSV local

```
import pandas as pd

clients = pd.read_csv("data/raw/clients_bruts.csv")
print(clients.head())
```

(Nous allons construire ce fichier nous-mêmes en §2.)

1.2. Une API JSON

```
import requests

repo_url = "https://api.github.com/repos/pandas-dev/pandas"
response = requests.get(repo_url, timeout=20)
response.raise_for_status()
repo = response.json()

print({"étoiles": repo["stargazers_count"], "forks": repo["forks_count"]})
```

1.3. Une base SQLite

```
import sqlite3

connexion = sqlite3.connect("data/interim/clients.sqlite")

connexion.execute("""
CREATE TABLE IF NOT EXISTS clients (
    client_id INTEGER,
    ville TEXT,
    revenu REAL
)
""")
connexion.executemany(
    "INSERT INTO clients VALUES (?, ?, ?)",
    [(101, "Rabat", 6500.0), (102, "Casablanca", 12000.0)],
)
connexion.commit()

resultat = pd.read_sql_query(
    "SELECT * FROM clients WHERE revenu >= ?", connexion, params=(7000,),
)
print(resultat)
connexion.close()
```

Paramètres SQL, pas concaténation

`params=(7000,)` évite de construire la requête par concaténation de chaînes — une concaténation directe de données utilisateur peut provoquer une injection SQL, même dans un projet pédagogique.

1.4. Extraire une information depuis une page HTML

```
from bs4 import BeautifulSoup

url = "https://books.toscrape.com/"
response = requests.get(url, headers={"User-Agent": "lab-educational/1.0"}, timeout=20)
response.raise_for_status()

soup = BeautifulSoup(response.text, "html.parser")
livres = soup.select("article.product_pod")[:5]

for livre in livres:
    titre = livre.select_one("h3 a")["title"]
    prix = livre.select_one(".price_color").get_text(strip=True)
    print(f"{titre} - {prix}")
```

⚠ Éthique et robustesse (rappel chapitre 3, §1.2)

`books.toscrape.com` est un site conçu pour l'entraînement au scraping. Sur un site réel, il faut vérifier `robots.txt`, les conditions d'utilisation, et ajouter un délai entre les requêtes (`time.sleep(1)`). Un scraper de ce type reste fragile : il casse dès que la structure HTML change.

2. Construire un jeu de données volontairement imparfait

Pour pratiquer le nettoyage, nous avons besoin de vraies imperfections. Créons-les nous-mêmes, en connaissance de cause :

```
from pathlib import Path

Path("data/raw").mkdir(parents=True, exist_ok=True)

clients_bruts = pd.DataFrame({
    "client_id":      [101, 102, 103, 103, 104, 105, 106, 107, 108],
    "age":            [25, 41, None, None, 72, 19, 36, 36, 250],
    "ville":          ["Rabat", "casablanca", "Tanger", "Tanger",
                      "Casablanca ", "rabat", "Marrakech", "Marrakech", None],
    "revenu_mensuel": [6500, 12000, 8300, 8300, 18000, 4200, 7600, 7600, 9100],
    "nombre_achats":  [4, 12, 8, 8, 20, 1, 6, 6, 5],
    "montant_total":  [1800.0, 15200.0, 7200.0, 7200.0, 25000.0, 250.0, 5100.0, 5100.0, 6300.0],
    "client_actif":   ["oui", "oui", "non", "non", "oui", "non", "oui", "oui", "oui"],
})
clients_bruts.to_csv("data/raw/clients_bruts.csv", index=False)
print(clients_bruts)
```

Ce petit jeu contient, volontairement :

- des **valeurs manquantes** (age, ville);
- un **doublon** (client_id = 103, présent deux fois à l'identique);
- une **valeur aberrante** (age = 250);
- des **incohérences de casse et d'espaces** dans ville ("casablanca", "Casablanca ", "rabat").

C'est exactement le type de désordre décrit au chapitre 3 comme la norme plutôt que l'exception.

3. Diagnostiquer avant de nettoyer

```
clients = pd.read_csv("data/raw/clients_bruts.csv")

print(clients.isna().sum())          # valeurs manquantes par colonne
print(clients.duplicated().sum())     # nombre de lignes strictement dupliquées
print(clients.duplicated(subset=["client_id"]).sum()) # doublons sur la clé
print(clients["age"].describe())      # repère vite les valeurs aberrantes
```

3.1. Supprimer les doublons

```
clients = clients.drop_duplicates(subset=["client_id"], keep="first")
print(clients.shape)
```

3.2. Détecter les valeurs aberrantes (IQR)

```
q1 = clients["age"].quantile(0.25)
q3 = clients["age"].quantile(0.75)
iqr = q3 - q1
borne_basse = q1 - 1.5 * iqr
borne_haute = q3 + 1.5 * iqr

print(f"Intervalle acceptable : [{borne_basse:.1f}, {borne_haute:.1f}]")
aberrantes = clients[(clients["age"] < borne_basse) | (clients["age"] > borne_haute)]
print(aberrantes[["client_id", "age"]])
```

Ici, `age = 250` est manifestement une **erreur de saisie** (un âge humain plausible) plutôt qu’une observation rare et légitime (chapitre 3, §3.2) — on la remplace par une valeur manquante, qui sera traitée comme telle :

```
clients.loc[clients["age"] > 120, "age"] = None
```

3.3. Harmoniser les catégories textuelles

```
clients["ville"] = clients["ville"].str.strip().str.title()
print(clients["ville"].value_counts(dropna=False))
```

4. Imputation : à la main, puis avec *scikit-learn*

4.1. Calculer une imputation *from scratch*

Avant d’utiliser `SimpleImputer`, calculons la médiane nous-mêmes, pour être certains de comprendre ce que l’outil fait réellement :

```
mediane_age = clients["age"].median()
print("Médiane calculée à la main :", mediane_age)

age_impute_a_la_main = clients["age"].fillna(mediane_age)
```

4.2. La même chose avec `SimpleImputer`

```
import numpy as np
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
age_impute_sklearn = imputer.fit_transform(clients[["age"]]).ravel()

print("Résultats identiques :", np.allclose(age_impute_a_la_main.to_numpy(), age_impute_sklearn))
```

! Pourquoi passer par **scikit-learn** si le résultat est identique ?

Sur une seule colonne, coder l'imputation à la main est simple. L'intérêt de **SimpleImputer** apparaît dès que l'on doit : (1) répéter l'opération sur de nombreuses colonnes, (2) l'intégrer dans un **Pipeline** qui garantit l'ordre **fit** sur train puis **transform** sur test (chapitre 3, §3.1), et (3) la combiner à d'autres étapes via **ColumnTransformer** (§6 de ce Lab).

4.3. Imputation catégorielle

```
imputer_ville = SimpleImputer(strategy="most_frequent")
clients["ville"] = imputer_ville.fit_transform(clients[["ville"]]).ravel()
```

5. Encodage : ne pas confondre **LabelEncoder** et **OrdinalEncoder**

Le chapitre 3 (§4.1) distingue encodage ordinal et One-Hot. En pratique, **scikit-learn** propose deux outils différents pour l'encodage ordinal, à ne pas confondre :

Outil	Usage prévu
LabelEncoder	Encoder la cible <i>y</i> (une seule colonne, en classification)
OrdinalEncoder	Encoder des variables explicatives ordinales (dans <i>X</i> , éventuellement plusieurs colonnes)

```
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder

# Simulons une variable réellement ordinale
clients["niveau_fidelite"] = pd.cut(
    clients["nombre_achats"], bins=[-1, 3, 10, 100], labels=["Faible", "Moyen", "Élevé"],
)

encodeur_ordinal = OrdinalEncoder(categories=[["Faible", "Moyen", "Élevé"]])
clients["niveau_fidelite_code"] = encodeur_ordinal.fit_transform(clients[["niveau_fidelite"]])
print(clients[["niveau_fidelite", "niveau_fidelite_code"]].drop_duplicates())
```

```
# ville n'a pas d'ordre naturel : One-Hot Encoding
encodeur_onehot = OneHotEncoder(sparse_output=False, drop="first")
ville_encodee = encodeur_onehot.fit_transform(clients[["ville"]])
colonnes_ville = encodeur_onehot.get_feature_names_out(["ville"])
print(pd.DataFrame(ville_encodee, columns=colonnes_ville).head())
```

6. Mise à l'échelle : calculer, puis vérifier

6.1. Standardisation *from scratch*

```
colonne = clients["revenu_mensuel"].to_numpy(dtype=float)

moyenne = colonne.mean()
ecart_type = colonne.std()
standardise_a_la_main = (colonne - moyenne) / ecart_type

print("Moyenne après standardisation :", round(standardise_a_la_main.mean(), 10))
print("Écart-type après standardisation :", round(standardise_a_la_main.std(), 10))
```

6.2. Vérifier avec StandardScaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
standardise_sklearn = scaler.fit_transform(clients[["revenu_mensuel"]]).ravel()

print("Résultats identiques :", np.allclose(standardise_a_la_main, standardise_sklearn))
print("Moyenne apprise par le scaler :", scaler.mean_)
print("Écart-type appris par le scaler :", scaler.scale_)
```

6.3. Et si une variable a des valeurs extrêmes ?

```
from sklearn.preprocessing import RobustScaler

# On ajoute une valeur extrême pour simuler une grande fortune
revenu_avec_extreme = pd.concat([
    clients["revenu_mensuel"], pd.Series([500_000]),
], ignore_index=True)

standard = StandardScaler().fit_transform(revenu_avec_extreme.to_frame()).ravel()
robuste = RobustScaler().fit_transform(revenu_avec_extreme.to_frame()).ravel()

comparaison = pd.DataFrame({"valeur": revenu_avec_extreme, "standard": standard.round(2), "robuste": robuste})
print(comparaison.tail(3))
```

Observez comme la valeur extrême écrase les autres avec **StandardScaler** (des valeurs standardisées proches de 0 pour presque tout le monde), alors que **RobustScaler** — fondé sur la médiane et l'écart interquartile — reste nettement plus stable (chapitre 3, Table 5).

7. Construire le pipeline complet, sans fuite

```

from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

clients_final = pd.read_csv("data/raw/clients_bruts.csv").drop_duplicates(subset=["client_id"])
clients_final.loc[clients_final["age"] > 120, "age"] = None
clients_final["ville"] = clients_final["ville"].str.strip().str.title()

X = clients_final[["age", "ville", "revenu_mensuel", "nombre_achats", "montant_total"]]
y = clients_final["client_actif"].map({"non": 0, "oui": 1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y,
)

colonnes_numeriques = ["age", "revenu_mensuel", "nombre_achats", "montant_total"]
colonnes_categorielles = ["ville"]

pipeline_numerique = Pipeline([
    ("imputation", SimpleImputer(strategy="median")),
    ("mise_a_l_echelle", StandardScaler()),
])

pipeline_categorielle = Pipeline([
    ("imputation", SimpleImputer(strategy="most_frequent")),
    ("encodage", OneHotEncoder(drop="first", handle_unknown="ignore")),
])

pretraitement = ColumnTransformer([
    ("num", pipeline_numerique, colonnes_numeriques),
    ("cat", pipeline_categorielle, colonnes_categorielles),
])

# fit UNIQUEMENT sur le train (chapitre 3, §5.3 et §3.1)
X_train_pret = pretraitement.fit_transform(X_train)
X_test_pret = pretraitement.transform(X_test)

print("Forme de X_train après prétraitement :", X_train_pret.shape)

```

💡 Fonctionnalité récente : sortie en DataFrame

Depuis les versions récentes de Scikit-Learn, l'appel `pretraitement.set_output(transform="pandas")` permet d'obtenir directement un `DataFrame` pandas en sortie du pipeline, avec des noms de colonnes explicites plutôt qu'un tableau `numpy` sans noms.

```

pretraitement.set_output(transform="pandas")
X_train_pret_df = pretraitement.fit_transform(X_train)
X_test_pret_df = pretraitement.transform(X_test)
print(X_train_pret_df.head())

```

8. Synthèse

i Auto-évaluation

- ☐ Je sais lire un CSV, appeler une API JSON et interroger une base SQLite avec **pandas**.
- ☐ Je sais diagnostiquer valeurs manquantes, doublons et valeurs aberrantes avant de nettoyer.
- ☐ Je sais calculer une imputation et une standardisation *from scratch* et vérifier qu'elles correspondent aux outils **scikit-learn**.
- ☐ Je sais expliquer la différence d'usage entre **LabelEncoder** et **OrdinalEncoder**.
- ☐ Je sais construire un **ColumnTransformer** combinant imputation, encodage et mise à l'échelle.
- ☐ Je sais expliquer pourquoi **fit_transform** s'applique au train et **transform** seul au test.

9. Lab Sheet — Projet guidé

! Jeu de données du projet : le Titanic

Ce projet utilise le célèbre jeu de données des passagers du Titanic, disponible publiquement :

```
titanic = pd.read_csv(
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
)
```

C'est un jeu **réel**, avec de véritables valeurs manquantes et un mélange de variables numériques et catégorielles — pas un jeu fabriqué pour l'exercice.

Partie A — Diagnostic

1. Chargez le jeu de données. Combien d'observations et de variables contient-il ?
2. Pour chaque colonne, calculez le pourcentage de valeurs manquantes. Quelle colonne est la plus touchée ? Proposez une hypothèse sur la raison probable de ces absences (est-ce plutôt MCAR, MAR ou MNAR, à votre avis ?).
3. Y a-t-il des lignes strictement dupliquées ?

Partie B — Nettoyage et décisions justifiées

4. La colonne **deck** (ou **cabin** selon la source) a un taux de valeurs manquantes très élevé. Que proposez-vous de faire, et pourquoi (référez-vous au tableau des stratégies d'imputation, Table 2) ?
5. Choisissez, en le justifiant, comment traiter les valeurs manquantes de **age** : suppression, imputation simple, imputation par groupe (par exemple, par classe de billet) ?
6. La variable **fare** (prix du billet) contient-elle des valeurs qui pourraient être considérées comme aberrantes selon la méthode IQR ? Affichez-les. S'agit-il probablement d'erreurs ou d'observations réelles mais extrêmes ?

Partie C — Encodage et mise à l'échelle

7. Parmi les variables catégorielles du jeu (`sex`, `embarked`, `class`...), laquelle est **ordinaire** et laquelle est **nominaire**? Justifiez, puis choisissez l'encodage approprié pour chacune.
8. La variable `fare` a une distribution très asymétrique (quelques billets très chers). Quelle méthode de mise à l'échelle du chapitre 3 (Table 5) recommanderiez-vous, et pourquoi?

Partie D — Pipeline complet

9. Construisez un `ColumnTransformer` complet (imputation + encodage + mise à l'échelle adaptés à chaque type de colonne) pour préparer `X`, avec `survived` comme cible `y`.
10. Séparez Train/Test (80/20, stratifié sur `survived`), ajustez votre pipeline **uniquement sur le train**, puis transformez le test. Affichez la forme (`shape`) du tableau obtenu.
11. (*Bonus*) Enchaînez votre `ColumnTransformer` avec un `LogisticRegression` dans un unique `Pipeline`, entraînez-le, et affichez son `accuracy` sur le jeu de test. Vous retrouverez ce même jeu de données pour aller plus loin au Lab 4 (classification).

Livrables

- un notebook ou script contenant le code des parties A à D ;
- un court texte (quelques phrases) justifiant chacune de vos décisions de nettoyage ;
- le pipeline final, exécutable de bout en bout sur les données brutes.

💡 Checklist finale

- ☐ Chaque décision de nettoyage est justifiée, pas seulement appliquée.
- ☐ Aucune statistique (moyenne, médiane, min/max) n'est calculée sur l'ensemble des données avant le `split`.
- ☐ Le pipeline distingue correctement colonnes numériques et catégorielles.
- ☐ Le pipeline s'exécute sans erreur de bout en bout sur des données brutes jamais vues.

Lab 3 — Régression : prédire une valeur numérique

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Implémenter** une régression linéaire simple *from scratch* et retrouver le même résultat qu'avec `scikit-learn`;
- **Entraîner** une régression linéaire multiple sur plusieurs variables;
- **Calculer et interpréter** les métriques MAE, MSE, RMSE et R^2 ;
- **Diagnostiquer** un modèle de régression à partir de ses résidus;
- **Utiliser** la régularisation (Ridge, Lasso) pour lutter contre le sur-apprentissage;
- **Régler un hyperparamètre proprement** avec `GridSearchCV` et la validation croisée;
- **Assembler** un pipeline complet, du prétraitement au modèle final.

i Lien avec le cours

Ce Lab correspond au chapitre 4 (*Modèles de régression*). Il réutilise directement les compétences des Labs précédents : construction de X/y et évaluation (Lab 1, chapitre 2), prétraitement et `ColumnTransformer` (Lab 2, chapitre 3). C'est ici que le réglage d'hyperparamètres, seulement esquissé au Lab 1, est traité **en profondeur** avec `GridSearchCV` — une méthode que vous réutiliserez telle quelle au Lab 4 (classification).

1. Rappel : qu'est-ce qu'apprend une régression linéaire ?

Une régression linéaire simple cherche les coefficients a et b qui minimisent l'erreur quadratique moyenne entre les valeurs prédites $\hat{y} = ax + b$ et les valeurs réelles y :

$$\text{MSE}(a, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (ax_i + b))^2$$

Pour la régression linéaire, cette minimisation possède une solution exacte, calculable directement (les **équations normales**), que nous allons calculer nous-mêmes avant de laisser `scikit-learn` s'en charger.

1.1. Régression linéaire simple, *from scratch*

```
import numpy as np

rng = np.random.default_rng(0)
surface = rng.uniform(30, 180, size=200)
prix = 8_500 * surface + 120_000 + rng.normal(0, 60_000, size=200)
```

```
# Équations normales pour a et b (régression simple)
x_moyenne, y_moyenne = surface.mean(), prix.mean()
a_calcule = np.sum((surface - x_moyenne) * (prix - y_moyenne)) / np.sum((surface - x_moyenne) ** 2)
b_calcule = y_moyenne - a_calcule * x_moyenne

print(f"a (pente)      = {a_calcule:,.1f} MAD par m²")
print(f"b (ordonnée)   = {b_calcule:,.1f} MAD")
```

1.2. Vérification avec scikit-learn

```
from sklearn.linear_model import LinearRegression

modele = LinearRegression()
modele.fit(surface.reshape(-1, 1), prix)

print("a (scikit-learn) :", round(modele.coef_[0], 1))
print("b (scikit-learn) :", round(modele.intercept_, 1))
print("Résultats identiques :", np.allclose([a_calcule, b_calcule], [modele.coef_[0], modele.intercept_]))
```

💡 Ce que `.fit()` a fait concrètement

`LinearRegression().fit()` n'est rien d'autre qu'une résolution optimisée des mêmes équations normales que celles calculées à la main ci-dessus. Comprendre ce calcul démystifie ce qui se cache derrière `.fit()`.

2. Passer à la régression multiple

En pratique, un seul prédicteur suffit rarement. Construisons un jeu de données d'annonces immobilières casablancaises plus riche, dans l'esprit du fil rouge du cours (chapitre 1, §3).

```
import pandas as pd

n = 300
rng = np.random.default_rng(7)

quartiers = rng.choice(["Maarif", "Hay Riad", "Gueliz", "Agdal", "Ain Diab"], size=n)
surface = rng.uniform(30, 220, size=n)
chambres = rng.integers(1, 6, size=n)
age_bien = rng.integers(0, 40, size=n)
etage = rng.integers(0, 12, size=n)

bonus_quartier = pd.Series(quartiers).map({
    "Maarif": 150_000, "Hay Riad": 260_000, "Gueliz": 90_000,
    "Agdal": 190_000, "Ain Diab": 320_000,
}).to_numpy()

prix = (
    8_200 * surface + 55_000 * chambres - 3_000 * age_bien + 15_000 * etage
    + bonus_quartier + rng.normal(0, 70_000, size=n)
```

```

)

annonces = pd.DataFrame({
    "surface_m2": surface.round(0), "nb_chambres": chambres, "age_bien": age_bien,
    "etage": etage, "quartier": quartiers, "prix_mad": prix.round(0),
})
annonces.to_csv("data/processed/annonces_casablanca.csv", index=False)
print(annonces.head())

from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = annonces.drop(columns="prix_mad")
y = annonces["prix_mad"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

colonnes_numeriques = ["surface_m2", "nb_chambres", "age_bien", "etage"]
colonnes_categorielles = ["quartier"]

pretraitement = ColumnTransformer([
    ("num", StandardScaler(), colonnes_numeriques),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), colonnes_categorielles),
])

pipeline_regression = Pipeline([
    ("pretraitement", pretraitement),
    ("modele", LinearRegression()),
])

pipeline_regression.fit(X_train, y_train)
print("R2 test :", round(pipeline_regression.score(X_test, y_test), 3))

```

3. Métriques de régression : MAE, MSE, RMSE, R^2

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_pred = pipeline_regression.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MAE = {mae:.0f} MAD")
print(f"MSE = {mse:.0f}")
print(f"RMSE = {rmse:.0f} MAD")
print(f"R2 = {r2:.3f}")

```

Métrique	Unité	Ce qu'elle mesure	À minimiser ou maximiser ?
MAE	Celle de y (MAD)	Erreur absolue moyenne, peu sensible aux valeurs extrêmes	Minimiser
MSE	Celle de y au carré	Pénalise fortement les grosses erreurs	Minimiser
RMSE	Celle de y (MAD)	Racine de la MSE, revient à l'unité d'origine — la plus lisible en pratique	Minimiser
R^2	Sans unité, ≤ 1	Proportion de variance expliquée par le modèle par rapport à une baseline qui prédit toujours la moyenne	Maximiser

i R^2 et baseline (rappel chapitre 2, §6)

Un R^2 de 0 correspond exactement à la performance d'une baseline qui prédirait toujours $\bar{y}$ (la moyenne). Un R^2 négatif signifie que votre modèle fait **pire** que cette baseline triviale — un signal d'alerte à ne jamais ignorer.

4. Diagnostiquer un modèle par ses résidus

```
import matplotlib.pyplot as plt

residus = y_test - y_pred

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

axes[0].scatter(y_pred, residus, alpha=0.6)
axes[0].axhline(0, color="red", linestyle="--")
axes[0].set_xlabel("Prix prédit (MAD)")
axes[0].set_ylabel("Résidu (réel - prédit)")
axes[0].set_title("Résidus vs prédictions")

axes[1].hist(residus, bins=25)
axes[1].set_xlabel("Résidu (MAD)")
axes[1].set_title("Distribution des résidus")

fig.tight_layout()
fig.savefig("outputs/diagnostic_residus.png", dpi=150)
plt.show()
```

Un bon modèle produit des résidus répartis **aléatoirement autour de zéro**, sans motif visible. Un nuage en forme d'entonnoir (variance croissante) ou une courbe systématique indique que le modèle rate quelque chose de structurel — une piste pour ajouter des variables ou transformer certaines d'entre elles (par exemple un logarithme sur `prix_mad`).

5. Régularisation : Ridge et Lasso

Avec de nombreuses variables (surtout après un One-Hot Encoding qui crée plusieurs colonnes), une régression linéaire classique peut sur-apprendre. La **régularisation** ajoute une pénalité sur la taille des coefficients :

$$\text{Ridge : } \text{MSE}(\beta) + \alpha \sum_j \beta_j^2 \qquad \text{Lasso : } \text{MSE}(\beta) + \alpha \sum_j |\beta_j|$$

où $\alpha \geq 0$ contrôle l'intensité de la pénalité — un **hyperparamètre**, à ne pas confondre avec les coefficients β_j , qui restent des paramètres appris.

```
from sklearn.linear_model import Lasso, Ridge

for nom, modele_reg in [("Ridge ( =1.0)", Ridge(alpha=1.0)), ("Lasso ( =1.0)", Lasso(alpha=1.0))]:
    pipeline_test = Pipeline([("pretraitement", pretraitement), ("modele", modele_reg)])
    pipeline_test.fit(X_train, y_train)
    print(f"{nom:20s} → R² test = {pipeline_test.score(X_test, y_test):.3f}")
```

💡 Ridge vs Lasso

Ridge réduit tous les coefficients sans jamais les annuler complètement. Lasso peut ramener certains coefficients **exactement à zéro** — il réalise de fait une sélection de variables automatique, utile quand on soupçonne que certaines colonnes n'apportent rien.

6. Régler α proprement avec GridSearchCV

Au Lab 1, nous avons comparé quelques valeurs de k à la main, dans une boucle. GridSearchCV automatise entièrement cette démarche : il essaie chaque valeur d'hyperparamètre, évalue chacune par validation croisée, et conserve la meilleure — **sans jamais toucher au jeu de test**.

```
from sklearn.model_selection import GridSearchCV

pipeline_ridge = Pipeline([
    ("pretraitement", pretraitement),
    ("modele", Ridge()),
])

grille_parametres = {
    "modele__alpha": [0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 300.0],
}

recherche = GridSearchCV(
    pipeline_ridge,
    param_grid=grille_parametres,
    cv=5,
    scoring="r2",
)

recherche.fit(X_train, y_train)
```

```
print("Meilleur :", recherche.best_params_)
print("Meilleur  $R^2$  (moyenne CV) :", round(recherche.best_score_, 3))
print(" $R^2$  sur le test, avec le meilleur modèle :", round(recherche.score(X_test, y_test), 3))
```

! Comment lire `param_grid`

"modele__alpha" utilise la syntaxe `nom_étape__nom_paramètre` de `scikit-learn` : `modele` est le nom donné à l'étape dans le `Pipeline`, et `alpha` est le paramètre de `Ridge` à faire varier. Cette syntaxe permet de régler des hyperparamètres à l'intérieur d'un pipeline complet, prétraitement inclus.

`recherche` se comporte ensuite exactement comme un modèle entraîné classique : `recherche.predict(...)` et `recherche.score(...)` utilisent automatiquement le meilleur pipeline trouvé.

i Pourquoi une grille plutôt qu'une seule valeur ?

Choisir α à l'œil, ou pire sur le jeu de test, revient à une fuite de données par les hyperparamètres (chapitre 2, §5) : le score obtenu serait artificiellement optimiste et ne refléterait pas la performance réelle attendue en production.

7. Synthèse

i Auto-évaluation

- ☐ Je sais calculer les coefficients d'une régression linéaire simple à la main et retrouver le même résultat avec `scikit-learn`.
- ☐ Je sais construire une régression multiple avec des variables numériques et catégorielles, via un pipeline.
- ☐ Je sais calculer et interpréter MAE, RMSE et R^2 , et dire laquelle privilégier selon le contexte.
- ☐ Je sais lire un graphique de résidus pour diagnostiquer un modèle.
- ☐ Je sais expliquer la différence entre `Ridge` et `Lasso`.
- ☐ Je sais utiliser `GridSearchCV` pour choisir un hyperparamètre par validation croisée, à l'intérieur d'un pipeline complet.

8. Lab Sheet — Projet guidé

! Jeu de données du projet

Ce projet utilise le jeu de données `diabetes` fourni par `scikit-learn` (`from sklearn.datasets import load_diabetes`) : 442 patients, 10 variables physiologiques standardisées, et une cible numérique mesurant la progression de la maladie un an après le diagnostic. C'est un vrai jeu de données médical, utilisé dans de nombreuses publications de référence sur la régression régularisée.

Partie A — Prise en main

1. Chargez `load_diabetes(as_frame=True)`. Combien d'observations et de variables explicatives ? Toutes les variables sont-elles déjà numériques ?
2. Séparez Train (80 %) / Test (20 %) avec un `random_state` fixé.

Partie B — Une première régression

3. Entraînez une `LinearRegression` simple (sans régularisation). Calculez MAE, RMSE et R^2 sur le jeu de test.
4. Affichez les coefficients appris (`.coef_`) associés à chaque variable. Quelle variable semble avoir le plus d'impact (en valeur absolue) ?
5. Tracez le graphique résidus vs prédictions (§4). Observez-vous un motif particulier ?

Partie C — Régularisation et réglage d'hyperparamètre

6. Construisez une grille de valeurs de α pour **Ridge** (au moins 6 valeurs, sur plusieurs ordres de grandeur, par exemple de 0.01 à 100).
7. Utilisez `GridSearchCV` (5 plis) pour trouver le meilleur α . Donnez la valeur retenue et le R^2 moyen obtenu en validation croisée.
8. Comparez le R^2 sur le jeu de test entre la régression linéaire simple (question 3) et le meilleur **Ridge** trouvé. La régularisation a-t-elle amélioré la généralisation ? Formulez une explication.
9. (*Bonus*) Refaites les questions 6-7 avec **Lasso**. Combien de coefficients sont ramenés exactement à zéro avec le meilleur α trouvé ? Quelles variables sont éliminées ?

Livrables

- un notebook ou script contenant le code des parties A à C ;
- au moins un graphique de diagnostic dans `outputs/` ;
- un court paragraphe comparant les modèles testés et justifiant le choix final.

💡 Checklist finale

- ☐ Le split Train/Test est effectué avant tout calcul de statistique.
- ☐ Au moins trois métriques de régression sont calculées et interprétées.
- ☐ L'hyperparamètre α est choisi par validation croisée, jamais sur le test.
- ☐ Le jeu de test n'est utilisé qu'une seule fois, pour l'évaluation finale.

Lab 4 — Classification : prédire une catégorie

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Entraîner** une régression logistique, un KNN et un arbre de décision sur une même tâche de classification ;
- **Calculer et interpréter** accuracy, précision, rappel, F1-score, ROC-AUC ;
- **Tracer et lire** une courbe ROC et une courbe précision-rappel ;
- **Ajuster le seuil de décision** d'un modèle et en observer l'effet sur précision et rappel ;
- **Gérer** un déséquilibre de classes avec `class_weight` et `stratify` ;
- **Réutiliser** `GridSearchCV` pour régler l'hyperparamètre d'un modèle de classification.

i Lien avec le cours

Ce Lab correspond au chapitre 5 (*Modèles de classification*). Il réutilise directement les acquis des Labs précédents : matrice de confusion (Lab 1, chapitre 2), pipeline de prétraitement (Lab 2, chapitre 3) et `GridSearchCV` (Lab 3, chapitre 4). Le fil conducteur est l'exemple de la **résiliation d'abonnement** (*churn*) introduit au chapitre 1 (§1.2) comme second exemple du triplet (T, P, E).

1. Construire un jeu de données de *churn*

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(3)
n = 500

anciennete_mois = rng.integers(1, 72, size=n)
appels_service_client = rng.poisson(1.5, size=n)
facture_mensuelle = rng.uniform(80, 600, size=n)
type_contrat = rng.choice(["Mensuel", "Annuel", "Deux ans"], size=n, p=[0.5, 0.3, 0.2])

# Probabilité de résiliation : augmente avec les appels au service client et
# les contrats mensuels, diminue avec l'ancienneté - une structure réaliste.
logit = (
    -2.0
    + 0.35 * appels_service_client
    - 0.03 * anciennete_mois
    + 0.004 * facture_mensuelle
    + pd.Series(type_contrat).map({"Mensuel": 1.2, "Annuel": -0.3, "Deux ans": -1.0}).to_numpy()
)
probabilite_churn = 1 / (1 + np.exp(-logit))
```

```

churn = rng.binomial(1, probabilite_churn)

clients = pd.DataFrame({
    "anciennete_mois": anciennete_mois,
    "appels_service_client": appels_service_client,
    "facture_mensuelle": facture_mensuelle.round(2),
    "type_contrat": type_contrat,
    "churn": churn,
})
print(clients["churn"].value_counts(normalize=True).round(3))

```

Comme souvent en pratique, les classes sont **déséquilibrées** : gardez ce chiffre en tête, il conditionne toute la suite (chapitre 2, §6, « le piège des classes déséquilibrées »).

2. Un pipeline de classification, de bout en bout

```

from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = clients.drop(columns="churn")
y = clients["churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y,
)

colonnes_numeriques = ["anciennete_mois", "appels_service_client", "facture_mensuelle"]
colonnes_categorielles = ["type_contrat"]

pretraitement = ColumnTransformer([
    ("num", StandardScaler(), colonnes_numeriques),
    ("cat", OneHotEncoder(drop="first"), colonnes_categorielles),
])

pipeline_logit = Pipeline([
    ("pretraitement", pretraitement),
    ("modele", LogisticRegression()),
])

pipeline_logit.fit(X_train, y_train)
print("Accuracy test :", round(pipeline_logit.score(X_test, y_test), 3))

```

3. Au-delà de l'accuracy : précision, rappel, F1, ROC-AUC

```
from sklearn.metrics import (
    ConfusionMatrixDisplay, RocCurveDisplay, classification_report,
    confusion_matrix, roc_auc_score,
)

y_pred = pipeline_logit.predict(X_test)
y_proba = pipeline_logit.predict_proba(X_test)[:, 1] # probabilité de la classe "churn"

print(classification_report(y_test, y_pred, target_names=["reste", "churn"]))
print("ROC-AUC :", round(roc_auc_score(y_test, y_proba), 3))
```

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred), display_labels=["reste", "churn"],
).plot(ax=axes[0], cmap="Blues")
axes[0].set_title("Matrice de confusion")

RocCurveDisplay.from_predictions(y_test, y_proba, ax=axes[1])
axes[1].set_title("Courbe ROC")

fig.tight_layout()
fig.savefig("outputs/evaluation_churn.png", dpi=150)
plt.show()
```

i Lire la courbe ROC

La courbe ROC (*Receiver Operating Characteristic*) trace le taux de vrais positifs contre le taux de faux positifs, pour **tous les seuils de décision possibles**. Un modèle inutile (équivalent à un tirage au hasard) suit la diagonale ; plus la courbe s'en écarte vers le coin supérieur gauche, meilleur est le modèle. L'aire sous la courbe (**AUC**) résume cette qualité en un seul nombre entre 0,5 (hasard) et 1 (parfait).

4. Le seuil de décision n'est pas obligatoirement 0,5

Par défaut, `.predict()` classe en « churn » toute observation dont la probabilité prédite dépasse 0,5. Mais rien n'impose ce seuil : selon le coût métier d'un faux négatif (un client qui part sans qu'on ait rien tenté) par rapport à un faux positif (une offre de rétention envoyée à tort), on peut vouloir un seuil différent.

```
from sklearn.metrics import precision_score, recall_score

for seuil in [0.3, 0.5, 0.7]:
    y_pred_seuil = (y_proba >= seuil).astype(int)
    precision = precision_score(y_test, y_pred_seuil)
    rappel = recall_score(y_test, y_pred_seuil)
    print(f"Seuil = {seuil} → précision = {precision:.3f}, rappel = {rappel:.3f}")
```

Abaissier le seuil **augmente le rappel** (on détecte plus de vrais churners) mais **diminue généralement la précision** (on se trompe aussi plus souvent sur des clients qui n'allaient pas partir). Ce compromis, illustré concrètement ici, est au cœur de toute décision de déploiement d'un classifieur.

5. Gérer le déséquilibre des classes

```
pipeline_pondere = Pipeline([
    ("pretraitement", pretraitement),
    ("modele", LogisticRegression(class_weight="balanced")),
])
pipeline_pondere.fit(X_train, y_train)

y_pred_pondere = pipeline_pondere.predict(X_test)
print(classification_report(y_test, y_pred_pondere, target_names=["reste", "churn"]))
```

`class_weight="balanced"` donne automatiquement plus de poids à la classe minoritaire pendant l'entraînement, sans avoir à dupliquer ou supprimer des observations. Comparez le rappel de la classe « churn » avec et sans cette option : c'est généralement là que l'écart est le plus visible.

6. Comparer plusieurs algorithmes, avec réglage d'hyperparamètres

Réutilisons `GridSearchCV`, déjà maîtrisé au Lab 3, pour comparer trois familles de modèles sur la même tâche.

```
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

modeles_a_comparer = {
    "Régression logistique": (
        LogisticRegression(class_weight="balanced"),
        {"modele__C": [0.01, 0.1, 1, 10]},
    ),
    "KNN": (
        KNeighborsClassifier(),
        {"modele__n_neighbors": [3, 5, 9, 15, 25]},
    ),
    "Arbre de décision": (
        DecisionTreeClassifier(class_weight="balanced", random_state=42),
        {"modele__max_depth": [2, 3, 4, 5, 8, None]},
    ),
}

for nom, (estimateur, grille) in modeles_a_comparer.items():
    pipeline_candidat = Pipeline([("pretraitement", pretraitement), ("modele", estimateur)])
    recherche = GridSearchCV(pipeline_candidat, param_grid=grille, cv=5, scoring="f1")
    recherche.fit(X_train, y_train)
    score_test = recherche.score(X_test, y_test)
    print(f"{nom:25s} → meilleurs paramètres = {recherche.best_params_}, F1 (test) = {score_test:.3f}")
```

💡 Pourquoi `scoring="f1"` plutôt que l'accuracy ?

Sur un jeu déséquilibré, optimiser l'accuracy par validation croisée pousserait naturellement `GridSearchCV` vers des modèles qui prédisent presque toujours la classe majoritaire (chapitre 2, §6). Le F1-score, qui combine précision et rappel, est un choix bien plus pertinent ici.

7. Synthèse

i Auto-évaluation

- ☐ Je sais construire un pipeline de classification complet, prétraitement inclus.
- ☐ Je sais calculer et interpréter précision, rappel, F1-score et ROC-AUC.
- ☐ Je sais expliquer ce que montre une courbe ROC et à quoi correspond l'AUC.
- ☐ Je sais faire varier le seuil de décision et en expliquer l'effet sur précision et rappel.
- ☐ Je sais utiliser `class_weight="balanced"` et justifier son intérêt sur des classes déséquilibrées.
- ☐ Je sais utiliser `GridSearchCV` avec `scoring="f1"` pour comparer plusieurs familles de modèles.

8. Lab Sheet — Projet guidé

! Jeu de données du projet : le Titanic, à nouveau

Reprenez le jeu de données du Titanic préparé au Lab 2 (§9). Cette fois, l'objectif n'est plus seulement de le nettoyer, mais de réellement **prédire** la survie (`survived`) d'un passager.

```
titanic = pd.read_csv(
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
)
```

Partie A — Reprendre le pipeline du Lab 2

1. Réappliquez (ou reconstruisez) le pipeline de prétraitement du Lab 2 sur les variables `pclass`, `sex`, `age`, `fare`, `embarked`, `sibsp`, `parch`, avec `survived` comme cible. Séparez Train (75 %) / Test (25 %), stratifiés.
2. Le jeu est-il déséquilibré ? Donnez la proportion de survivants.

Partie B — Premier modèle et métriques

3. Entraînez une `LogisticRegression` (pipeline complet). Calculez accuracy, précision, rappel, F1-score et ROC-AUC sur le test.
4. Tracez la matrice de confusion et la courbe ROC.
5. Une `DummyClassifier` (baseline) obtiendrait quelle accuracy sur ce jeu ? Votre modèle fait-il significativement mieux ?

Partie C — Seuil et déséquilibre

6. Testez trois seuils de décision différents (par exemple 0.3, 0.5, 0.7). Lequel choisiriez-vous si l'objectif prioritaire était de **ne rater aucun survivant potentiel** (maximiser le rappel), quitte à avoir plus de fausses alertes ?
7. Comparez, avec et sans `class_weight="balanced"`, le rappel obtenu pour la classe minoritaire.

Partie D — Comparaison de modèles et hyperparamètres

8. Comparez, par `GridSearchCV` avec `scoring="f1"`, une régression logistique (hyperparamètre `C`), un KNN (hyperparamètre `n_neighbors`) et un arbre de décision (hyperparamètre `max_depth`).
9. Quel modèle obtient le meilleur F1-score sur le jeu de test ? Ce résultat était-il celui que vous attendiez, compte tenu des données (variables catégorielles nombreuses, taille du jeu, etc.) ?
10. (*Bonus*) Affichez l'importance des variables pour votre meilleur modèle (`.coef_` pour la régression logistique, ou `.feature_importances_` pour l'arbre). Quelle variable semble la plus déterminante pour la survie ? Ce résultat correspond-il à ce que vous savez de l'histoire du naufrage ?

Livrables

- un notebook ou script contenant le code des parties A à D ;
- au moins deux graphiques (matrice de confusion, courbe ROC) dans `outputs/` ;
- un court paragraphe de conclusion, comparant les modèles testés et justifiant votre choix final.

💡 Checklist finale

- ☐ Le déséquilibre des classes est mesuré avant toute modélisation.
- ☐ Au moins trois métriques de classification (au-delà de l'accuracy) sont calculées.
- ☐ L'effet du seuil de décision est illustré avec au moins deux valeurs différentes.
- ☐ Tous les hyperparamètres sont réglés par validation croisée sur le train, jamais sur le test.
- ☐ Le jeu de test n'est utilisé qu'une seule fois, pour l'évaluation finale.